1. Java Fundamentals [40 Questions]

Q: 1. What is the JVM, JRE, and JDK?

A: JDK = JRE + compiler/tools. JRE = JVM + core libraries. JVM = runtime that executes bytecode. You develop with JDK, distribute with JRE, and code runs on JVM.

Q: 2. What is bytecode and why is Java platform-independent?

A: Java source is compiled to bytecode (.class files), not machine code. The JVM on each OS interprets/JIT-compiles bytecode to native instructions. Write once, run anywhere.

Q: 3. What are primitive types in Java?

A: byte(1B), short(2B), int(4B), long(8B), float(4B), double(8B), char(2B), boolean. They are not objects — stored on stack, not heap.

Q: 4. What is autoboxing and unboxing?

A: Autoboxing: automatic conversion of primitive to wrapper (int -> Integer). Unboxing: wrapper to primitive (Integer -> int). Be careful: unboxing null Integer throws NullPointerException.

Q: 5. What is the difference between == and .equals()?

A: == compares object references (memory address) for objects, and values for primitives. .equals() compares logical content. For String: 'hello' == 'hello' may be true (string pool), but new String('hello') == new String('hello') is false.

Q: 6. What is String immutability and why is it important?

A: Strings cannot be modified after creation. Any operation returns a new String. Benefits: thread safety, safe HashMap key, string pool caching, security (passwords, class names).

Q: 7. What is the String pool (intern pool)?

A: A JVM cache of String literals. String s = 'hello' reuses pool entry if it exists. String s = new String('hello') creates a new object (not pooled). s.intern() adds to pool.

Q: 8. What is StringBuilder vs StringBuffer?

A: Both are mutable string sequences. StringBuffer is synchronized (thread-safe, slower). StringBuilder is not synchronized (faster, use in single-threaded contexts). Prefer StringBuilder.

Q: 9. What are static members?

A: Static fields/methods belong to the class, not instances. Shared across all objects. Accessed via ClassName.member. Static methods cannot access instance fields or this.

Q: 10. What is the final keyword?

A: final variable: cannot be reassigned. final method: cannot be overridden. final class: cannot be subclassed (e.g., String, Integer).

Q: 11. What is the difference between final, finally, and finalize?

A: final: keyword for immutability/no override. finally: block in try-catch that always executes. finalize(): deprecated Object method called before GC (don't use it).

Q: 12. What is a varargs method?

A: Variable arguments: void method(String... args). args is treated as array inside method. Must be last parameter. Caller can pass 0 or more args: method(), method('a'), method('a','b').

Q: 13. What is the difference between checked and unchecked exceptions?

A: Checked exceptions (IOException, SQLException) must be declared or caught — compiler enforces. Unchecked exceptions (RuntimeException subclasses) need not be declared. Errors (OutOfMemoryError) are fatal and not caught.

Q: 14. What are Java access modifiers?

A: private: same class only. (default/package): same package. protected: same package + subclasses. public: everywhere.

Q: 15. What is the difference between an abstract class and an interface?

A: Abstract class: can have state (fields), constructors, concrete methods, access modifiers. Interface: all fields are public static final, methods default to public abstract (Java 8 adds default/static methods). Class can implement multiple interfaces but extend only one class.

Q: 16. What is a constructor?

A: Special method called when creating an object. Same name as class, no return type. Java provides default no-arg constructor if none defined. Constructors can be chained via this() or super().

Q: 17. What is constructor chaining?

A: Calling one constructor from another. this(args) calls another constructor in same class. super(args) calls parent constructor. Must be first statement.

Q: 18. What is the difference between method overloading and overriding?

A: Overloading: same class, same method name, different parameter list. Resolved at compile time (static polymorphism). Overriding: subclass redefines parent method with same signature. Resolved at runtime (dynamic polymorphism).

Q: 19. What is the instanceof operator?

A: Checks if an object is an instance of a class/interface: obj instanceof String. Returns true if obj is String or a subclass. Also can be used with pattern matching in Java 16+: if (obj instanceof String s) { s.toUpperCase(); }.

Q: 20. What is a static initializer block?

A: static { ... } block runs once when class is loaded, before any instance is created. Used to initialize static fields with complex logic.

Q: 21. What is object cloning?

A: Creating a copy of an object. Implement Cloneable and override clone(). Shallow copy: copies references (default). Deep copy: copies referenced objects too. Prefer copy constructors or serialization over clone().

Q: 22. What is the Comparable interface?

A: java.lang.Comparable with compareTo(T other). Defines natural ordering. Implemented by the class itself. Used by Collections.sort(), TreeMap, etc.

Q: 23. What is the Comparator interface?

A: java.util.Comparator with compare(T a, T b). External ordering strategy — separate from the class. Allows multiple orderings. Java 8: Comparator.comparing(Person::getName).thenComparing(Person::getAge).

Q: 24. What is Java reflection?

A: Inspecting/modifying class structure at runtime: Class.forName(), getDeclaredFields(), getDeclaredMethods(), invoke(). Used by frameworks (Spring, Hibernate). Slower than direct calls; bypasses access control.

Q: 25. What is the transient keyword?

A: Marks a field to be excluded from serialization. When object is serialized (ObjectOutputStream), transient fields are skipped and will be null/default on deserialization.

Q: 26. What is the volatile keyword?

A: Ensures field is read/written directly to main memory, not CPU cache. All threads see the latest value. Guarantees visibility but NOT atomicity (use AtomicInteger for compound operations like i++).

Q: 27. What is the difference between Serializable and Externalizable?

A: Serializable: marker interface; JVM handles serialization automatically. Externalizable: extends Serializable; you implement readExternal/writeExternal for full control over serialization format.

Q: 28. What is a functional interface?

A: Interface with exactly one abstract method. @FunctionalInterface annotation (optional, enforced by compiler). Examples: Runnable, Callable, Comparator, Predicate, Function, Consumer, Supplier.

Q: 29. What is a lambda expression?

A: Concise way to implement a functional interface: (a, b) -> a + b. Captures variables from enclosing scope (must be effectively final). Syntax: (params) -> expression or (params) -> { statements; return val; }.

Q: 30. What is a method reference?

A: Shorthand for lambdas that call a method. Types: ClassName::staticMethod, instance::method, ClassName::instanceMethod (applies first arg as receiver), ClassName::new (constructor). Example: list.forEach(System.out::println).

Q: 31. What are Java 8 default interface methods?

A: Methods in interfaces with a body: default void method() { ... }. Allows adding methods to interfaces without breaking implementations. Classes can override. Multiple default methods from different interfaces: must override to resolve conflict.

Q: 32. What is Optional in Java 8?

A: Container that may or may not hold a non-null value. Avoids NullPointerException. Create: Optional.of(val), Optional.ofNullable(val), Optional.empty(). Use: orElse(), orElseGet(), map(), filter(), ifPresent(). Don't overuse as field type — designed for return types.

Q: 33. What is try-with-resources?

A: Automatically closes resources after try block: try (InputStream is = new FileInputStream(f)) { ... }. Resource must implement AutoCloseable. Closes even if exception thrown. Replaces finally { is.close(); }.

Q: 34. What are generics?

A: Type parameters for classes/methods: List, Map. Provide compile-time type safety without casting. Type erased at runtime (type erasure). Wildcards: ? extends T (upper bound), ? super T (lower bound).

Q: 35. What is PECS (Producer Extends Consumer Super)?

A: Guideline for generics wildcards. Use extends when reading (producing): List. Use super when writing (consuming): List. Read-only: extends. Write-only: super.

Q: 36. What is the difference between List and List?

A: List accepts only List. List (wildcard) accepts List, List, any List. But List is read-only — you can only add null to it, since the type is unknown.

Q: 37. What is the difference between int[] and Integer[]?

A: int[] is an array of primitive ints (stack-stored values). Integer[] is an array of Integer objects (heap references). int[] cannot be used with generics; Integer[] can. Autoboxing happens when assigning between them.

Q: 38. What is a record in Java 16?

A: Immutable data class: record Point(int x, int y) {}. Auto-generates: constructor, getters (x(), y()), equals(), hashCode(), toString(). Cannot have mutable state or extend other classes. Great for DTOs.

Q: 39. What is a sealed class in Java 17?

A: Restricts which classes can extend/implement it: sealed interface Shape permits Circle, Rectangle {}. Only listed classes can be subtypes. Enables exhaustive switch patterns without default case.

Q: 40. What is pattern matching for switch (Java 21)?

A: switch (obj) { case Integer i -> 'int: '+i; case String s -> 'str: '+s; case null -> 'null'; default -> 'other'; }. Type-tests and casts in one step. Compiler checks exhaustiveness for sealed types.

2. OOP & SOLID Principles [40 Questions]

Q: 1. What are the four pillars of OOP?

A: Encapsulation (hiding state), Inheritance (reusing via parent-child), Polymorphism (one interface, many implementations), Abstraction (hiding complexity behind interfaces).

Q: 2. What is encapsulation?

A: Bundling data and methods that operate on it, hiding internal state behind a public API (private fields, public getters/setters). Protects invariants and allows internal changes without breaking callers.

Q: 3. What is inheritance?

A: A class extends another class, inheriting its fields and methods. Child can override parent methods. Java supports single class inheritance (one parent) but multiple interface implementation.

Q: 4. What is polymorphism?

A: An object can be referenced by its type or any supertype. The actual method called is determined at runtime (dynamic dispatch). Example: Animal a = new Dog(); a.speak() calls Dog.speak() not Animal.speak().

Q: 5. What is abstraction?

A: Exposing only essential features, hiding implementation details. Achieved via abstract classes and interfaces. Callers depend on the abstraction, not the concrete implementation.

Q: 6. What is the difference between is-a and has-a relationships?

A: is-a = inheritance: Dog is-a Animal. has-a = composition: Car has-a Engine. Prefer composition over inheritance (more flexible, less coupling).

Q: 7. What is method hiding vs method overriding?

A: Overriding: instance methods in subclass with same signature. Dynamic dispatch chooses subclass version. Hiding: static methods with same signature in subclass. Static dispatch — determined by reference type at compile time.

Q: 8. What is covariant return type?

A: Overriding method can return a subtype of the parent's return type. class Animal { Animal get() {} } class Dog extends Animal { Dog get() {} } — valid in Java 5+.

Q: 9. What is the Liskov Substitution Principle (L in SOLID)?

A: Objects of a subclass must be usable wherever the parent class is expected, without breaking correctness. Subclasses should not narrow preconditions or broaden postconditions. Violated by Square-extends-Rectangle (setWidth breaks square invariant).

Q: 10. What is the Single Responsibility Principle?

A: A class should have only one reason to change — one responsibility. A UserService shouldn't also do email sending and DB logging. Split into UserService, EmailService, AuditLogger.

Q: 11. What is the Open/Closed Principle?

A: Classes should be open for extension, closed for modification. Add new behavior by extending (new subclass, new strategy) rather than editing existing code. Reduces regression risk.

Q: 12. What is the Interface Segregation Principle?

A: Prefer many specific interfaces over one large interface. Don't force classes to implement methods they don't need. Instead of one big Worker interface, have Workable and Eatable separately.

Q: 13. What is the Dependency Inversion Principle?

A: High-level modules should not depend on low-level modules; both should depend on abstractions. Inject dependencies via interfaces, not concrete classes. Enables testability and loose coupling.

Q: 14. What is composition over inheritance?

A: Favor assembling behavior by composing objects (has-a) rather than extending classes (is-a). More flexible: change behavior at runtime by swapping components. Avoids fragile base class problem.

Q: 15. What is the fragile base class problem?

A: When a subclass breaks because the parent class changed its implementation (not its interface). Internal parent changes ripple down unexpectedly. Composition avoids this since components have no inheritance relationship.

Q: 16. What is multiple inheritance and how does Java handle the diamond problem?

A: Java doesn't allow inheriting from multiple classes. For interfaces: if two interfaces have same default method, implementing class must override it to resolve the diamond. Java makes this explicit — compile error if not resolved.

Q: 17. What is an abstract class?

A: Class declared abstract — cannot be instantiated. May have abstract methods (no body) that subclasses must implement. Can also have concrete methods, constructors, and state.

Q: 18. What is a marker interface?

A: Interface with no methods: Serializable, Cloneable. Signals a capability to JVM or frameworks. Modern alternative: annotations (@Serializable). Marker interfaces still used for instanceof checks.

Q: 19. What is object composition?

A: Building complex objects by combining simpler ones. class Car { private Engine engine; private Wheels wheels; }. Car delegates engine start to engine.start(). Behavior can be changed at runtime by swapping implementations.

Q: 20. What is the difference between an interface and an abstract class in Java 8+?

A: Abstract class: can have constructors, state, access modifiers. Interface: now has default/static methods, but no state (no instance fields). Use abstract class for shared state; use interface for defining contracts.

Q: 21. What is method overriding vs hiding?

A: Overriding (instance methods): runtime polymorphism — subclass method called based on object type. Hiding (static methods): compile-time — method called based on reference type.

Q: 22. What is super() and when is it needed?

A: super() calls the parent constructor. Must be first statement. If parent has no default constructor, child must explicitly call super(param). super.method() calls parent's version of an overridden method.

Q: 23. What is the this keyword?

A: Refers to current object instance. Used to disambiguate fields from parameters (this.name = name), to call another constructor (this()), and to pass current object as argument.

Q: 24. What are access modifiers and their scope?

A: private: class only. package-private (default): same package. protected: package + subclasses. public: everywhere. Choose the most restrictive that works.

Q: 25. What is the difference between overloading and overriding?

A: Overloading: same name, different params, same class, compile-time. Overriding: same name, same params, subclass, runtime. @Override annotation catches mistakes.

Q: 26. What is dynamic dispatch?

A: The mechanism that determines which overriding method to call at runtime based on the actual object type, not the reference type. Enables runtime polymorphism.

Q: 27. What is the template method design pattern?

A: Abstract class defines algorithm skeleton with abstract steps. Subclasses implement the steps. The overall algorithm (template) stays in the parent. Hollywood principle: don't call us, we'll call you.

Q: 28. What is coupling vs cohesion?

A: Coupling: degree of dependency between modules (low is better). Cohesion: degree to which elements inside a module belong together (high is better). Aim for low coupling, high cohesion.

Q: 29. What is the Law of Demeter?

A: A module should only talk to its immediate dependencies, not to the dependencies of its dependencies. Example: avoid a.getB().getC().doSomething() — chains expose internal structure.

Q: 30. What is encapsulation violation?

A: Returning mutable objects from getters: getList() returns the internal list; caller modifies it, breaking invariants. Fix: return Collections.unmodifiableList() or a copy.

Q: 31. What is an immutable class?

A: State cannot be changed after construction. Rules: final class, final fields, no setters, defensive copy of mutable parameters in constructor and getters. Examples: String, Integer, LocalDate.

Q: 32. What is the builder pattern?

A: Creates complex objects step by step. Fluent API: Person.builder().name('Alice').age(30).build(). Avoids telescoping constructors with many optional parameters. Immutable object produced at the end.

Q: 33. What is the singleton pattern?

A: Ensures only one instance of a class exists. Best implementation: enum Singleton { INSTANCE; }. Also: private constructor + static instance + getInstance(). Be careful with multithreading — use double-checked locking or enum.

Q: 34. What is the factory method pattern?

A: Define an interface for creating objects; let subclasses decide which class to instantiate. Decouples object creation from usage. ProductFactory.create('PDF') returns PdfProduct without caller knowing the class.

Q: 35. What is the strategy pattern?

A: Define a family of algorithms, encapsulate each, make them interchangeable. Injected at runtime. Example: SortContext holds a SortStrategy; set to BubbleSort or QuickSort at runtime. Avoids long if-else chains.

Q: 36. What is the observer pattern?

A: Subject maintains a list of observers and notifies them on state change. Publisher-subscriber. Listeners registered via addObserver(). Java: PropertyChangeListener, reactive streams (Reactor/RxJava).

Q: 37. What is the decorator pattern?

A: Adds behavior to objects dynamically by wrapping them. class LoggingService implements Service { private Service delegate; }. Layers can be stacked. Java IO streams use this: BufferedReader(new FileReader(file)).

Q: 38. What is the proxy pattern?

A: Provides a surrogate for another object. Types: virtual (lazy init), protection (access control), remote (network), logging/caching (AOP). Spring AOP creates proxies for @Transactional, @Cacheable.

Q: 39. What is the adapter pattern?

A: Converts one interface to another. TargetInterface adapter = new Adapter(adaptee). Used to integrate legacy code or third-party libraries without modifying them.

Q: 40. What is the facade pattern?

A: Single simplified interface over a complex subsystem. OrderFacade.placeOrder() internally calls InventoryService, PaymentService, NotificationService. Client doesn't know the complexity.

3. Java Collections Framework [35 Questions]

Q: 1. What is the Java Collections hierarchy?

A: Iterable -> Collection -> List (ArrayList, LinkedList, Vector), Set (HashSet, LinkedHashSet, TreeSet), Queue (PriorityQueue, ArrayDeque). Map (HashMap, LinkedHashMap, TreeMap, Hashtable) is separate from Collection hierarchy.

Q: 2. What is the difference between ArrayList and LinkedList?

A: ArrayList: array-backed, O(1) get by index, O(n) insert/delete in middle. LinkedList: doubly-linked, O(n) get by index, O(1) insert/delete at known position. Use ArrayList for most cases; LinkedList for frequent head/tail operations.

Q: 3. What is the difference between HashSet, LinkedHashSet, and TreeSet?

A: HashSet: no order, O(1) operations. LinkedHashSet: insertion order maintained, O(1). TreeSet: sorted (natural or Comparator), O(log n). All backed by their corresponding Map implementations.

Q: 4. How does HashSet work internally?

A: HashSet is backed by a HashMap. Elements are stored as keys with a dummy value (PRESENT). All HashMap semantics apply: uses hashCode() + equals() for uniqueness, no guaranteed order.

Q: 5. What is the difference between HashMap and TreeMap?

A: HashMap: O(1) avg, no order, uses hashCode/equals. TreeMap: O(log n), sorted by keys (natural order or Comparator), uses compareTo/Comparator. TreeMap supports subMap(), headMap(), tailMap(), floorKey(), ceilingKey().

Q: 6. What is a PriorityQueue?

A: A heap-based queue where elements are ordered by natural order or Comparator. peek()/poll() returns minimum (min-heap by default). Not sorted — only guarantees head is minimum. O(log n) insert/remove, O(1) peek.

Q: 7. What is ArrayDeque and when to use it?

A: Double-ended queue backed by resizable array. Use as stack (push/pop) or queue (offer/poll). Faster than Stack (which extends Vector). No null elements. Better than LinkedList for stack/queue use cases.

Q: 8. What is fail-fast vs fail-safe iteration?

A: Fail-fast: throws ConcurrentModificationException if collection is modified during iteration (ArrayList, HashMap). Fail-safe: iterates over a snapshot copy, no exception (CopyOnWriteArrayList, ConcurrentHashMap).

Q: 9. What is CopyOnWriteArrayList?

A: Thread-safe ArrayList variant. On write (add/remove), creates a new array copy. Reads are lock-free (always see consistent snapshot). Use when reads heavily outnumber writes (e.g., event listener lists).

Q: 10. What is ConcurrentHashMap?

A: Thread-safe HashMap. Java 8+: bucket-level CAS for empty slots, synchronized for non-empty. Reads are fully lock-free. Better concurrency than Hashtable or Collections.synchronizedMap(). No null keys/values.

Q: 11. What is the difference between poll() and remove() in Queue?

A: poll(): removes and returns head; returns null if queue is empty. remove(): removes and returns head; throws NoSuchElementException if empty. Similarly: peek() vs element() for viewing head.

Q: 12. What is a Deque?

A: Double-Ended Queue: supports add/remove at both ends. addFirst/addLast, pollFirst/pollLast, peekFirst/peekLast. ArrayDeque is the main implementation. Can be used as both stack and queue.

Q: 13. What is Collections.unmodifiableList()?

A: Returns a view of the list that throws UnsupportedOperationException on mutating operations. The underlying list can still be modified by original reference. For truly immutable: List.of() (Java 9+).

Q: 14. What is List.of() vs Arrays.asList()?

A: List.of(): immutable — no null elements, no add/remove/set. Arrays.asList(): fixed size (no add/remove) but allows set(). Both backed by arrays. List.copyOf() creates an independent copy.

Q: 15. What is Collections.sort() vs List.sort()?

A: Both sort in-place using merge sort (TimSort). Collections.sort(list) delegates to list.sort(). List.sort(comparator) is the recommended Java 8 way. Both require Comparable or a Comparator.

Q: 16. What is the time complexity of HashMap operations?

A: Average: O(1) for put/get/remove. Worst case (all keys in same bucket): O(n) Java 7, O(log n) Java 8+ (tree bucket).

Q: 17. What is the time complexity of TreeMap operations?

A: O(log n) for put/get/remove/containsKey. Based on Red-Black Tree. O(n) for iteration (in-order traversal).

Q: 18. What is the time complexity of ArrayList vs LinkedList?

A: ArrayList: get O(1), add end O(1) amortized, add middle O(n), remove middle O(n). LinkedList: get O(n), add/remove at head/tail O(1), add/remove middle O(n) (finding node) + O(1) (pointer update).

Q: 19. What is a BlockingQueue?

A: Queue that blocks on put() when full and on take() when empty. Used in producer-consumer. Implementations: ArrayBlockingQueue (bounded), LinkedBlockingQueue (optionally bounded), PriorityBlockingQueue, SynchronousQueue (no storage, handoff).

Q: 20. What is SynchronousQueue?

A: A queue with no internal storage. put() blocks until take() is called, and vice versa. Used for direct handoff between producer and consumer threads. Used by Executors.newCachedThreadPool().

Q: 21. What is EnumMap?

A: Map implementation where keys must be enum values. Internally uses an array indexed by enum ordinal — very fast (O(1)) and memory-efficient. Maintains enum declaration order.

Q: 22. What is WeakHashMap?

A: Map where keys are held by weak references. If a key has no strong references elsewhere, GC can collect it and the entry is automatically removed. Used for caches (e.g., metadata caches keyed by object).

Q: 23. What is IdentityHashMap?

A: Uses reference equality (==) for key comparison instead of equals(). Intentional violation of Map contract. Used for special cases like object graph traversal to track visited objects.

Q: 24. What is the difference between Iterator and ListIterator?

A: Iterator: forward-only, works for all collections. ListIterator: extends Iterator, supports backward traversal (hasPrevious/previous), index access, and set/add during iteration. Only for Lists.

Q: 25. What is the Spliterator?

A: Java 8 interface for parallel iteration. Used by Stream.parallel() to split collection into chunks. Supports tryAdvance() and trySplit(). All collections provide spliterators.

Q: 26. What happens if you modify a collection while iterating with for-each?

A: Throws ConcurrentModificationException (fail-fast). Fix: use Iterator.remove() during iteration, or collect changes and apply after loop, or use removeIf() (Java 8), or use CopyOnWriteArrayList.

Q: 27. What is Collections.frequency()?

A: Returns the count of a specified element in a collection. Uses equals() for comparison. O(n) scan.

Q: 28. What is Collections.disjoint()?

A: Returns true if two collections have no elements in common. Uses equals(). O(n*m) in worst case.

Q: 29. What is Collections.nCopies()?

A: Returns an immutable List of n copies of a specified object. All elements are the same reference. Useful for initializing lists to a default value.

Q: 30. What is PriorityQueue's natural ordering vs Comparator?

A: Default: min-heap using natural ordering (Comparable). Custom: pass Comparator to constructor. new PriorityQueue<>(Comparator.reverseOrder()) creates a max-heap.

Q: 31. What is the difference between Comparable and Comparator?

A: Comparable: implemented by the class itself, defines natural ordering (compareTo). Comparator: external, defines alternative ordering (compare). One natural order per class; unlimited Comparators.

Q: 32. What is stream() vs parallelStream()?

A: stream() creates sequential pipeline. parallelStream() splits work across ForkJoinPool threads. Use parallel when: large dataset, CPU-bound operations, operations are stateless and independent. Parallel has overhead; test before assuming it's faster.

Q: 33. What is collect(Collectors.toList()) vs toUnmodifiableList()?

A: toList() (Java 16+) and toUnmodifiableList() return unmodifiable lists. Collectors.toList() returns a mutable ArrayList. For immutable result: use stream().collect(Collectors.toUnmodifiableList()) or Stream.toList() in Java 16+.

Q: 34. What are the intermediate vs terminal stream operations?

A: Intermediate: lazy, return Stream, chainable (filter, map, flatMap, sorted, distinct, limit, peek). Terminal: eager, trigger pipeline execution, return result (collect, forEach, reduce, count, anyMatch, findFirst).

Q: 35. What is flatMap in streams?

A: Maps each element to a Stream, then flattens all those streams into one. Example: List> -> Stream. flatMap(Collection::stream) flattens nested lists.

4. Java Multithreading & Concurrency [40 Questions]

Q: 1. What is a thread?

A: The smallest unit of execution within a process. Java creates threads via extending Thread, implementing Runnable, or using Callable + ExecutorService. JVM threads map to OS threads.

Q: 2. What are the thread states?

A: NEW, RUNNABLE (running or ready), BLOCKED (waiting for monitor lock), WAITING (indefinitely: Object.wait, Thread.join), TIMED_WAITING (with timeout: sleep, wait(n)), TERMINATED.

Q: 3. What is the difference between process and thread?

A: Process: independent program with its own memory space. Thread: lightweight unit within a process, shares heap and static memory with other threads. Context switch between threads is cheaper than between processes.

Q: 4. What is synchronized keyword?

A: Acquires the intrinsic monitor lock before entering block/method. Only one thread at a time. synchronized(object) for block, synchronized on method (uses 'this'). Provides mutual exclusion and visibility guarantees.

Q: 5. What is the volatile keyword?

A: Ensures field is read/written to main memory, not CPU cache. All threads see the latest write. Does not provide atomicity. Suitable for flags (boolean running) and double-checked locking.

Q: 6. What is the difference between synchronized and volatile?

A: synchronized: mutual exclusion + visibility, one thread at a time. volatile: visibility only, no mutual exclusion. Use volatile for simple flags; synchronized or AtomicXxx for compound operations.

Q: 7. What is a race condition?

A: Two threads access shared data concurrently, and outcome depends on execution order. Example: counter++ is not atomic (read-increment-write); two threads can overwrite each other's writes. Fix: synchronize or use AtomicInteger.

Q: 8. What is a deadlock?

A: Two threads each hold a lock the other needs, both waiting forever. Example: Thread1 holds lockA, wants lockB. Thread2 holds lockB, wants lockA. Fix: always acquire locks in same order, use tryLock() with timeout.

Q: 9. What is a livelock?

A: Threads are active but not making progress — they keep reacting to each other. Example: two people in a corridor both moving to the same side to let the other pass, repeatedly.

Q: 10. What is thread starvation?

A: A thread is perpetually denied CPU time because other higher-priority threads always run. In fair scheduling: use ReentrantLock(true) for fair ordering.

Q: 11. What is ReentrantLock?

A: More flexible than synchronized. Supports: tryLock() (non-blocking), lockInterruptibly() (interruptible wait), fairness mode, multiple Condition variables. Must be unlocked in finally block.

Q: 12. What is ReadWriteLock?

A: Allows multiple concurrent readers OR one exclusive writer. readLock().lock() for reads (shared), writeLock().lock() for writes (exclusive). Better concurrency than synchronized for read-heavy workloads.

Q: 13. What is StampedLock?

A: Java 8 advanced lock. tryOptimisticRead() gets a stamp without blocking. Validate stamp after read — if invalidated (writer wrote), fall back to readLock. 3 modes: write, read, optimistic read.

Q: 14. What is a Condition variable?

A: Associated with a Lock. Allows threads to wait for a condition: condition.await() releases lock and waits. condition.signal()/signalAll() wakes waiting threads. Equivalent to Object.wait()/notifyAll() but for Locks.

Q: 15. What is the Executor framework?

A: Decouples task submission from thread management. ExecutorService manages a thread pool. submit(Callable) returns Future. Executors factory: newFixedThreadPool(n), newCachedThreadPool(), newSingleThreadExecutor(), newScheduledThreadPool(n).

Q: 16. What is ThreadPoolExecutor?

A: Core class for thread pools. Key params: corePoolSize, maximumPoolSize, keepAliveTime, workQueue, rejectedExecutionHandler. If queue full and at max threads: RejectedExecutionException (or custom policy).

Q: 17. What is a Future?

A: Represents result of async computation. future.get() blocks until done. future.isDone(), cancel(). Java 8: CompletableFuture is much more powerful.

Q: 18. What is CompletableFuture?

A: Async pipeline: supplyAsync(() -> compute()).thenApply(x -> transform(x)).thenAccept(x -> print(x)). Chaining, combining (allOf, anyOf), exception handling (exceptionally, handle). Non-blocking.

Q: 19. What is the Fork/Join framework?

A: Divide-and-conquer parallelism. ForkJoinPool uses work-stealing. RecursiveTask (returns result) and RecursiveAction (no result). subtask.fork() submits async; subtask.join() gets result. Used by parallelStream() internally.

Q: 20. What is the happens-before relationship?

A: JMM guarantee that memory writes in one action are visible to another. Examples: synchronized unlock happens-before subsequent lock; volatile write happens-before read; Thread.start() happens-before thread's first action.

Q: 21. What is AtomicInteger?

A: Thread-safe integer using CAS (Compare-And-Swap) hardware instruction. incrementAndGet(), compareAndSet(expected, update). No lock, very fast. Use for counters. AtomicLong, AtomicBoolean, AtomicReference also available.

Q: 22. What is LongAdder vs AtomicLong?

A: LongAdder is faster under high contention — maintains multiple cells, each updated independently; sum() combines them. AtomicLong is faster when contention is low (single CAS). Use LongAdder for high-throughput counters.

Q: 23. What is CountDownLatch?

A: A one-time gate. countDown() decrements counter. await() blocks until counter reaches 0. Cannot be reset. Use case: wait for N services to initialize before starting processing.

Q: 24. What is CyclicBarrier?

A: All parties call await(); they all proceed only when N parties have called await(). Reusable (unlike CountDownLatch). Optional barrier action runs when all arrive. Use case: parallel computation where all phases must sync.

Q: 25. What is Semaphore?

A: Limits concurrent access to a resource. acquire() decrements permits (blocks if 0). release() increments permits. Semaphore(5) allows max 5 concurrent accesses. Use for connection pools, rate limiting.

Q: 26. What is ThreadLocal?

A: Per-thread variable — each thread has its own independent copy. threadLocal.set(val), threadLocal.get(). Used for: request context (userId), database connections, SimpleDateFormat. Must remove() in finally to prevent memory leaks in thread pools.

Q: 27. What is a thread-safe singleton?

A: Best: enum Singleton { INSTANCE; }. Alternative: double-checked locking with volatile: private volatile static Singleton instance; then synchronized block only on first creation.

Q: 28. What is the Phaser?

A: More flexible than CyclicBarrier or CountDownLatch. Supports dynamic party registration and multiple phases. arrive(), awaitAdvance(phase), arriveAndAwaitAdvance(). Use for multi-phase parallel tasks.

Q: 29. What is Exchanger?

A: Two threads exchange objects at a synchronization point. thread1.exchange(objA) blocks until thread2.exchange(objB). Both threads receive the other's object. Use for pipeline buffer filling between stages.

Q: 30. What is wait() and notify()?

A: Object methods. wait() releases the intrinsic lock and sleeps until notify() or notifyAll() is called. Must be called inside synchronized block. notify() wakes one waiting thread; notifyAll() wakes all. Prefer Condition variables for clarity.

Q: 31. What is a blocking method?

A: A method that may pause the calling thread until some condition is met: Thread.sleep(), BlockingQueue.take(), Future.get(), Lock.lock(). Non-blocking alternatives: tryLock(), poll() with timeout.

Q: 32. What is the producer-consumer pattern?

A: Producers add items to a shared queue; consumers take and process them. BlockingQueue handles synchronization: put() blocks if full, take() blocks if empty. Decouples production and consumption rates.

Q: 33. What are virtual threads (Java 21)?

A: Lightweight threads managed by JVM, not OS. Millions can be created cheaply. Thread.ofVirtual().start(runnable). Blocking operations unmount from carrier thread (OS thread), freeing it. Best for I/O-bound workloads. No need for reactive/async programming for concurrency.

Q: 34. What is the difference between concurrency and parallelism?

A: Concurrency: multiple tasks making progress (may interleave on one CPU). Parallelism: multiple tasks executing simultaneously (requires multiple CPUs). Concurrency is about structure; parallelism is about execution.

Q: 35. What is CAS (Compare-And-Swap)?

A: Atomic hardware instruction: if current value == expected, set to new value. Returns whether swap succeeded. Basis of all java.util.concurrent.atomic classes. Lock-free but may spin on contention (busy-wait until CAS succeeds).

Q: 36. What is the ABA problem?

A: In CAS: value A changed to B then back to A. CAS sees A and thinks nothing changed, but state may have changed. Fix: use AtomicStampedReference or AtomicMarkableReference which include a version stamp/mark.

Q: 37. What is lock striping?

A: Instead of one lock for the entire structure, use multiple locks (one per segment/stripe). ConcurrentHashMap uses this: different bucket groups have independent locks. Reduces contention significantly.

Q: 38. What is a spin lock?

A: Thread repeatedly checks if lock is available (busy-wait) instead of sleeping. Low latency when wait is short. Wastes CPU if wait is long. JVM internally uses short spin before OS sleep for synchronized.

Q: 39. What is the ForkJoinPool.commonPool()?

A: Shared pool used by parallelStream() and CompletableFuture.supplyAsync() (if no executor specified). Its parallelism = available processors - 1. Long-blocking tasks can starve other parallel operations — provide custom executor for blocking work.

Q: 40. What is the difference between submit() and execute() in ExecutorService?

A: execute(Runnable): fire-and-forget, no return value, uncaught exceptions propagate to UncaughtExceptionHandler. submit(Runnable/Callable): returns Future, exceptions are held in Future and thrown when get() is called.

5. Exception Handling [25 Questions]

Q: 1. What is the exception hierarchy?

A: Throwable -> Error (JVM errors: OOM, StackOverflow — don't catch) and Exception. Exception -> RuntimeException (unchecked: NPE, IAE, IOOBE) and checked exceptions (IOException, SQLException).

Q: 2. When should you use checked vs unchecked exceptions?

A: Checked: caller can reasonably recover (file not found, network error). Unchecked: programming error or unrecoverable state (null pointer, illegal argument). Modern practice: prefer unchecked for clean APIs.

Q: 3. What is try-with-resources?

A: try (Connection c = getConn()) { ... } — automatically closes AutoCloseable resources in reverse order of declaration, even if exception thrown. Replaces verbose finally blocks.

Q: 4. What are suppressed exceptions?

A: When both the try block and the close() method throw exceptions, the close() exception is suppressed (attached to primary). Access via Throwable.getSuppressed(). Pre-Java 7, close() exceptions silently swallowed.

Q: 5. What is multi-catch?

A: catch (IOException | SQLException e) { ... } — handle multiple exception types in one block. e is effectively final in the block.

Q: 6. What is exception chaining?

A: Wrapping a cause: throw new ServiceException('Failed', cause). Preserves original exception. Access via getCause(). Essential for translating low-level to high-level exceptions without losing stack trace.

Q: 7. What is the difference between throw and throws?

A: throw: statement that throws an exception instance. throws: method signature declaration of checked exceptions the method may throw. throw is action; throws is contract.

Q: 8. What is a custom exception?

A: Extend RuntimeException (unchecked) or Exception (checked). Include message and cause constructors. Add domain fields (errorCode, httpStatus) for context.

Q: 9. What is @RestControllerAdvice in Spring Boot?

A: Global exception handler for REST controllers. @ExceptionHandler methods catch exceptions and return ResponseEntity with appropriate HTTP status and error body. Centralizes error handling.

Q: 10. When should you not catch Exception or Throwable?

A: Catching Exception hides bugs — swallows unexpected RuntimeExceptions. Catching Throwable also catches Errors (OOM) — worse. Catch specific exceptions. If must catch broadly, log and rethrow.

Q: 11. What is re-throwing an exception?

A: catch (Exception e) { ... doSomething(); throw e; } — handle partially then re-throw. Or wrap: throw new WrapperException(e). Original stack trace preserved with chaining.

Q: 12. What is the finally block?

A: Executes after try/catch regardless of exception, always (except System.exit() or JVM crash). Used for cleanup: closing connections, releasing locks. try-with-resources is preferred over explicit finally.

Q: 13. Can a finally block return a value?

A: Yes, but it overrides any value returned from try or catch — the finally return wins. This masks exceptions. Avoid returning from finally.

Q: 14. What happens to an exception thrown in finally?

A: It replaces the exception from the try block. The original exception is lost unless manually caught and chained. Another reason to prefer try-with-resources (which properly handles this via suppressed exceptions).

Q: 15. What is StackOverflowError?

A: Thrown when call stack exceeds its limit — typically due to infinite recursion. It's an Error, not Exception. Fix the recursion; don't catch it.

Q: 16. What is NullPointerException and how to prevent it?

A: Thrown when accessing a member of null reference. Prevention: null checks, Optional, @NonNull annotations, Objects.requireNonNull(). Java 14+ provides helpful NPE messages showing which variable was null.

Q: 17. What is IllegalArgumentException?

A: RuntimeException for invalid method arguments. Throw when caller passes bad input: if (n < 0) throw new IllegalArgumentException('n must be positive: ' + n).

Q: 18. What is IllegalStateException?

A: RuntimeException for invalid object state: method called at wrong time. Example: calling iterator.remove() without prior next(), or starting an already-started service.

Q: 19. What is UnsupportedOperationException?

A: Thrown by operations not supported by the implementation. Example: add() on Collections.unmodifiableList(). Optional operations in the Collections framework throw this.

Q: 20. What is ConcurrentModificationException?

A: Thrown by fail-fast iterators when collection is modified during iteration. Fix: use Iterator.remove(), removeIf(), or CopyOnWriteArrayList.

Q: 21. What is ClassCastException?

A: Thrown when casting an object to a class it's not an instance of. Use instanceof before casting, or use generics to avoid raw types.

Q: 22. What is ArrayIndexOutOfBoundsException?

A: Accessing array with index < 0 or >= length. Always validate index or use collection (which gives cleaner error).

Q: 23. How do you handle exceptions in Java 8 streams?

A: Streams don't propagate checked exceptions. Wrap checked exception in RuntimeException inside lambda, or use a helper: try { ... } catch (CheckedEx e) { throw new RuntimeException(e); }. Or use a functional interface that throws.

Q: 24. How do you handle exceptions in CompletableFuture?

A: exceptionally(ex -> fallback): recover from exception with fallback value. handle(result, ex -> ...): handles both normal and exceptional cases. whenComplete(result, ex -> ...): side effect on completion, doesn't transform.

Q: 25. What is the difference between Optional.orElse() and orElseGet()?

A: orElse(val): always evaluates val, even if Optional has a value (eager). orElseGet(supplier): evaluates supplier only if Optional is empty (lazy). Prefer orElseGet() when default is expensive to compute.

6. Generics, Lambdas & Streams [30 Questions]

Q: 1. What is type erasure in generics?

A: Generic type parameters are removed at compile time. List and List both become List at runtime. Cannot do new T[] or instanceof List. Bridge methods generated for overriding.

Q: 2. What is a bounded type parameter?

A: > restricts T to types implementing Comparable. Upper bound: extends. Lower bound: super (only for wildcards, not type params directly).

Q: 3. What is the difference between T and E and K,V in generics?

A: Convention only — all are type parameters. T = Type (general). E = Element (collections). K,V = Key,Value (maps). N = Number. S,U = second/third type. All interchangeable; convention improves readability.

Q: 4. What is a raw type?

A: Using a generic class without type parameter: List list = new ArrayList(). No type safety — compiler warnings. Exists for backwards compatibility. Avoid in new code.

Q: 5. What is Predicate?

A: Functional interface: boolean test(T t). Compose: and(), or(), negate(). Example: Predicate isLong = s -> s.length() > 10. filter(isLong.and(s -> s.startsWith('A'))).

Q: 6. What is Function?

A: Functional interface: R apply(T t). Transforms T to R. compose(before): apply before first. andThen(after): apply after. Example: Function len = String::length.

Q: 7. What is Consumer?

A: Functional interface: void accept(T t). Side-effect operation. andThen() chains consumers. Example: list.forEach(System.out::println) — println is Consumer.

Q: 8. What is Supplier?

A: Functional interface: T get(). Provides value with no input. Used for lazy initialization, default values. Example: Optional.orElseGet(() -> computeDefault()).

Q: 9. What is BiFunction?

A: Like Function but takes two inputs. R apply(T t, U u). Example: BiFunction repeat = (s, n) -> s.repeat(n).

Q: 10. What is UnaryOperator and BinaryOperator?

A: UnaryOperator extends Function — same input and output type. BinaryOperator extends BiFunction. Example: BinaryOperator add = Integer::sum. Used in Stream.reduce().

Q: 11. What is Stream.filter()?

A: Intermediate operation that keeps elements matching Predicate. Lazy — elements processed only when terminal op runs. Example: stream.filter(s -> s.length() > 3).

Q: 12. What is Stream.map()?

A: Transforms each element. stream.map(String::toUpperCase). Type can change: stream.map(String::length) returns Stream.

Q: 13. What is Stream.flatMap()?

A: Maps to streams, then flattens. Use for nested collections: list.stream().flatMap(Collection::stream). Null-safe: Optional.stream() (Java 9) in flatMap.

Q: 14. What is Stream.distinct()?

A: Removes duplicates using equals(). Stateful intermediate operation. Maintains order for ordered streams.

Q: 15. What is Stream.sorted()?

A: Sorts stream elements. sorted() uses natural order (Comparable). sorted(comparator) uses Comparator. Stateful — must buffer entire stream. Expensive for large streams.

Q: 16. What is Stream.reduce()?

A: Combines elements into single result: stream.reduce(0, Integer::sum). Three forms: identity+accumulator (always returns value), accumulator only (returns Optional), identity+accumulator+combiner (parallel).

Q: 17. What is Stream.collect()?

A: Terminal operation to gather stream elements. toList(), toSet(), toMap(), groupingBy(), joining(), counting(), summarizingInt() etc. via Collectors factory.

Q: 18. What is Collectors.groupingBy()?

A: Groups stream elements by classifier function into Map>. Can chain downstream collector: groupingBy(Person::getDept, counting()) for Map.

Q: 19. What is Collectors.partitioningBy()?

A: Special groupingBy with boolean predicate. Returns Map>. true key: matches predicate; false key: doesn't match.

Q: 20. What is Collectors.joining()?

A: Concatenates stream of strings. joining() — no delimiter. joining(', ') — with delimiter. joining(', ', '[', ']') — with delimiter, prefix, suffix.

Q: 21. What is Stream.peek()?

A: Intermediate operation for debugging/side effects: .peek(System.out::println). Does not modify stream. Only fires if terminal operation is reached (lazy).

Q: 22. What is Stream.limit() and skip()?

A: limit(n): take first n elements (short-circuits). skip(n): skip first n elements. Combine: stream.skip(10).limit(5) = elements 11-15 (pagination equivalent).

Q: 23. What is Stream.findFirst() vs findAny()?

A: findFirst(): returns first element in encounter order (Optional). findAny(): returns any element — may be non-first in parallel streams (faster for parallel). Both short-circuit.

Q: 24. What is Stream.anyMatch(), allMatch(), noneMatch()?

A: Short-circuit terminal predicates. anyMatch: true if any element matches. allMatch: true if all match. noneMatch: true if none match. Efficient: stops as soon as answer is determined.

Q: 25. What is Stream.toArray()?

A: Converts stream to array: stream.toArray() returns Object[]. stream.toArray(String[]::new) returns String[]. Uses constructor reference for typed array.

Q: 26. What is IntStream, LongStream, DoubleStream?

A: Primitive specializations that avoid boxing overhead. range(1, 10), sum(), average(), min(), max(). Create: IntStream.range(0, 10), mapToInt(), Arrays.stream(intArray).

Q: 27. What is Stream.of() vs Arrays.stream()?

A: Stream.of(a, b, c): varargs, creates stream of those elements. Stream.of(array) creates Stream (stream of one element). Arrays.stream(intArray) creates IntStream properly.

Q: 28. What is a parallel stream and when should you use it?

A: parallelStream() uses ForkJoinPool to process in parallel. Use when: large dataset, CPU-intensive operations, operations are stateless and independent. Avoid for: small datasets, stateful ops, ordered operations that prevent parallelism.

Q: 29. What is method reference to instance method of arbitrary object?

A: String::toUpperCase — applied to each element of the stream as receiver: stream.map(String::toUpperCase). Equivalent to: stream.map(s -> s.toUpperCase()).

Q: 30. What is Collectors.toUnmodifiableMap()?

A: Collects to an unmodifiable Map. Requires key mapper, value mapper, and merge function (for duplicate keys). Result is immutable — add/remove throw UnsupportedOperationException.

7. Spring Core [40 Questions]

Q: 1. What is Spring IoC container?

A: Inversion of Control container manages object creation, configuration, and lifecycle. You define beans; Spring creates, wires, and manages them. ApplicationContext is the main IoC container interface.

Q: 2. What is Dependency Injection?

A: Pattern where dependencies are provided externally rather than created inside the class. Types: Constructor injection (recommended), Setter injection, Field injection (@Autowired on field — avoid in prod code).

Q: 3. What is a Spring Bean?

A: An object managed by the Spring IoC container. Defined via @Component, @Service, @Repository, @Controller, or @Bean in @Configuration class. Container creates, initializes, and destroys beans.

Q: 4. What is the difference between @Component, @Service, @Repository, @Controller?

A: @Component: generic stereotype. @Service: service layer — semantic. @Repository: persistence layer — also translates DB exceptions to Spring DataAccessException. @Controller: web layer for MVC. All trigger component scanning.

Q: 5. What is @Autowired?

A: Injects a bean dependency by type. Can inject constructors, setters, or fields. If multiple beans of same type: use @Qualifier('beanName') or @Primary. Prefer constructor injection for mandatory dependencies.

Q: 6. What is @Qualifier?

A: Disambiguates when multiple beans of same type exist. @Autowired @Qualifier('emailNotifier') NotificationService ns. Works with @Bean methods and @Component classes.

Q: 7. What is @Primary?

A: Marks a bean as preferred when multiple beans of same type exist. Used as default without needing @Qualifier at each injection point.

Q: 8. What are Spring bean scopes?

A: singleton (default — one instance per container), prototype (new instance per injection), request (per HTTP request), session (per HTTP session), application (per ServletContext), websocket.

Q: 9. What is the difference between singleton and prototype scope?

A: Singleton: one instance, shared, Spring manages lifecycle including destroy. Prototype: new instance per request, Spring does NOT call destroy — caller responsible for cleanup.

Q: 10. What is @Lazy?

A: Bean created only when first requested, not at container startup. Useful for expensive beans used rarely, or to break circular dependency. @Lazy on @Autowired: proxy injected, actual bean created on first use.

Q: 11. What is @Scope('prototype') on a singleton-scoped bean?

A: Problem: prototype bean injected into singleton is created once (not per use). Fix: inject ApplicationContext and call getBean() each time, or use @Lookup method injection, or use scoped proxy: @Scope(value='prototype', proxyMode=TARGET_CLASS).

Q: 12. What is ApplicationContext?

A: Central IoC container. Extends BeanFactory with: i18n (MessageSource), event publishing, AOP, environment/profiles. Types: AnnotationConfigApplicationContext, ClassPathXmlApplicationContext, WebApplicationContext.

Q: 13. What is BeanFactory vs ApplicationContext?

A: BeanFactory: basic IoC, lazy initialization, minimal features. ApplicationContext: extends BeanFactory, eager initialization by default, events, i18n, AOP support. Always use ApplicationContext in real apps.

Q: 14. What is the Spring bean lifecycle?

A: Instantiate -> Populate properties -> BeanNameAware.setBeanName -> BeanFactoryAware.setBeanFactory -> @PostConstruct / InitializingBean.afterPropertiesSet -> init-method -> Bean ready -> @PreDestroy / DisposableBean.destroy -> destroy-method.

Q: 15. What is @PostConstruct and @PreDestroy?

A: @PostConstruct: method runs after DI is complete. @PreDestroy: method runs before bean is removed from container. Better than implementing InitializingBean/DisposableBean (less coupling to Spring).

Q: 16. What is Spring AOP?

A: Aspect-Oriented Programming: modularizes cross-cutting concerns (logging, security, transactions). Key concepts: Aspect (the concern), Joinpoint (method execution), Pointcut (where to apply), Advice (what to do), Weaving (applying aspects).

Q: 17. What are the types of AOP advice?

A: @Before: before method. @After: after method (always). @AfterReturning: after normal return. @AfterThrowing: after exception. @Around: wraps method — most powerful (can suppress, modify result, handle exception).

Q: 18. What is a pointcut expression?

A: Defines where advice applies. execution(* com.app.service.*.*(..)) — all methods in service package. @annotation(Logged) — all methods annotated with @Logged. @within, args, target, within also available.

Q: 19. What is @Transactional?

A: Declarative transaction management. Spring creates a proxy that begins a transaction before method, commits on success, rolls back on RuntimeException. Must be on public methods of Spring-managed beans.

Q: 20. What is transaction propagation?

A: REQUIRED (default): join existing or create new. REQUIRES_NEW: always create new, suspend existing. NESTED: nested savepoint within existing. MANDATORY: must exist or throw. NEVER: must not exist. SUPPORTS: join if exists. NOT_SUPPORTED: suspend existing.

Q: 21. What is transaction isolation?

A: DEFAULT (DB default), READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE. Higher isolation: safer but slower. Configure per @Transactional(isolation=SERIALIZABLE).

Q: 22. What is @Transactional rollbackFor?

A: By default, Spring rolls back on unchecked exceptions (RuntimeException, Error). For checked exceptions: @Transactional(rollbackFor=IOException.class). Or: noRollbackFor to prevent rollback on specific unchecked exceptions.

Q: 23. What is Spring Expression Language (SpEL)?

A: Expression language for runtime evaluation: @Value('#{systemProperties.myProp}'), @PreAuthorize('hasRole(ADMIN)'), @Cacheable(key='#userId'). Supports: property access, method calls, conditionals, collections.

Q: 24. What is @Value?

A: Injects values: @Value('${app.name}') from properties. @Value('#{beanName.property}') from SpEL. @Value('hardcoded') literal. Supports default: @Value('${app.timeout:30}').

Q: 25. What is @Profile?

A: Activates beans only when specific profile is active. @Profile('dev') on @Component or @Bean. Activate: spring.profiles.active=dev in properties, or -Dspring.profiles.active=dev JVM arg.

Q: 26. What is @Conditional?

A: @Conditional(MyCondition.class) — bean created only if condition matches. Basis for all Spring Boot @ConditionalOnXxx annotations. Implement Condition.matches() with ConditionContext.

Q: 27. What is @Import?

A: Imports additional @Configuration classes into the current configuration. Used to modularize configuration. @ImportResource imports XML config. @Import({DbConfig.class, CacheConfig.class}).

Q: 28. What is the Spring Event system?

A: ApplicationEventPublisher.publishEvent(new MyEvent(this)) publishes events. @EventListener on method handles events. Async events: @Async + @EventListener. Decouples components without direct dependency.

Q: 29. What is @Lookup?

A: Allows singleton bean to get new prototype bean instance on each call. Spring overrides the @Lookup method at runtime via CGLIB subclass, returning new prototype instance each time.

Q: 30. What is Spring Environment?

A: Abstraction for properties and profiles. env.getProperty('key'). Sources: system properties, system env, application.properties, JNDI, servlet params. PropertySourcesPlaceholderConfigurer resolves ${...} placeholders.

Q: 31. What is @ConfigurationProperties?

A: Binds prefixed properties to a POJO. @ConfigurationProperties(prefix='app.db') class DbProperties { String url; int poolSize; }. Type-safe, IDE completion, validation with @Validated.

Q: 32. What is component scanning?

A: @ComponentScan(basePackages='com.app') tells Spring where to look for @Component annotated classes. @SpringBootApplication includes @ComponentScan of the application package and sub-packages.

Q: 33. What is a CGLIB proxy vs JDK proxy?

A: JDK proxy: interface-based (requires interface). CGLIB proxy: subclass-based (works without interface, used for @Transactional on concrete classes). Spring Boot uses CGLIB by default for @Configuration classes.

Q: 34. What is circular dependency and how to resolve it?

A: Bean A needs B; B needs A — container cannot create either first. Fixes: refactor design (usually better), @Lazy on one injection point, setter injection instead of constructor injection.

Q: 35. What is the BeanPostProcessor?

A: Allows custom modification of new bean instances after initialization. postProcessBeforeInitialization and postProcessAfterInitialization. Used by AOP proxy creation, @Autowired processing, @Value injection.

Q: 36. What is the BeanFactoryPostProcessor?

A: Modifies the bean factory metadata (bean definitions) before beans are instantiated. PropertySourcesPlaceholderConfigurer is a BeanFactoryPostProcessor. Runs very early in container lifecycle.

Q: 37. What is Spring's @Async?

A: Makes method run in a separate thread pool (SimpleAsyncTaskExecutor by default or custom @Bean TaskExecutor). Returns void or Future/CompletableFuture. Must be on a bean method called from another bean (proxy bypass issue if same class).

Q: 38. What is the difference between @Bean and @Component?

A: @Component: auto-detected via component scanning, on class. @Bean: explicit bean declaration in @Configuration class, on method. @Bean gives full control over creation (third-party classes you can't annotate).

Q: 39. What is @Configuration vs @Component?

A: @Configuration: denotes a configuration class, enables full CGLIB proxy for @Bean methods (inter-bean method calls work correctly — singleton guaranteed). @Component with @Bean: lite mode — direct method calls not intercepted, may create multiple instances.

Q: 40. What is @EnableXxx pattern?

A: Meta-annotation that @Import a configuration or register a BeanDefinitionRegistrar. @EnableAsync, @EnableCaching, @EnableScheduling, @EnableWebMvc. Used to modularly enable Spring features.

8. Spring Boot [35 Questions]

Q: 1. What is Spring Boot?

A: Opinionated framework built on Spring that simplifies setup via auto-configuration, embedded server, starter dependencies, and sensible defaults. No XML config needed. Run as a standalone JAR.

Q: 2. What is auto-configuration?

A: Spring Boot auto-configures beans based on classpath dependencies and properties. @ConditionalOnClass, @ConditionalOnMissingBean etc. control activation. spring.factories / AutoConfiguration.imports lists all candidates.

Q: 3. What is a Spring Boot starter?

A: Convenience dependency that aggregates related dependencies. spring-boot-starter-web pulls in Spring MVC, Tomcat, Jackson. spring-boot-starter-data-jpa pulls Hibernate, Spring Data JPA, JDBC.

Q: 4. What is @SpringBootApplication?

A: Combines: @Configuration + @EnableAutoConfiguration + @ComponentScan. Entry point annotation. Scans the current package and sub-packages for components.

Q: 5. What is Spring Boot's property resolution order (highest to lowest)?

A: Command-line args > SPRING_APPLICATION_JSON > OS env vars > System properties > application-{profile}.properties > application.properties > @PropertySource > default values. Later sources override earlier.

Q: 6. What is application.properties vs application.yml?

A: Both configure Spring Boot. YAML supports hierarchy (less repetition) and lists natively. Both are equivalent in capability. application.yml is often preferred for complex config. application.properties is simpler.

Q: 7. What is Spring Boot Actuator?

A: Provides production-ready endpoints: /actuator/health, /actuator/metrics, /actuator/env, /actuator/beans, /actuator/loggers, /actuator/threaddump, /actuator/heapdump, /actuator/prometheus. Secure: expose only needed, require auth.

Q: 8. What is the health endpoint?

A: /actuator/health shows application health status (UP/DOWN/OUT_OF_SERVICE). Auto-includes: DB, disk space, Redis, Kafka. Custom: implement HealthIndicator. Expose details: management.endpoint.health.show-details=always.

Q: 9. What is Spring Boot DevTools?

A: Automatic restart on classpath changes (faster than cold restart via class reloading). LiveReload support. Relaxed property binding. Disabled in production. Added via spring-boot-devtools dependency.

Q: 10. What is @ConditionalOnProperty?

A: Creates bean only if specified property has a specific value. @ConditionalOnProperty(name='feature.x.enabled', havingValue='true'). Powerful for feature toggling via config.

Q: 11. What is @ConditionalOnMissingBean?

A: Creates bean only if no bean of that type already exists. Used for default implementations: auto-configure provides default, but user can override by providing their own bean.

Q: 12. What is @SpringBootTest?

A: Integration test annotation that loads the full application context. @SpringBootTest(webEnvironment=RANDOM_PORT) starts actual server. Slow but tests full stack. Use slice tests (@WebMvcTest, @DataJpaTest) for faster focused tests.

Q: 13. What is @WebMvcTest?

A: Slice test for Spring MVC layer only. Loads controllers, filters, @ControllerAdvice — no service/repo beans. Use @MockBean for dependencies. Tests HTTP handling, serialization, validation without starting full context.

Q: 14. What is @DataJpaTest?

A: Slice test for JPA layer. Configures in-memory H2, scans @Entity and repositories. Transactions rolled back after each test. Use @AutoConfigureTestDatabase(replace=NONE) for real DB tests.

Q: 15. What is MockMvc?

A: Test utility to perform HTTP requests against controllers without starting a real server. perform(get('/api/users')).andExpect(status().isOk()).andExpect(jsonPath('$.name').value('Alice')). Fast.

Q: 16. What is Spring Boot's embedded server?

A: Tomcat is default (spring-boot-starter-web). Swap to Jetty or Undertow by excluding Tomcat and adding starter. Runs as standalone JAR — no WAR deployment needed.

Q: 17. What is Spring Boot's fat/uber JAR?

A: All dependencies packaged inside a single executable JAR. Spring Boot Maven/Gradle plugin creates it via repackage. Run with java -jar app.jar. Internally uses custom classloader (BOOT-INF/classes, BOOT-INF/lib).

Q: 18. What is @RequestMapping vs @GetMapping?

A: @GetMapping is @RequestMapping(method=GET) — shorthand. Similarly @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping. @RequestMapping can handle multiple methods and is more general.

Q: 19. What is @RequestBody and @ResponseBody?

A: @RequestBody: deserializes HTTP request body (JSON -> Java object) via Jackson. @ResponseBody: serializes return value (Java object -> JSON response). @RestController = @Controller + @ResponseBody.

Q: 20. What is @PathVariable and @RequestParam?

A: @PathVariable: extracts value from URL path: /users/{id} -> @PathVariable Long id. @RequestParam: extracts query parameter: /users?name=Alice -> @RequestParam String name. Optional with defaultValue.

Q: 21. What is @Valid and @Validated?

A: @Valid: triggers JSR-303 bean validation on method parameter. Validation errors -> MethodArgumentNotValidException (400). @Validated: Spring variant, also enables group validation and cascade.

Q: 22. What is ResponseEntity?

A: Wrapper for HTTP response with body, status, and headers. return ResponseEntity.ok(body), ResponseEntity.created(uri).body(dto), ResponseEntity.status(CONFLICT).body(error). Full control over response.

Q: 23. What is Spring Boot's logging?

A: Uses SLF4J API with Logback as default implementation. Configure via application.properties: logging.level.com.app=DEBUG. Logback config: logback-spring.xml (supports spring profiles). Auto-rotates logs.

Q: 24. What is Spring Boot Externalized Configuration?

A: Configuration via environment variables, system properties, config files, command-line args — without code changes. 12-factor app compliance. Kubernetes ConfigMaps/Secrets map naturally to Spring properties.

Q: 25. What is @EnableCaching and @Cacheable?

A: @EnableCaching activates Spring's cache abstraction. @Cacheable('products') caches method return value by key (default: method args). @CacheEvict removes entry. @CachePut always updates cache. Pluggable backend: Redis, Caffeine, EhCache.

Q: 26. What is @Scheduled?

A: Runs method on schedule. @Scheduled(fixedRate=5000) every 5 sec. @Scheduled(cron='0 0 * * * *') cron expression. Requires @EnableScheduling. Runs in single thread by default — configure TaskScheduler for parallelism.

Q: 27. What is @RestControllerAdvice?

A: Global exception handler for REST controllers. @ExceptionHandler methods return ResponseEntity with error details. Centralized error mapping: business exception -> HTTP status + error body.

Q: 28. What is Spring Boot Actuator security?

A: Expose actuator endpoints at management.server.port=9090 (separate port). Secure: spring.security.user.name/password for basic auth, or exclude /health for public and require auth for rest. Never expose /actuator/heapdump publicly.

Q: 29. What is the difference between @Bean and @Component for configuration?

A: @Component needs class annotation and auto-discovery. @Bean in @Configuration gives full control: create third-party objects, conditional logic, explicit method call. Use @Bean for infrastructure, @Component for domain.

Q: 30. What is Spring Boot's property binding?

A: spring.datasource.url in properties -> DataSourceProperties.url via @ConfigurationProperties(prefix='spring.datasource'). Relaxed binding: spring.my-prop = SPRING_MY_PROP = spring.myProp all bind to same field.

Q: 31. What is @ImportAutoConfiguration?

A: Used in tests to selectively import specific auto-configuration classes without loading all. @DataJpaTest uses @ImportAutoConfiguration with hibernate, datasource, transaction auto-configs only.

Q: 32. What is spring.factories?

A: Spring Boot loads auto-configuration candidates from META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (Spring Boot 2.7+) or META-INF/spring.factories (older). Lists EnableAutoConfiguration implementations.

Q: 33. What is Micrometer?

A: Vendor-neutral application metrics facade (like SLF4J for metrics). Integrates with Prometheus, Datadog, InfluxDB etc. Spring Boot auto-configures MeterRegistry. @Timed, Counter, Gauge, Timer are key types.

Q: 34. What is the difference between spring-boot-starter-test and JUnit?

A: spring-boot-starter-test includes: JUnit 5, Mockito, AssertJ, Hamcrest, MockMvc, @SpringBootTest support, Testcontainers integration starter. JUnit alone is just the testing framework.

Q: 35. What is TestRestTemplate?

A: RestTemplate variant for @SpringBootTest with RANDOM_PORT. No 4xx/5xx exceptions by default — returns ResponseEntity. Used for real HTTP integration tests against embedded server.

9. Microservices [39 Questions]

Q: 1. What is a microservice?

A: Small, independently deployable service focused on a single business capability. Has its own database, communicates over HTTP/gRPC/messaging. Can be scaled, deployed, and developed independently.

Q: 2. What are the benefits and drawbacks of microservices?

A: Benefits: independent deployment, technology diversity, focused teams, resilience isolation. Drawbacks: distributed system complexity, network latency, eventual consistency, operational overhead, harder debugging.

Q: 3. What is the difference between monolith and microservices?

A: Monolith: single deployable unit with all components. Simple but hard to scale and change independently. Microservices: multiple services; each scales and deploys independently but adds distribution complexity.

Q: 4. What is the strangler fig pattern?

A: Gradually migrate a monolith to microservices. New features built as microservices; traffic routed via API gateway. Over time, monolith functionality is replaced piece by piece until it's completely replaced.

Q: 5. What is the API Gateway pattern?

A: Single entry point for all clients. Handles: authentication, rate limiting, SSL termination, request routing, load balancing, response aggregation. Examples: AWS API Gateway, Kong, Nginx, Spring Cloud Gateway.

Q: 6. What is service discovery?

A: Mechanism for services to find each other without hardcoded addresses. Client-side (Eureka + Ribbon): client queries registry and load-balances. Server-side (AWS ALB): load balancer queries registry. Kubernetes: built-in DNS service discovery.

Q: 7. What is the circuit breaker pattern?

A: Prevents cascading failures. States: CLOSED (normal), OPEN (fail fast — no calls made, return fallback), HALF-OPEN (test calls — if succeed: CLOSED, else: OPEN). Libraries: Resilience4j, Hystrix (legacy).

Q: 8. What is the bulkhead pattern?

A: Isolates failures by partitioning resources. Thread pool per service: if Service A pool exhausts, Service B still has its own pool. Prevents one slow downstream from exhausting all threads.

Q: 9. What is the saga pattern?

A: Manages distributed transactions across services. Choreography: each service publishes events; others react. Orchestration: central orchestrator calls each service. Each step has compensating transaction for rollback.

Q: 10. What is CQRS?

A: Command Query Responsibility Segregation. Separate read (Query) and write (Command) models. Write model: optimized for consistency. Read model: optimized for queries (denormalized, cached). Often combined with event sourcing.

Q: 11. What is event sourcing?

A: Instead of storing current state, store all events that led to it. State = replay of events. Benefits: full audit log, time travel, easy projection of new read models. Complexity: event versioning, eventual consistency.

Q: 12. What is the outbox pattern?

A: Solves dual-write problem: write to DB + publish event atomically. Insert event into outbox table in same DB transaction. Separate process/Debezium reads outbox and publishes to message broker. Guarantees at-least-once delivery.

Q: 13. What is an anti-corruption layer?

A: Translates between different domain models when integrating legacy systems or external services. Prevents foreign domain concepts from leaking into your bounded context. Implemented as a service or adapter layer.

Q: 14. What is a bounded context?

A: DDD concept: explicit boundary within which a domain model applies consistently. Terms, rules, and models are consistent inside. Different bounded contexts communicate via well-defined APIs or events.

Q: 15. What is service mesh?

A: Infrastructure layer for service-to-service communication: Istio, Linkerd. Provides: mTLS, traffic routing, observability (tracing, metrics), circuit breaking, retries — all without code changes. Sidecar proxy (Envoy) per pod.

Q: 16. What is distributed tracing?

A: Tracks a request across multiple services using trace ID (spans). Jaeger, Zipkin, Datadog APM. Spring Boot: Micrometer Tracing (replaces Sleuth) auto-instruments and propagates B3/W3C trace headers.

Q: 17. What is the two-phase commit (2PC) and why avoid it in microservices?

A: 2PC coordinates a distributed transaction: prepare phase then commit/rollback. Works but: blocking protocol (coordinator crash = all participants block), tight coupling, performance overhead. Prefer saga pattern instead.

Q: 18. What is idempotency and why is it critical in microservices?

A: An operation is idempotent if repeating it has same effect as doing it once. Critical because: retries happen (network failures, timeouts). Implement via idempotency keys (UUID per request) stored in DB with result.

Q: 19. What is the health check pattern?

A: Each service exposes /health endpoint. Orchestrator (Kubernetes) or load balancer queries it. Liveness: is app running? Readiness: is app ready to receive traffic? Startup: has app finished initializing?

Q: 20. What is the sidecar pattern?

A: Deploy a helper container alongside main app container in same pod. Sidecar handles cross-cutting concerns: logging (Fluentd), proxy (Envoy), secrets rotation, cert management. Main app stays simple.

Q: 21. What is backend-for-frontend (BFF)?

A: Separate API gateway/backend per client type (mobile BFF, web BFF, partner BFF). Each BFF aggregates and transforms data for its specific client needs. Avoids one-size-fits-all API.

Q: 22. What is the retry pattern?

A: Automatically retry failed calls with backoff: exponential backoff (1s, 2s, 4s, 8s...) with jitter. Limit retry count. Only retry idempotent operations. Resilience4j @Retry, Spring Retry @Retryable.

Q: 23. What is rate limiting in microservices?

A: Limit requests per client/user per time window. Protects services from overload. Algorithms: token bucket (burst-friendly), sliding window (precise), fixed window (simple). Implement at API gateway or service level via Redis.

Q: 24. What is service contract testing?

A: Verify that service API matches what consumers expect. Consumer-driven contract testing (Pact): consumer generates contract; provider verifies against it. Detects breaking changes before deployment.

Q: 25. What is blue-green deployment?

A: Maintain two identical environments (blue=live, green=new). Deploy to green. Switch traffic (load balancer) to green. If issues: instantly switch back to blue. Zero-downtime deployments.

Q: 26. What is canary deployment?

A: Route small percentage of traffic (1-5%) to new version. Monitor metrics. Gradually increase to 100% if healthy. Roll back quickly if error rate spikes. More gradual than blue-green.

Q: 27. What is a feature flag?

A: Toggle features on/off without deploying. Target specific users, regions, or percentages. Tools: LaunchDarkly, Unleash, Split. Enables: trunk-based development, A/B testing, dark launches, gradual rollout.

Q: 28. What is the difference between REST and gRPC?

A: REST: HTTP/1.1, JSON, human-readable, wide support. gRPC: HTTP/2, Protobuf (binary, faster, smaller), bidirectional streaming, strong typing via IDL. gRPC is faster for inter-service; REST for public APIs.

Q: 29. What is choreography vs orchestration in saga?

A: Choreography: services publish events; others subscribe and react — decentralized, no single coordinator, harder to track flow. Orchestration: central saga orchestrator calls each service — centralized, easier to track, single point of failure.

Q: 30. What is eventual consistency?

A: System will become consistent over time, not immediately. Acceptable in many microservice scenarios. Design for it: idempotent operations, compensating transactions, read-your-writes with cache, conflict resolution.

Q: 31. What is the shared database anti-pattern?

A: Multiple microservices sharing one database — creates tight coupling, schema changes break multiple services, no independent deployment. Each service should own its data. Communicate via API or events, not shared tables.

Q: 32. What is the data aggregator pattern?

A: Aggregate data from multiple services for a single response (e.g., order details = order + customer + product). API gateway or composite service fetches and merges. Consider: API composition vs CQRS read model for complex aggregations.

Q: 33. What is service versioning?

A: Maintain backward compatibility when changing APIs. Strategies: URL versioning (/v1/users, /v2/users), header versioning (Accept: application/vnd.app.v2+json), query param (?version=2). Deprecate old versions gracefully.

Q: 34. What is the ambassador pattern?

A: Sidecar that acts as proxy for network calls: adds retry, circuit breaking, logging, monitoring. Decouples network resilience logic from application code. Similar to service mesh but app-level.

Q: 35. What is chaos engineering?

A: Intentionally introducing failures (kill pods, inject network latency, kill dependencies) to verify system resilience. Tools: Chaos Monkey, LitmusChaos, AWS Fault Injection Simulator. Practice: start in dev, expand to prod gradually.

Q: 36. What are the 12-factor app principles?

A: Codebase (one repo), Dependencies (explicit), Config (environment), Backing services (attached resources), Build/Release/Run (separated stages), Processes (stateless), Port binding, Concurrency, Disposability, Dev/Prod parity, Logs (streams), Admin processes.

Q: 37. What is gRPC streaming?

A: gRPC supports: unary (one request, one response), server streaming (one request, multiple responses), client streaming (multiple requests, one response), bidirectional streaming (both). Built on HTTP/2 multiplexing.

Q: 38. What is Kafka in microservices?

A: Async communication between services via events. Decouples producers and consumers. Enables: event sourcing, CQRS read model update, audit log, fan-out to multiple consumers, replay. Topics as service contracts.

Q: 39. What is the timeout pattern?

A: Every network call must have a timeout. Without timeout: thread held indefinitely, pool exhausts, cascading failure. Set timeout < downstream SLA. Use circuit breaker to open after N timeouts.

10. REST APIs & HTTP [35 Questions]

Q: 1. What is REST?

A: Representational State Transfer — architectural style for distributed hypermedia systems. Constraints: stateless, client-server, cacheable, uniform interface, layered system, code-on-demand (optional).

Q: 2. What are the HTTP methods and their semantics?

A: GET: read (idempotent, safe). POST: create. PUT: replace entire resource (idempotent). PATCH: partial update. DELETE: remove (idempotent). HEAD: GET without body. OPTIONS: allowed methods.

Q: 3. What is idempotency in HTTP?

A: Repeating a request multiple times has the same effect as once. GET, PUT, DELETE, HEAD are idempotent. POST is not (creates new resource each time). PATCH is not necessarily idempotent.

Q: 4. What are HTTP status codes?

A: 1xx: informational. 2xx: success (200 OK, 201 Created, 204 No Content). 3xx: redirect (301 Moved, 304 Not Modified). 4xx: client error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable). 5xx: server error (500, 503).

Q: 5. What is 401 vs 403?

A: 401: not authenticated (missing or invalid credentials). 403: authenticated but not authorized (logged in but no permission). 404 sometimes used instead of 403 to avoid revealing resource existence.

Q: 6. What is REST HATEOAS?

A: Hypermedia As The Engine Of Application State. Responses include links to related actions. Client navigates API by following links, not hardcoding URLs. Rarely implemented in practice but part of REST maturity level 3.

Q: 7. What is the Richardson Maturity Model?

A: Level 0: one URL, one method. Level 1: multiple resources (URLs). Level 2: HTTP methods + status codes properly used. Level 3: HATEOAS. Most real APIs are level 2.

Q: 8. What is content negotiation?

A: Client specifies preferred format via Accept header (Accept: application/json). Server responds in that format or 406 Not Acceptable. Server announces format via Content-Type header in response.

Q: 9. What is CORS?

A: Cross-Origin Resource Sharing. Browser security: blocks JS requests to different origin. Server sets Access-Control-Allow-Origin header. Preflight OPTIONS request for non-simple requests. Spring: @CrossOrigin or WebMvcConfigurer.addCorsMappings().

Q: 10. What is an API versioning strategy?

A: URL: /v1/users. Header: API-Version: 1. Content type: Accept: application/vnd.company.v1+json. Query param: ?version=1. URL versioning is simplest and most visible.

Q: 11. What is pagination in REST?

A: Return subsets of large datasets. Offset: ?page=2&size;=20. Cursor: ?cursor=abc&size;=20. Include pagination metadata in response or Link header. Cursor-based preferred for large/dynamic datasets.

Q: 12. What is filtering, sorting, and field selection in REST?

A: Filtering: GET /products?category=electronics&minPrice;=100. Sorting: ?sort=price,asc. Field selection (sparse fieldsets): ?fields=id,name,price. All via query params for GET.

Q: 13. What is an ETag?

A: Entity Tag — hash of response body. Server sends ETag header. Client sends If-None-Match on next request. If ETag matches: 304 Not Modified (no body). Reduces bandwidth, enables caching.

Q: 14. What is a conditional request?

A: If-None-Match (ETag-based), If-Modified-Since (date-based) for GET: return 304 if unchanged. If-Match for PUT/DELETE: update only if resource hasn't changed since client last read it (optimistic concurrency).

Q: 15. What is the difference between PUT and PATCH?

A: PUT replaces the entire resource (missing fields become null/default). PATCH applies partial update (only provided fields change). Use PATCH for partial updates to avoid sending full payload.

Q: 16. What is idempotency key?

A: Client-generated UUID sent with POST request. Server uses key to detect duplicate requests and return cached response instead of creating again. Prevents double creation on retry.

Q: 17. What is HTTP caching?

A: Cache-Control: max-age=3600 (cache for 1 hour). Expires header (legacy). no-cache: always revalidate. no-store: don't cache. Vary: header specifies which request headers to consider for cache key.

Q: 18. What is a REST API error response format?

A: Standardized: {status, error, message, timestamp, path, traceId}. RFC 7807 Problem Details: {type, title, status, detail, instance}. Consistent format allows clients to handle errors programmatically.

Q: 19. What is OpenAPI / Swagger?

A: OpenAPI Specification: machine-readable API description in YAML/JSON. Swagger UI: interactive documentation and testing tool. SpringDoc or Springfox auto-generates from Spring MVC annotations.

Q: 20. What is API Gateway vs API documentation?

A: API Gateway is a runtime infrastructure component (routing, auth, rate limiting). API documentation (OpenAPI/Swagger) is a specification for how the API works. Two different concerns.

Q: 21. What is gRPC vs REST?

A: REST: JSON over HTTP/1.1, human-readable, wide client support. gRPC: Protobuf over HTTP/2, binary (faster/smaller), strongly typed IDL, streaming support. gRPC better for internal service-to-service; REST for public APIs.

Q: 22. What is GraphQL?

A: Query language for APIs: client specifies exactly what fields it needs in the query. Single endpoint. Solves over-fetching and under-fetching. Mutation for writes, Subscription for real-time. Tradeoff: complex caching, N+1 query problem.

Q: 23. What is server-sent events (SSE)?

A: Server pushes events to client over persistent HTTP connection. One-directional (server to client). Client: EventSource API. Server: text/event-stream content type. Simpler than WebSocket for one-way streaming.

Q: 24. What is WebSocket?

A: Full-duplex communication over single TCP connection. Upgrades from HTTP. Both server and client can send messages anytime. Use for real-time: chat, live updates, gaming. Spring: @MessageMapping with STOMP.

Q: 25. What is HTTP/2?

A: Binary protocol (not text). Multiplexing: multiple requests over one connection (no head-of-line blocking). Header compression (HPACK). Server push. Faster than HTTP/1.1 especially for many small requests.

Q: 26. What is TLS/HTTPS?

A: Transport Layer Security encrypts HTTP traffic. HTTPS = HTTP + TLS. Prevents eavesdropping, man-in-the-middle. Certificate validates server identity. Always use in production. Spring Boot: server.ssl.key-store properties.

Q: 27. What is mTLS?

A: Mutual TLS: both client and server present certificates. Server validates client cert. Used for service-to-service auth in zero-trust networks. More secure than API keys.

Q: 28. What is request throttling vs rate limiting?

A: Rate limiting: limit total requests per time window per client. Throttling: slow down requests when limit is approached (queue or delay). Both protect backend from overload.

Q: 29. What is a webhook?

A: HTTP callback: when an event occurs, server sends POST request to registered URL (provided by client). Push model (server pushes to client) vs polling (client asks repeatedly). Used by Stripe, GitHub, Twilio.

Q: 30. What is REST API security best practices?

A: Always use HTTPS. Authenticate every request (JWT/OAuth2). Validate and sanitize all input. Rate limit. Return minimal error detail. Use CORS correctly. Rotate secrets. Audit log all mutations. Use RBAC/ABAC for authorization.

Q: 31. What is JSON vs XML in REST?

A: JSON: lighter, native JavaScript support, human-readable, default for modern APIs. XML: verbose, supports namespaces/schemas (XSD), required in some legacy/enterprise/SOAP contexts. Accept/Content-Type headers negotiate format.

Q: 32. What is HAL (Hypertext Application Language)?

A: JSON format for including hyperlinks in API responses. _links object with href for related resources. Partially implements HATEOAS. Used in Spring HATEOAS. Example: {id:1, name:'Alice', _links:{self:{href:'/users/1'}}}.

Q: 33. What is the difference between synchronous and asynchronous API?

A: Synchronous: client waits for response (HTTP request-response). Asynchronous: client submits request, gets job ID, polls for result or receives callback/webhook. Async for long-running operations (report generation, video processing).

Q: 34. What is multipart/form-data?

A: Content type for file upload alongside form data. Spring: @RequestParam MultipartFile file. Each part has its own Content-Disposition header. Max file size: spring.servlet.multipart.max-file-size.

Q: 35. What is @ResponseStatus in Spring MVC?

A: @ResponseStatus(HttpStatus.CREATED) on method or exception class sets default HTTP status. On exception class: returned when that exception is thrown. Can be overridden by ResponseEntity.

11. JPA & Hibernate [35 Questions]

Q: 1. What is JPA?

A: Java Persistence API — specification for ORM. Defines annotations (@Entity, @Id, @OneToMany etc.) and EntityManager API. Implementations: Hibernate (most common), EclipseLink, OpenJPA.

Q: 2. What is an Entity?

A: A Java class mapped to a database table. @Entity annotation required. Must have @Id (primary key). Each instance represents a row. Default table name = class name (override with @Table).

Q: 3. What is EntityManager?

A: Core JPA API for CRUD operations. persist(entity), find(Entity.class, id), merge(entity), remove(entity), createQuery(jpql). Spring Data JPA wraps it with Repository abstraction.

Q: 4. What is the persistence context?

A: First-level cache (unit of work). EntityManager maintains a map of managed entities. Within a transaction, loading same entity twice returns same object. Changes tracked automatically (dirty checking). Flushed to DB on commit or explicit flush().

Q: 5. What is the difference between persist() and merge()?

A: persist(): new entity becomes managed (must have no existing ID or detached state throws). merge(): returns a new managed copy of a detached entity — does not change the passed entity. Use merge for update after deserialization.

Q: 6. What is dirty checking?

A: Hibernate compares entity state at transaction start vs flush time. If changed, generates UPDATE automatically — no need to call update/save explicitly. Works only for managed entities within a persistence context.

Q: 7. What is the N+1 problem?

A: Loading N entities, then for each, a separate query for related entity = N+1 queries. Example: list of orders, each lazy-loading customer = 1+N queries. Fix: JPQL JOIN FETCH, @EntityGraph, or @BatchSize.

Q: 8. What is lazy vs eager loading?

A: LAZY: related entity loaded only when accessed (default for collections). EAGER: loaded immediately with parent query (default for @ManyToOne, @OneToOne). Prefer LAZY and fetch explicitly when needed to avoid N+1 and cartesian product.

Q: 9. What is @OneToMany?

A: Maps one entity to a collection of another. Usually on the inverse side. Bidirectional: @OneToMany(mappedBy='parent') on parent, @ManyToOne on child. Use Set not List for bag semantics. LAZY fetch recommended.

Q: 10. What is cascade type?

A: Propagates operations from parent to child. CascadeType.ALL propagates persist, merge, remove, refresh, detach. Most common: PERSIST + MERGE. Be careful with REMOVE — can delete all children unintentionally.

Q: 11. What is orphanRemoval?

A: Automatically deletes child entities when removed from parent's collection: @OneToMany(orphanRemoval=true). Useful for composition relationships where child has no meaning without parent.

Q: 12. What is JPQL?

A: Java Persistence Query Language — object-oriented query language. Queries on entities/fields not tables/columns. SELECT o FROM Order o WHERE o.status = :status. Portable across JPA implementations.

Q: 13. What is a Criteria API?

A: Type-safe programmatic query building. CriteriaBuilder, CriteriaQuery, Root, Predicate. More verbose than JPQL but safe for dynamic queries. Spring Data JPA Specifications wrap Criteria API.

Q: 14. What is a named query?

A: @NamedQuery on entity class, referenced by name: entityManager.createNamedQuery('Order.findByStatus'). Parsed and validated at startup. Use @Query in Spring Data JPA instead — cleaner.

Q: 15. What is Spring Data JPA repository?

A: Interface extending JpaRepository. Auto-implements: findById, findAll, save, delete, count. Derived queries: findByNameAndStatus(name, status) auto-generates JPQL. Custom: @Query annotation.

Q: 16. What is a derived query method?

A: Spring Data JPA generates query from method name. findByEmailAndActive(email, true) generates: SELECT u FROM User u WHERE u.email = :email AND u.active = :active.

Q: 17. What is @Modifying?

A: Required for @Query methods that modify data (UPDATE, DELETE). Combined with @Transactional. @Modifying(clearAutomatically=true) clears persistence context after modification.

Q: 18. What is a Spring Data Projection?

A: Return only selected fields. Interface projection: interface UserSummary { String getName(); String getEmail(); }. DTO projection via constructor expression in @Query. Reduces data transfer.

Q: 19. What are second-level (L2) cache and query cache?

A: L2 cache: shared across sessions/EntityManagers. Caches entities by ID. Providers: EhCache, Infinispan, Redis. @Cacheable on entity. Query cache: caches query result sets (list of IDs). Both require careful invalidation.

Q: 20. What is optimistic locking?

A: @Version on entity field (int or Long). Hibernate includes version in WHERE clause on UPDATE. If version mismatch (someone else updated): throws OptimisticLockException. Good for low contention.

Q: 21. What is pessimistic locking?

A: Database-level lock on row. LockModeType.PESSIMISTIC_WRITE: SELECT FOR UPDATE. Prevents concurrent reads/writes. Use for high contention on single rows (inventory, balance). Has performance cost.

Q: 22. What is the difference between @JoinColumn and mappedBy?

A: @JoinColumn: owns the FK column (usually the Many side). mappedBy: inverse side of bidirectional association, no FK. mappedBy must reference the field on the owning side. The owning side controls FK updates.

Q: 23. What is a Hibernate proxy?

A: Hibernate uses CGLIB to create proxy objects for lazy loading. proxy.getId() does not trigger load. proxy.getName() triggers DB query. instanceof check may fail — use Hibernate.getClass(proxy) or Hibernate.initialize(proxy).

Q: 24. What is connection pooling in JPA?

A: Sharing DB connections across requests instead of creating new ones per request. Spring Boot auto-configures HikariCP (default). Key settings: maximumPoolSize, minimumIdle, connectionTimeout, idleTimeout.

Q: 25. What is HikariCP?

A: Fastest JDBC connection pool for Java. Default in Spring Boot 2+. Key metrics: hikaricp_connections_active, hikaricp_connections_pending (> 0 = pool exhausted). Configure: spring.datasource.hikari.*.

Q: 26. What is @Embedded and @Embeddable?

A: @Embeddable: value object with no own identity (Address with street, city, zip). @Embedded in entity: Address address. Maps to same table as parent entity. Reusable across entities.

Q: 27. What is an entity lifecycle?

A: New/Transient -> Managed (after persist/find) -> Detached (after session close or detach()) -> Removed (after remove()). Lifecycle callbacks: @PrePersist, @PostPersist, @PreUpdate, @PostUpdate, @PreRemove, @PostRemove.

Q: 28. What is fetch join in JPQL?

A: SELECT o FROM Order o JOIN FETCH o.items — loads Order and items in one query, avoiding N+1. Cannot use DISTINCT with collection fetch join to avoid duplicate results (or use Set).

Q: 29. What is the difference between save() and saveAndFlush() in Spring Data?

A: save(): persists or merges in current transaction; sync to DB on transaction commit or explicit flush. saveAndFlush(): immediately flushes to DB within current transaction. Useful when next operation in same transaction needs the persisted ID.

Q: 30. What is Flyway vs Liquibase?

A: Both manage DB schema migrations. Flyway: SQL-based, simple, convention-over-configuration (V1__create_table.sql). Liquibase: XML/YAML/SQL changelogs, more powerful (rollback, preconditions). Spring Boot auto-runs both on startup.

Q: 31. What is the open-session-in-view anti-pattern?

A: Spring Boot default: EntityManager stays open through view rendering, allowing lazy loading in templates. Anti-pattern: lazy loads outside service layer, unpredictable queries. Disable: spring.jpa.open-in-view=false. Fetch all needed data in service layer.

Q: 32. What is @EntityGraph?

A: Defines fetch plan: which associations to eagerly load per query. @EntityGraph(attributePaths={'items','customer'}) on repository method. Alternative to JOIN FETCH in @Query. More reusable.

Q: 33. What is a composite primary key?

A: @IdClass(OrderItemId.class) or @EmbeddedId(OrderItemId). The IdClass has non-annotated fields matching @Id fields in entity. EmbeddedId wraps it in an @Embeddable. Required for join tables or legacy schemas.

Q: 34. What is the Spring Data Page and Slice?

A: Page: includes total count (extra COUNT query). Slice: knows only hasNext() (no count query, faster). Use Page for UI pagination with total pages. Use Slice for infinite scroll or when count is expensive.

Q: 35. What is @Transactional(readOnly=true)?

A: Hints Hibernate to skip dirty checking and flush — improves read performance. Also hints DB driver/pool for optimizations. Required for read operations on Spring Data repositories that use JPQL.

12. Apache Kafka [35 Questions]

Q: 1. What is Kafka?

A: Distributed event streaming platform. Producers publish messages to topics; consumers read from topics. Immutable append-only log. Horizontally scalable, fault-tolerant, high-throughput.

Q: 2. What is a Kafka topic?

A: A named stream of records. Partitioned across brokers. Each record has offset (position in partition), key, value, timestamp, headers.

Q: 3. What is a Kafka partition?

A: Ordered, immutable sequence of records within a topic. Unit of parallelism. Assigned to one broker (leader). Replicated to other brokers (followers). More partitions = more parallelism.

Q: 4. What is a consumer group?

A: Group of consumers sharing a topic subscription. Each partition is consumed by exactly one consumer in the group. More consumers than partitions = idle consumers. Different groups each get all messages.

Q: 5. What is offset?

A: Integer position of a record in a partition. Consumer tracks committed offset = last processed position. On restart: resume from committed offset. Manual commit for at-least-once; auto for at-most-once.

Q: 6. What are Kafka delivery guarantees?

A: At-most-once: auto-commit before processing (can lose). At-least-once: manual commit after processing (can duplicate). Exactly-once: idempotent producer + transactional API (EOS). Exactly-once is hardest and slowest.

Q: 7. What is the ISR (In-Sync Replicas)?

A: Set of replicas fully caught up with leader. Producer acks=all requires all ISR members to acknowledge. If ISR shrinks below min.insync.replicas: producer gets exception. Guarantees durability.

Q: 8. What is acks setting in Kafka producer?

A: acks=0: fire-and-forget (fastest, may lose). acks=1: leader acknowledges (can lose if leader fails before replication). acks=all/-1: all ISR replicas acknowledge (safest, slowest).

Q: 9. What is idempotent producer?

A: enable.idempotence=true: producer assigns sequence number; broker deduplicates retried messages. Prevents duplicates from producer retries. Required for exactly-once semantics.

Q: 10. What is a Kafka transaction?

A: Atomic write across multiple partitions/topics. Begin -> produce to multiple partitions -> commit/abort atomically. Consumer isolation: read_committed reads only committed data. Enables exactly-once.

Q: 11. What is consumer rebalance?

A: Redistribution of partition ownership among consumers in a group when: consumer joins/leaves, topic partitions change, heartbeat timeout. During rebalance: all consumption pauses (stop-the-world). Cooperative rebalance (incremental) reduces impact.

Q: 12. What is the difference between Kafka and RabbitMQ?

A: Kafka: log-based, messages retained (replay supported), pull model, high throughput, ordering per partition. RabbitMQ: queue-based, messages deleted after ACK, push model, flexible routing, lower throughput but lower latency.

Q: 13. What is Kafka log compaction?

A: Retains only the last value per key (like a changelog). Older records with same key are garbage-collected. Produces a compacted topic = latest state per key. Used for changelogs, configurations, current state snapshots.

Q: 14. What is Schema Registry?

A: Stores Avro/Protobuf/JSON Schema versions. Producer registers schema; consumer fetches it to deserialize. Schema ID embedded in message. Enforces compatibility: BACKWARD, FORWARD, FULL. Prevents deserialization errors.

Q: 15. What is Kafka Streams?

A: Java library for stream processing directly on Kafka topics. KStream (record stream), KTable (changelog stream/state). Joins, aggregations, windowing built-in. Runs inside your application — no separate cluster needed.

Q: 16. What is Kafka Connect?

A: Framework for streaming data between Kafka and external systems. Source connectors (DB -> Kafka), Sink connectors (Kafka -> DB/S3/ES). Debezium = popular source connector for CDC (Change Data Capture).

Q: 17. What is CDC (Change Data Capture)?

A: Capture database changes (insert/update/delete) as events in Kafka. Debezium reads DB transaction log. Used for: sync to search index, cache invalidation, event sourcing, audit trail.

Q: 18. What is the outbox pattern with Kafka?

A: Write event to outbox table in same DB transaction as business data. Debezium reads outbox table and publishes to Kafka. Guarantees at-least-once delivery without dual-write problem.

Q: 19. What is @KafkaListener?

A: Spring Kafka annotation to consume messages. @KafkaListener(topics='orders', groupId='order-service') on method. concurrency for parallel consumers. @Header for accessing Kafka headers (offset, partition, timestamp).

Q: 20. What is RetryableTopic?

A: Spring Kafka: automatically retry failed messages with exponential backoff using retry topics (orders-retry-1, orders-retry-2) and final DLT (dead letter topic). @RetryableTopic(attempts=4, backoff=@Backoff(delay=1000, multiplier=2)).

Q: 21. What is the difference between poll() timeout and max.poll.records?

A: poll(Duration timeout): max wait for records (0 = return immediately). max.poll.records: max records per poll. max.poll.interval.ms: max time between polls before consumer considered dead and rebalance triggered.

Q: 22. What is KRaft mode?

A: Kafka Raft — runs Kafka without ZooKeeper. Kafka manages its own metadata using Raft consensus. Available since Kafka 2.8, stable in 3.3+, ZooKeeper removed in 4.0. Simpler ops, faster metadata propagation.

Q: 23. What is a Kafka consumer lag?

A: Difference between latest offset (end of log) and committed consumer offset. High lag = consumer falling behind. Monitor via: kafka-consumer-groups.sh or JMX metrics. Alert if lag grows uncontrollably.

Q: 24. What is a dead letter topic (DLT)?

A: Topic where messages that cannot be processed are sent after max retries. @DltHandler method processes DLT messages. Used for manual inspection, alerting, or replay after fix.

Q: 25. What is Kafka quotas?

A: Throttle producer/consumer bandwidth or request rate per client. Prevents one client from saturating broker. Set via AdminClient or kafka-configs.sh. Important for multi-tenant Kafka clusters.

Q: 26. What is min.insync.replicas?

A: Minimum replicas that must acknowledge write. If ISR < min.insync.replicas and acks=all: producer gets NotEnoughReplicasException. Typically set to 2 for 3-broker cluster (replication.factor=3).

Q: 27. What is a Kafka compacted topic vs regular?

A: Regular: time/size based retention; old records deleted. Compacted: per-key retention; only last value per key kept. Compacted enables event replay to current state without replaying all history.

Q: 28. What is consumer group coordinator?

A: One broker elected as group coordinator per consumer group. Manages join/sync protocol during rebalance. Tracks member heartbeats. Stores committed offsets.

Q: 29. What is the difference between subscribe() and assign() in Kafka consumer?

A: subscribe(topics): dynamic partition assignment managed by group coordinator (consumer group). assign(partitions): manual partition assignment, no group coordinator, no rebalance. Use assign for custom partition management.

Q: 30. What is Kafka's record key?

A: Optional bytes field. Used for partitioning: records with same key go to same partition (hash(key) % numPartitions). Null key: round-robin. Key enables ordering guarantees and log compaction.

Q: 31. What is a Kafka batch consumer?

A: Spring Kafka: @KafkaListener with List> parameter. Processes batch of records per poll. More efficient than one-by-one for DB bulk inserts. batchListener=true in container factory.

Q: 32. What is Kafka message timestamp?

A: CREATE_TIME (set by producer, default) or LOG_APPEND_TIME (set by broker). Used in time-based retention, windowed operations in Kafka Streams. Access via ConsumerRecord.timestamp().

Q: 33. What is Kafka topic auto-creation?

A: auto.create.topics.enable=true (default): topic created automatically when produced to. Prod best practice: set false and manage topics explicitly via AdminClient or Kafka CLI. Prevents accidental topic creation.

Q: 34. What is the segment in Kafka?

A: A topic partition is divided into segments (files on disk). Active segment: current write target. Older segments: eligible for deletion based on retention policy. Default segment size: 1GB or 7 days.

Q: 35. What is Kafka consumer heartbeat?

A: Consumer sends heartbeat to group coordinator every heartbeat.interval.ms (default 3s). If no heartbeat for session.timeout.ms (default 45s): consumer declared dead, rebalance triggered. Keep heartbeat interval < session timeout / 3.

13. RabbitMQ & ActiveMQ [20 Questions]

Q: 1. What is AMQP?

A: Advanced Message Queuing Protocol — open standard for message-oriented middleware. RabbitMQ natively implements AMQP 0-9-1. Defines: producer, exchange, queue, binding, consumer, channel, virtual host.

Q: 2. What is a RabbitMQ exchange?

A: Receives messages from producers and routes to queues based on type (Direct, Fanout, Topic, Headers) and routing key. Producer never sends directly to queue.

Q: 3. What is a RabbitMQ channel?

A: Lightweight virtual connection inside a TCP connection. Publish/consume on channels, not connections directly. Multiple channels per connection reduces TCP overhead.

Q: 4. What is message acknowledgment in RabbitMQ?

A: Consumer sends ACK after successful processing: channel.basicAck(deliveryTag). NACK with requeue=false sends to DLX. Auto-ACK: broker removes message when delivered (before processing) — risk of loss.

Q: 5. What is publisher confirm?

A: Producer reliability feature. Broker sends ACK when message persisted. Enable: channel.confirmSelect(). addConfirmListener for async handling. Complements consumer ACK for end-to-end reliability.

Q: 6. What is prefetch/QoS in RabbitMQ?

A: channel.basicQos(n): consumer receives max n unacknowledged messages. Prevents fast producer overwhelming slow consumer. n=1: strict round-robin. n=0: unlimited (can cause memory issues).

Q: 7. What is the difference between durable queue and persistent message?

A: Durable queue: survives broker restart. Persistent message: written to disk (deliveryMode=2). Both needed for message durability. Non-durable queue or non-persistent message = lost on restart.

Q: 8. What is a RabbitMQ vhost?

A: Virtual host: logical separation within one RabbitMQ node. Separate exchanges, queues, bindings, users, permissions. Like database schemas. Used for multi-tenancy or environment separation (dev/staging).

Q: 9. What is Shovel plugin in RabbitMQ?

A: Moves messages from one queue (source) to another (destination) — even across different RabbitMQ clusters/servers. Used for cross-cluster routing, dead letter forwarding, or queue migration.

Q: 10. What is RabbitMQ federation?

A: Enables brokers in different locations to share exchanges/queues over unreliable WAN connections. Unlike clustering (same datacenter, reliable network), federation handles geographically distributed brokers.

Q: 11. What is the difference between RabbitMQ clustering and mirroring?

A: Clustering: multiple nodes share the same exchange/queue metadata. Queue data stored on one node by default. Mirroring (classic) or Quorum Queues: replicate queue data across nodes for HA.

Q: 12. What is a ActiveMQ Queue vs Topic?

A: Queue: point-to-point. One consumer receives each message. Competing consumers for scale. Topic: publish-subscribe. All active subscribers receive each message. Durable subscribers receive offline messages.

Q: 13. What is the Virtual Topic in ActiveMQ?

A: Publisher sends to topic VirtualTopic.X. Each consumer subscribes to Consumer.GroupName.VirtualTopic.X (a queue). Fan-out + competing consumers within each group. Best of queue and topic.

Q: 14. What is ActiveMQ Artemis vs Classic?

A: Artemis: next-gen, JMS 2.0, high-performance journal storage, AMQP 1.0 native. Classic: JMS 1.1, KahaDB, OpenWire native. New projects use Artemis. Classic is legacy/maintenance mode.

Q: 15. What is Spring AMQP's RabbitTemplate?

A: Main class for sending messages. convertAndSend(exchange, routingKey, object): serializes and sends. convertSendAndReceive(): RPC pattern with reply queue. CorrelationData for publisher confirms.

Q: 16. What is @RabbitListener?

A: Spring AMQP: listen to RabbitMQ queue. @RabbitListener(queues='orders'). Manual ACK: inject Channel and @Header(AmqpHeaders.DELIVERY_TAG) long tag; call basicAck/basicNack. concurrency for parallel consumers.

Q: 17. What is JMS?

A: Java Message Service — API specification for messaging. Defines: ConnectionFactory, Connection, Session, Destination (Queue/Topic), MessageProducer, MessageConsumer, Message types (Text, Object, Map, Bytes, Stream). Implemented by ActiveMQ, IBM MQ, etc.

Q: 18. What is the difference between ActiveMQ and Kafka?

A: ActiveMQ: traditional message broker, JMS, push model, messages deleted after ACK, complex routing, ordering per queue. Kafka: distributed log, pull model, messages retained for configurable period (replay supported), ordering per partition, high throughput.

Q: 19. When to use RabbitMQ over Kafka?

A: RabbitMQ: complex routing needed, messages should be deleted after processing, per-message TTL/priority, RPC patterns, lower volume (<100k/s). Kafka: high throughput, replay needed, event log/sourcing, long retention, streaming analytics.

Q: 20. What is message priority in RabbitMQ?

A: Declare queue with x-max-priority argument (e.g., 10). Set priority on message (0-10). Broker delivers higher priority messages first. Useful for urgent messages but adds overhead and breaks strict FIFO.

14. Docker & Containers [35 Questions]

Q: 1. What is Docker?

A: Platform for containerizing applications. Packages app + dependencies into a portable container image. Same image runs identically on dev, staging, prod. Uses Linux namespaces and cgroups for isolation.

Q: 2. What is the difference between VM and container?

A: VM: full OS + hypervisor, heavyweight, minutes to start, strong isolation. Container: shares host OS kernel, lightweight, seconds to start, process-level isolation. Containers more efficient; VMs stronger security boundary.

Q: 3. What is a Docker image vs container?

A: Image: immutable, read-only template (layers of filesystem changes). Container: running instance of an image (writable layer on top). One image -> many containers. Images stored in registry.

Q: 4. What is a Dockerfile?

A: Text file with instructions to build an image: FROM (base image), COPY/ADD (files), RUN (shell command), EXPOSE (ports), ENV (env vars), CMD/ENTRYPOINT (startup command).

Q: 5. What is the difference between CMD and ENTRYPOINT?

A: ENTRYPOINT: command that always runs (executable). CMD: default arguments, can be overridden at docker run. Together: ENTRYPOINT=['java'] CMD=['-jar','app.jar']. Override CMD: docker run img -jar other.jar.

Q: 6. What is multi-stage build?

A: Use multiple FROM stages in Dockerfile. Build stage: compile with full JDK. Final stage: copy artifact into minimal JRE image. Result: smaller production image without build tools.

Q: 7. What is a Docker layer?

A: Each Dockerfile instruction creates a read-only layer. Layers cached — only changed layers rebuilt. Order matters: put frequently changing instructions (COPY app.jar) last to maximize cache hits.

Q: 8. What is .dockerignore?

A: Like .gitignore for Docker build context. Excludes files from being sent to Docker daemon during build. Reduces build time and image size. Include: target/, .git/, *.md.

Q: 9. What is Docker Compose?

A: Tool to define and run multi-container applications via docker-compose.yml. Services, networks, volumes defined declaratively. docker compose up starts all services. Used for local development environments.

Q: 10. What is a Docker volume?

A: Persists data beyond container lifecycle. Managed by Docker (stored in /var/lib/docker/volumes). Bind mounts: host directory mounted into container. tmpfs: in-memory. Volumes survive container removal.

Q: 11. What are Docker networking modes?

A: bridge (default): isolated network, containers communicate via name. host: container shares host network stack. none: no network. overlay: multi-host networking (Swarm/Kubernetes). macvlan: container gets MAC address on physical network.

Q: 12. What is Docker registry?

A: Repository for Docker images. Docker Hub (public), ECR (AWS), GCR (Google), ACR (Azure), Nexus, Harbor (private). Push: docker push registry/image:tag. Pull: docker pull registry/image:tag.

Q: 13. What is the difference between COPY and ADD?

A: COPY: simple, recommended. Copies files from build context to image. ADD: also extracts .tar files and supports URLs (not recommended for URLs — use curl/wget in RUN). Prefer COPY unless tar extraction needed.

Q: 14. What is Docker security best practices?

A: Run as non-root user. Use minimal base images (alpine, distroless). Scan images for vulnerabilities (Trivy, Snyk). No secrets in Dockerfile (use secrets or env). Read-only filesystem. Drop capabilities.

Q: 15. What is a health check in Docker?

A: HEALTHCHECK INTERVAL=30s CMD curl -f http://localhost:8080/health || exit 1. Docker marks container as unhealthy if check fails. Compose and Swarm use this for orchestration decisions.

Q: 16. What is Docker Swarm?

A: Docker's native orchestration tool. Manages clusters of Docker nodes. Services, stacks (Compose format). Simpler than Kubernetes but less powerful. Largely superseded by Kubernetes in production.

Q: 17. What is the difference between docker run and docker exec?

A: docker run: create and start a new container from image. docker exec: run command inside already-running container (e.g., docker exec -it container-name bash for interactive shell).

Q: 18. What is the difference between docker stop and docker kill?

A: docker stop: sends SIGTERM (graceful shutdown, waits 10s), then SIGKILL. docker kill: sends SIGKILL immediately (no graceful shutdown). Always prefer stop for graceful shutdown.

Q: 19. What is a Docker entrypoint script?

A: Shell script set as ENTRYPOINT. Performs initialization: wait for dependencies, set env vars, run migrations, then exec the main process (exec java -jar app.jar). exec replaces shell with Java process (proper PID 1).

Q: 20. What is PID 1 problem in Docker?

A: First process in container is PID 1. It must handle signals (SIGTERM) and reap zombie processes. Java doesn't handle zombies. Fix: use tini or dumb-init as PID 1, or docker run --init.

Q: 21. What is a Spring Boot Docker layered JAR?

A: Spring Boot splits JAR into layers: dependencies, spring-boot-loader, snapshot-dependencies, application. COPY each layer separately in Dockerfile. Changes in application layer don't invalidate dependency layers — fast rebuilds.

Q: 22. What is Docker BuildKit?

A: Improved Docker build engine. Features: parallel stage execution, SSH forwarding, secret mounts (--secret), improved caching. Enable: DOCKER_BUILDKIT=1 or BuildKit backend in daemon.json.

Q: 23. What is docker build cache?

A: Docker caches each layer. Subsequent builds skip unchanged layers. Cache invalidated: instruction changes, file changes (COPY), or --no-cache flag. Order instructions from least to most changing.

Q: 24. What is distroless image?

A: Minimal base image containing only the app and its runtime, no shell, package manager, or OS tools. From Google: gcr.io/distroless/java17. Reduced attack surface, smaller size. Debug variant available.

Q: 25. What is a Docker secret?

A: docker secret create my_secret file.txt. Mounted into container at /run/secrets/my_secret. Not in image, not in environment variables. Only available in Docker Swarm natively; use Kubernetes Secrets or Vault in k8s.

Q: 26. What is the FROM scratch instruction?

A: Empty base image. Used for statically compiled binaries (Go, Rust). Smallest possible image. Java apps can't use it (need JRE). Use distroless or alpine instead.

Q: 27. What is Docker image tagging?

A: Images identified by name:tag. Tag: version label (latest, 1.0, 1.0.1, git-sha). latest: mutable, dangerous in production. Use immutable tags (SHA or semantic version) for reproducible deployments.

Q: 28. What is Trivy?

A: Open-source vulnerability scanner for container images, filesystems, git repos. Scans OS packages and language dependencies. Integrate in CI: trivy image my-app:latest. Reports CVEs with severity.

Q: 29. What is docker inspect?

A: Returns detailed JSON metadata about container/image: network settings, mounts, env vars, labels, state, ports. docker inspect container-name.

Q: 30. What is the --no-cache flag?

A: Forces Docker to rebuild all layers without using cached layers. Ensures fresh pull of base image and fresh package installs. Slower but ensures latest security patches.

Q: 31. What is a multi-platform Docker image?

A: Build for multiple architectures (amd64, arm64) using docker buildx build --platform linux/amd64,linux/arm64. Push multi-platform manifest to registry. Docker automatically pulls correct architecture.

Q: 32. What is the difference between docker-compose.yml version 2 and 3?

A: Version 3 added Swarm support (deploy, resources). Version 2 has features removed in v3 (depends_on with condition, mem_limit). For non-Swarm local dev: v2 may be more feature-rich. Compose Spec (v3.8+) is the current standard.

Q: 33. What is resource limiting in Docker?

A: docker run --memory=512m --cpus=1.5. Without limits: container can use all host resources. In Kubernetes: specified in Pod spec (resources.limits/requests). Always set limits in production.

Q: 34. What is docker system prune?

A: Removes all stopped containers, unused images, networks, build cache. Frees disk space. docker system prune -a removes all unused images (not just dangling). Use regularly in CI agents.

Q: 35. What is the overlay filesystem?

A: Docker's default storage driver on Linux. Layers stacked: lower (read-only image layers) + upper (writable container layer). Union view presented to container. Efficient because layers are shared across containers using same base image.

15. Kubernetes (K8s) [35 Questions]

Q: 1. What is Kubernetes?

A: Container orchestration platform. Manages deployment, scaling, networking, and lifecycle of containerized applications across a cluster of nodes. Declarative: define desired state; K8s ensures actual state matches.

Q: 2. What is a Pod?

A: Smallest deployable unit in K8s. Wraps one or more containers sharing network namespace (same IP) and storage. Usually one container per Pod (sidecar pattern = multiple). Ephemeral: pods die and are replaced.

Q: 3. What is a Deployment?

A: Manages a set of identical Pods (ReplicaSet). Declarative updates: rolling update or recreate strategy. rollback: kubectl rollout undo. Self-healing: replaces failed pods. Most common workload type.

Q: 4. What is a ReplicaSet?

A: Ensures specified number of Pod replicas running. Deployment manages ReplicaSets. Usually not used directly — managed by Deployment. Selects pods via label selector.

Q: 5. What is a StatefulSet?

A: For stateful apps (databases, Kafka). Provides stable network identity (pod-0, pod-1), stable persistent storage, ordered deployment/scaling. Pods not interchangeable — each has unique identity.

Q: 6. What is a DaemonSet?

A: Runs one Pod per Node (or selected nodes). Use for: log collector (Fluentd), metrics agent (Prometheus Node Exporter), security agent. Automatically deployed to new nodes.

Q: 7. What is a ConfigMap?

A: Stores non-sensitive configuration as key-value pairs or files. Mounted as env vars or file volume in pods. Decouples config from container image. Change ConfigMap: pods need restart to pick up (or use reloader).

Q: 8. What is a Secret?

A: Like ConfigMap but base64-encoded (not encrypted by default). Stores passwords, tokens, keys. Mount as env var or file. Use with encryption-at-rest for K8s secrets. Better: Vault or AWS Secrets Manager.

Q: 9. What is a Service?

A: Stable network endpoint for pods (pods have ephemeral IPs). ClusterIP (internal), NodePort (external via node IP:port), LoadBalancer (external via cloud LB), ExternalName (DNS alias). Selects pods by labels.

Q: 10. What is an Ingress?

A: HTTP/HTTPS routing into the cluster. Routes by hostname and path to Services. Requires Ingress Controller (nginx-ingress, Traefik, AWS ALB Ingress). SSL termination, virtual hosting, path rewriting.

Q: 11. What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?

A: PV: cluster-wide storage resource (actual disk). PVC: request for storage by pod. K8s binds PVC to matching PV. Pod uses PVC. StorageClass enables dynamic provisioning (PV created on PVC request).

Q: 12. What is a Namespace?

A: Virtual cluster within physical cluster. Isolates resources by team/environment. Resource names unique within namespace. RBAC policies scoped per namespace. Default namespaces: default, kube-system, kube-public.

Q: 13. What is a HorizontalPodAutoscaler (HPA)?

A: Automatically scales pod replicas based on CPU/memory or custom metrics. kubectl autoscale deploy app --min=2 --max=10 --cpu-percent=70. Requires metrics-server. Custom metrics via Prometheus Adapter.

Q: 14. What is a liveness probe?

A: K8s periodically checks if container is healthy. If fails: container restarted. Types: HTTP GET, TCP Socket, Exec command. Use for detecting deadlock or hung state (not for startup — use startupProbe).

Q: 15. What is a readiness probe?

A: K8s checks if container is ready to serve traffic. If fails: pod removed from Service endpoints (no traffic sent). Use for: waiting for DB connection, cache warm-up, dependency availability.

Q: 16. What is a startup probe?

A: Replaces liveness probe during slow startup. Gives slow-starting containers time to initialize before liveness checks begin. Once startup probe succeeds, liveness/readiness take over.

Q: 17. What is a resource request vs limit?

A: request: guaranteed minimum (used for scheduling). limit: maximum allowed. If pod exceeds memory limit: OOMKilled. If exceeds CPU limit: throttled (not killed). Always set both for predictable behavior.

Q: 18. What is kubectl?

A: CLI to interact with K8s cluster. kubectl get pods, describe pod pod-name, logs pod-name, exec -it pod-name -- bash, apply -f manifest.yaml, rollout status deploy/app, scale deploy/app --replicas=5.

Q: 19. What is a Kubernetes label?

A: Key-value metadata on K8s objects. Used for: Service pod selection, HPA targeting, scheduling constraints, organization. kubectl get pods -l app=my-app. Required for all production objects.

Q: 20. What is a Kubernetes annotation?

A: Key-value metadata for non-identifying info. Not used for selection. Used by tools: Ingress controller config, Prometheus scraping (prometheus.io/scrape: true), deployment notes.

Q: 21. What is RBAC in Kubernetes?

A: Role-Based Access Control. Role/ClusterRole: set of permissions (get, list, create, delete on resources). RoleBinding/ClusterRoleBinding: binds role to user/group/ServiceAccount. Namespace-scoped vs cluster-scoped.

Q: 22. What is a ServiceAccount?

A: Identity for processes running in pods. Pods use ServiceAccount to authenticate to K8s API. Mount as token. RBAC binds permissions to ServiceAccount. Default: default ServiceAccount in each namespace.

Q: 23. What is a Kubernetes Job and CronJob?

A: Job: runs pod to completion (not continuously). Useful for batch tasks, migrations. CronJob: runs Job on schedule (cron expression). Manages Job creation and history.

Q: 24. What is a rolling update strategy?

A: Gradually replaces old pods with new. maxSurge: extra pods during update. maxUnavailable: pods that can be down. Zero-downtime deployments. If new version fails health checks: rollout pauses.

Q: 25. What is a K8s init container?

A: Runs to completion before main containers start. Use for: schema migration, wait for dependency, download config, security setup. Multiple init containers run sequentially.

Q: 26. What is Helm?

A: Package manager for K8s. Charts: packages of K8s manifests with templates and values. helm install, upgrade, rollback, list. Values files for environment-specific config. Chart repository for sharing.

Q: 27. What is Kustomize?

A: Native K8s configuration management. Overlay base manifests with environment-specific patches. No templating — uses strategic merge patches and JSON patches. Built into kubectl: kubectl apply -k dir/.

Q: 28. What is a K8s network policy?

A: Firewall rules for pod communication. Default: all pods can communicate. NetworkPolicy restricts ingress/egress by pod/namespace selector, port, protocol. Requires CNI plugin that supports it (Calico, Cilium).

Q: 29. What is a PodDisruptionBudget (PDB)?

A: Ensures minimum number of pods stay available during voluntary disruptions (node drain, cluster upgrade). minAvailable: N pods must remain. maxUnavailable: at most N can be down. Protects HA during maintenance.

Q: 30. What is node affinity?

A: Constrains pods to run on nodes with specific labels. requiredDuringSchedulingIgnoredDuringExecution: hard requirement. preferredDuringScheduling: soft preference. Use for: GPU nodes, SSD nodes, zone placement.

Q: 31. What is a taint and toleration?

A: Taint on node: repels pods unless pod has matching toleration. Use to dedicate nodes: kubectl taint nodes node1 dedicated=gpu:NoSchedule. Pod tolerates: tolerations: [{key:'dedicated', value:'gpu'}].

Q: 32. What is Kubernetes etcd?

A: Distributed key-value store — K8s control plane's database. Stores all cluster state (pods, configs, secrets). Highly consistent (Raft consensus). Backup regularly — losing etcd = losing cluster state.

Q: 33. What is kubectl port-forward?

A: Forward local port to pod port: kubectl port-forward pod-name 8080:8080. Useful for debugging without exposing service externally. kubectl port-forward svc/my-service 8080:80 also works.

Q: 34. What is Kubernetes operator?

A: Custom controller + CRD (Custom Resource Definition) that manages complex stateful apps. Extends K8s API with domain knowledge. Examples: Prometheus Operator, Strimzi (Kafka), Percona Operator (MySQL).

Q: 35. What is cluster autoscaler?

A: Automatically adds/removes nodes based on resource pressure. Adds node when pods can't be scheduled. Removes underutilized nodes. Works with HPA for full auto-scaling: HPA scales pods, CA scales nodes.

16. AWS Cloud Services [40 Questions]

Q: 1. What is EC2?

A: Elastic Compute Cloud — virtual servers in AWS. Choose instance type (CPU/memory/GPU). AMI = machine image. Auto Scaling Group manages EC2 fleet. Use for: long-running workloads, custom runtimes.

Q: 2. What is Lambda?

A: Serverless function execution. Event-triggered (API Gateway, S3, DynamoDB Streams, SQS). Scales automatically. Pay per invocation+duration. Max 15 min execution, 10GB memory, 512MB-10GB /tmp storage.

Q: 3. What is the difference between EC2 and Lambda?

A: EC2: always-on VM, full control, long-running processes, predictable load. Lambda: event-driven, auto-scaling, no server management, short-lived, pay per use. Lambda: cold start latency. EC2: cost-effective for sustained traffic.

Q: 4. What is ECS vs EKS?

A: ECS: AWS-native container orchestration. Simpler to operate. Fargate = serverless containers. EKS: managed Kubernetes on AWS. Use ECS for AWS-native teams; EKS for K8s portability or existing K8s expertise.

Q: 5. What is AWS Fargate?

A: Serverless compute for containers. Run ECS/EKS tasks without managing EC2. Pay per pod CPU/memory. No patching or capacity provisioning. Slower scaling than EC2 but zero ops overhead.

Q: 6. What is S3?

A: Simple Storage Service — object storage. Store any file. 99.999999999% durability. Versioning, lifecycle policies, cross-region replication. Storage classes: Standard, Infrequent Access, Glacier (archive).

Q: 7. What is the difference between EBS, EFS, and S3?

A: EBS: block storage attached to one EC2 instance (like hard drive). EFS: elastic file system, NFS, shared by multiple EC2. S3: object storage, accessed via HTTP, unlimited scale, no direct mount (except S3FS). Choose by access pattern.

Q: 8. What is RDS?

A: Managed relational DB. Supports MySQL, PostgreSQL, Oracle, SQL Server, MariaDB, Aurora. Handles: backups, patching, Multi-AZ failover, read replicas. No SSH to DB server.

Q: 9. What is Aurora?

A: AWS-native relational DB. MySQL/PostgreSQL compatible. Distributed storage (6 copies across 3 AZs). 5x faster than MySQL, 3x than PostgreSQL. Serverless v2: auto-scales capacity. Aurora Global Database: multi-region.

Q: 10. What is DynamoDB?

A: AWS-managed NoSQL key-value and document DB. Single-digit ms latency at any scale. DynamoDB Streams for change events. GSI/LSI for alternate access patterns. On-demand or provisioned capacity.

Q: 11. What is DynamoDB single-table design?

A: Store all entities in one table. Partition key (PK) + sort key (SK) model different entities. Overload PK/SK: USER#123 PK, ORDER#456 SK. GSI for alternative access. Reduces costs and simplifies operations.

Q: 12. What is ElastiCache?

A: Managed Redis or Memcached. Sub-ms latency for caching. Redis: data structures, persistence, Pub/Sub, Cluster mode. Memcached: simple, multi-threaded. Use for session, result cache, rate limiting.

Q: 13. What is SQS?

A: Simple Queue Service — managed message queue. Standard (at-least-once, unordered) or FIFO (exactly-once, ordered). Dead letter queue for failed messages. Visibility timeout prevents duplicate processing. Infinite scale.

Q: 14. What is SNS?

A: Simple Notification Service — pub/sub. Publish to topic; all subscribers receive. Subscribers: SQS, Lambda, HTTP, email, SMS. Fan-out pattern: one SNS publish -> multiple SQS queues.

Q: 15. What is EventBridge?

A: Serverless event bus. Events from AWS services, SaaS apps, custom apps. Rules route events to targets (Lambda, SQS, Step Functions). Schema registry. More powerful than SNS for event routing.

Q: 16. What is Kinesis?

A: Real-time data streaming. Kinesis Data Streams: ordered stream, replay within retention. Kinesis Data Firehose: delivery to S3/Redshift/Elasticsearch automatically. Kinesis Analytics: SQL on streams.

Q: 17. What is VPC?

A: Virtual Private Cloud — isolated network in AWS. Subnets: public (internet-accessible) and private (no direct internet). Route tables, Internet Gateway, NAT Gateway. Security Groups (stateful firewall), NACLs (stateless).

Q: 18. What is the difference between Security Group and NACL?

A: Security Group: stateful, instance-level, allow rules only (no deny). NACL: stateless (must allow inbound AND outbound), subnet-level, allow and deny rules. SG: default deny; NACL: default allow.

Q: 19. What is an ALB vs NLB?

A: ALB (Application LB): Layer 7, HTTP/HTTPS, path/host routing, WAF integration. NLB (Network LB): Layer 4, TCP/UDP, ultra-low latency, static IP, TLS passthrough. Use ALB for HTTP apps; NLB for non-HTTP or extreme performance.

Q: 20. What is CloudFront?

A: AWS CDN. Caches content at edge PoPs globally. HTTP/HTTPS, custom domain, WAF, signed URLs/cookies for private content. Origin: S3, ALB, EC2, API Gateway. Reduces latency and origin load.

Q: 21. What is Route 53?

A: AWS DNS service. Hosted zones, record types (A, CNAME, Alias). Routing policies: simple, weighted, latency-based, failover, geolocation, multi-value. Health checks trigger failover.

Q: 22. What is IAM?

A: Identity and Access Management. Users, Groups, Roles, Policies (JSON). Principle of least privilege. Roles assumed by services (EC2 role, Lambda role). Never use root account. MFA everywhere.

Q: 23. What is an IAM role?

A: Identity with permissions, assumable by AWS services, EC2 instances, Lambda, etc. No credentials stored in app — temporary credentials via STS. EC2 instance profile = EC2 assuming a role.

Q: 24. What is KMS?

A: Key Management Service. Create and manage encryption keys. Envelope encryption: generate data key, encrypt data with data key, encrypt data key with KMS master key. Integrates with S3, RDS, EBS, SSM.

Q: 25. What is Secrets Manager?

A: Securely store and auto-rotate secrets (DB passwords, API keys). Accessible via SDK/CLI. Rotation via Lambda. Alternative: SSM Parameter Store (simpler, cheaper). Never hardcode credentials.

Q: 26. What is CloudWatch?

A: AWS monitoring and observability service. Metrics, Logs, Alarms, Dashboards, Events. Log Insights for querying logs. Alarms trigger Auto Scaling, SNS, Lambda. Unified agent for custom metrics from EC2.

Q: 27. What is X-Ray?

A: AWS distributed tracing. Traces requests across Lambda, EC2, ECS, API Gateway, SQS. Service map visualizes dependencies. Identifies latency bottlenecks. SDK instruments your code.

Q: 28. What is CloudFormation?

A: Infrastructure as Code for AWS. Templates (JSON/YAML) describe resources. Stacks manage lifecycle (create/update/delete). Change sets show what will change. CDK generates CloudFormation in code (Python, TypeScript, Java).

Q: 29. What is CDK (Cloud Development Kit)?

A: Define AWS infrastructure in code (Python, TypeScript, Java, Go). High-level constructs. Compiles to CloudFormation. Version controlled, tested, reusable. TypeScript CDK is most popular.

Q: 30. What is EKS?

A: Elastic Kubernetes Service — managed Kubernetes control plane. AWS manages etcd, API server, controller manager. You manage worker nodes (EC2 or Fargate). Integrates with IAM, ALB, EBS, EFS.

Q: 31. What is Cognito?

A: User identity service. User Pools: user directory with sign-up/sign-in (OIDC/OAuth2 provider). Identity Pools: federated identity, vend AWS credentials. Integrates with social IdPs (Google, Facebook) and enterprise SAML.

Q: 32. What is AWS WAF?

A: Web Application Firewall. Filter HTTP traffic with managed rule groups (OWASP Top 10, bot detection, IP reputation) or custom rules. Attached to ALB, CloudFront, API Gateway. Rate limiting at WAF level.

Q: 33. What is Auto Scaling Group (ASG)?

A: Manages EC2 fleet. Scales in/out based on: CloudWatch metrics, scheduled actions, predictive scaling. Min/desired/max instance counts. Integrates with ALB, ELB. Health checks replace failed instances.

Q: 34. What is the Shared Responsibility Model?

A: AWS: security OF the cloud (hardware, networking, facilities, managed service maintenance). Customer: security IN the cloud (data encryption, OS patching, IAM, network config, app security).

Q: 35. What is S3 lifecycle policy?

A: Automatically transition objects between storage classes (Standard -> IA after 30 days -> Glacier after 90 days) or expire/delete objects. Reduces storage costs for aging data.

Q: 36. What is AWS CodePipeline?

A: CI/CD service. Source (CodeCommit/GitHub) -> Build (CodeBuild) -> Deploy (CodeDeploy/ECS/Lambda). Integrates with all AWS services. Visualizes pipeline stages. Alternative: GitHub Actions with AWS deployments.

Q: 37. What is a NAT Gateway?

A: Allows private subnet instances to access internet (outbound only). Deployed in public subnet with Elastic IP. No inbound connections from internet. Required for private EC2 instances to download packages, call external APIs.

Q: 38. What is SQS visibility timeout?

A: After consumer receives message: message invisible for visibility timeout duration. Consumer must delete before timeout or message reappears. Set visibility timeout > max processing time. DLQ: message after maxReceiveCount retries.

Q: 39. What is the difference between SQS Standard and FIFO?

A: Standard: at-least-once delivery, best-effort ordering, ~unlimited TPS. FIFO: exactly-once processing, strict ordering by MessageGroupId, 3000 TPS with batching, 300 TPS without. FIFO more expensive.

Q: 40. What is Lambda cold start?

A: First invocation after idle period: Lambda downloads code, starts runtime, runs init code. Latency: 100ms-10s depending on runtime and code size. Mitigation: provisioned concurrency (pre-warmed), keep functions warm, use smaller packages, choose faster runtimes (Java GraalVM, Node.js, Python over Java JVM).

17. System Design [40 Questions]

Q: 1. What is horizontal vs vertical scaling?

A: Vertical (scale up): bigger server (more CPU, RAM). Limited by hardware. No code change. Single point of failure. Horizontal (scale out): more servers. Limited by architecture. Requires statelessness or distributed state.

Q: 2. What is a load balancer?

A: Distributes traffic across multiple servers. Round-robin, least connections, IP hash, weighted algorithms. Layer 4 (TCP) or Layer 7 (HTTP, path/header-based routing). Also provides health checks and failover.

Q: 3. What is the CAP theorem?

A: Distributed system can guarantee only 2 of: Consistency (all nodes see same data), Availability (every request gets a response), Partition Tolerance (works despite network partition). CA systems don't exist in distributed systems — P always possible.

Q: 4. What is PACELC?

A: Extension of CAP: when Partition: A vs C tradeoff. Else (no partition): L (Latency) vs C (Consistency) tradeoff. More nuanced than CAP. Cassandra: PA/EL (available + low latency). PostgreSQL: PC/EC (consistent, higher latency).

Q: 5. What is consistent hashing?

A: Hash ring maps both nodes and keys to positions. Key assigned to nearest node clockwise. Adding/removing node: only K/N keys move. Virtual nodes improve distribution. Used in: Cassandra, DynamoDB, Redis Cluster, CDN.

Q: 6. What is database sharding?

A: Horizontally partition data across multiple DB instances. Shard key determines placement. Range, hash, or directory sharding. Challenges: cross-shard queries, hot spots, resharding. Used when single DB can't handle load.

Q: 7. What is a read replica?

A: Copy of primary DB that serves reads. Primary handles writes; replicas handle reads. Reduces primary load. Eventual consistency between primary and replica. Use for: read-heavy workloads, reporting, geographic distribution.

Q: 8. What is a CDN?

A: Content Delivery Network: globally distributed servers that cache static assets (images, JS, CSS, video). Serves content from edge PoP closest to user. Reduces latency, reduces origin load. Examples: CloudFront, Akamai, Fastly, Cloudflare.

Q: 9. What is the difference between cache-aside and read-through?

A: Cache-aside: application checks cache, on miss loads from DB, stores in cache. App manages cache. Read-through: cache checks itself on miss, loads from DB — transparent to app. Cache is smarter in read-through.

Q: 10. What is the difference between write-through and write-behind?

A: Write-through: write to cache AND DB synchronously. Consistent. Slower writes. Write-behind: write to cache only, flush to DB asynchronously. Faster writes, risk of data loss.

Q: 11. What is an API rate limiter design?

A: Token bucket: refill tokens at rate R, consume 1 per request, burst allowed. Sliding window log: track timestamps, count in window. Fixed window counter: count per window, resets at boundary. Store counters in Redis for distributed limiting.

Q: 12. What is a URL shortener design?

A: Generate short code (base62 of auto-increment ID or random hash). Store mapping: short -> long URL in DB (Cassandra or MySQL). Read path: cache short -> long in Redis. 301 vs 302 redirect (caching vs analytics).

Q: 13. What is a messaging queue design pattern?

A: Decouple producer from consumer. Producer pushes to queue; consumer pulls at own pace. Handles: load spikes (queue absorbs burst), retry (re-queue failed), fan-out (multiple consumers), async processing.

Q: 14. What is a content-addressable store?

A: Objects stored and retrieved by hash of content (e.g., SHA256). Dropbox: file chunks stored by SHA256 hash. Deduplication: same content -> same hash -> same storage object. Git uses this for blobs and trees.

Q: 15. What is a bloom filter?

A: Probabilistic data structure: tests if element is possibly in set (true) or definitely not in set (false). No false negatives. Use case: cache penetration prevention (check bloom filter before DB query).

Q: 16. What is a distributed lock?

A: Coordinate exclusive access across multiple processes. Redis: SETNX key value PX timeout (or Redlock algorithm). ZooKeeper ephemeral nodes. Issues: lock expiry during long operations, process crash before unlock.

Q: 17. What is a coordination service?

A: ZooKeeper, etcd, Consul. Provides: distributed lock, leader election, service discovery, configuration management. Uses consensus algorithm (ZAB, Raft). Critical infrastructure — deploy in HA mode.

Q: 18. What is eventual consistency?

A: System will converge to consistent state over time (not immediately). Trade consistency for availability. DNS, shopping carts, social media likes. Design: idempotent operations, conflict resolution, read-your-writes.

Q: 19. What is a write-ahead log (WAL)?

A: Changes written to log before applying to DB. If crash: replay log to recover. Used by PostgreSQL, Kafka (commit log), LevelDB. Enables point-in-time recovery and replication.

Q: 20. What is a Merkle tree?

A: Binary tree where each leaf = hash of data, each node = hash of children. Efficient comparison: two Merkle trees identical if root hashes match. Detect exactly which leaves differ efficiently. Used in: Bitcoin, Cassandra anti-entropy, Git.

Q: 21. What is geo-replication?

A: Replicate data across geographic regions. Active-active: any region accepts writes (conflict resolution needed). Active-passive: primary region handles writes, passive is replica for reads/failover. Reduces latency for global users.

Q: 22. What is a leader election?

A: One node elected as leader to coordinate operations. Raft (etcd, CockroachDB), ZAB (ZooKeeper), Paxos. Leader handles writes; followers replicate. On leader failure: election with quorum vote.

Q: 23. What is a fanout approach for social media feeds?

A: On post: fan out to all followers' feed lists immediately (write-time). On read: fast (O(1) Redis list). Problem: celebrity with 100M followers = 100M writes. Hybrid: celebrities use pull model; regular users use push.

Q: 24. What is the thundering herd problem?

A: Many requests simultaneously hit backend (on cache miss, server restart, cron job). Overwhelms DB. Solutions: cache lock/single-flight (one loads, others wait), jitter in retries, probabilistic early expiry.

Q: 25. What is back-pressure?

A: Mechanism where downstream system signals upstream to slow down when overwhelmed. Queue fills up -> producer pauses. Reactive streams: request(n) backpressure. Prevents OOM from unbounded queue.

Q: 26. What is the shard nothing architecture?

A: Each node is fully independent: no shared disk, no shared memory, no shared state. Each handles its own data shard. Infinitely scalable. Examples: Cassandra, Dynamo.

Q: 27. What is the difference between OLTP and OLAP?

A: OLTP: transactional, many small reads/writes, normalized, millisecond latency. OLAP: analytical, few large queries, denormalized/columnar, seconds to minutes. Use different DB engines: PostgreSQL (OLTP) vs Redshift/BigQuery (OLAP).

Q: 28. What is data partitioning vs replication?

A: Partitioning (sharding): split data across nodes — each node has different data. Increases capacity and throughput. Replication: copy same data to multiple nodes — each has same data. Increases availability and read throughput.

Q: 29. What is a quorum read/write?

A: N replicas. W writes required to succeed (durability). R reads required (freshness). W + R > N ensures at least one node has latest value. Cassandra: configure consistency level per query (ONE, QUORUM, ALL).

Q: 30. What is an idempotency key?

A: Client-generated UUID identifying a specific operation. Server uses key to detect duplicates: if key seen before, return cached response instead of re-executing. Prevents double payments, double sends on network retry.

Q: 31. What is back-of-envelope estimation?

A: Rough capacity planning: daily requests -> RPS, data per request -> storage per day/year, read:write ratio -> read/write RPS. Powers of 2: 1 million = 2^20. Latency numbers: L1 cache 1ns, RAM 100ns, SSD 100us, network 1ms.

Q: 32. What is a time-series database?

A: Optimized for time-stamped data (metrics, IoT, financial). Efficient writes of sequential time-ordered data. Columnar storage, compression. Time-range queries fast. Examples: InfluxDB, TimescaleDB, Prometheus TSDB, OpenTSDB.

Q: 33. What is the two-generals problem?

A: Two armies must attack simultaneously but can only communicate via unreliable messenger. No protocol guarantees both generals confirm — illustrates impossibility of guaranteed consensus over unreliable network. Real implication: two-phase commit can block.

Q: 34. What is service degradation vs circuit breaker?

A: Service degradation: return reduced-quality response (cached, simplified, default) when dependency fails. Circuit breaker: stop calling failed service for a period, return fallback. Together: graceful degradation without cascading failure.

Q: 35. What is a streaming vs batch processing?

A: Batch: process accumulated data at intervals (hourly, daily). Simple, high throughput. Latency = batch interval. Streaming: process each event as it arrives. Near-real-time. More complex. Tools: Kafka Streams, Flink, Spark Streaming.

Q: 36. What is a data lake vs data warehouse?

A: Data lake: raw data in any format (S3, HDFS), schema on read, cheap. Data warehouse: structured, processed, schema on write, expensive, fast queries. Modern lakehouse (Delta Lake, Iceberg) combines both.

Q: 37. What is a service level objective (SLO)?

A: Target for service reliability: 99.9% availability, p99 latency < 200ms. SLA: contractual commitment with penalties. SLI: metric measuring compliance. Error budget = 1 - SLO target = allowed downtime.

Q: 38. What is rate of change vs volume of data for storage decisions?

A: High rate of change + high volume: consider append-only stores (Kafka, event sourcing). High volume + low change: object storage (S3). High read/write, low latency: Redis. High volume relational: PostgreSQL + read replicas + sharding.

Q: 39. What is the retry with exponential backoff pattern?

A: On transient failure: wait 1s, retry; wait 2s, retry; wait 4s, retry... with jitter (add random delay to avoid synchronized retries). Max retries = 5-7. Only retry idempotent operations. Circuit breaker after sustained failures.

Q: 40. What is a checkpoint in stream processing?

A: Periodic snapshot of stream processing state. If failure: resume from last checkpoint instead of start. Kafka: committed offsets. Flink: distributed snapshots. Reduces reprocessing on failure.

18. Data Structures & Algorithms [35 Questions]

Q: 1. What is Big-O notation?

A: Asymptotic notation describing algorithm time/space complexity in worst case. O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, O(n^2) quadratic, O(2^n) exponential. Ignore constants and lower-order terms.

Q: 2. What is amortized complexity?

A: Average time per operation over a sequence. ArrayList.add(): usually O(1), occasionally O(n) for resize. Amortized: O(1). HashMap: O(1) amortized despite occasional O(n) resize.

Q: 3. What is a Binary Search Tree (BST)?

A: Tree where left child < node < right child. Search/insert/delete: O(h) where h = height. Balanced BST (AVL, Red-Black): O(log n). Degenerate (sorted input): O(n). Java: TreeMap/TreeSet use Red-Black Tree.

Q: 4. What is a heap?

A: Complete binary tree satisfying heap property. Min-heap: parent <= children. Max-heap: parent >= children. peek/top: O(1). insert/delete: O(log n). Build heap: O(n). Used for priority queues and heap sort.

Q: 5. What is dynamic programming?

A: Optimization technique: break problem into overlapping subproblems, solve each once, store results (memoization/tabulation). Apply when: optimal substructure + overlapping subproblems. Examples: Fibonacci, knapsack, LCS, LIS.

Q: 6. What is memoization vs tabulation?

A: Memoization (top-down): recursive + cache results. Natural problem decomposition but stack overhead. Tabulation (bottom-up): iterative, fill DP table from base case. More efficient, no stack overflow.

Q: 7. What is a trie?

A: Prefix tree for strings. Each node = character. Root to node = prefix. Common prefix sharing. O(L) insert/search (L=string length). Use for: autocomplete, spell check, IP routing (radix trie).

Q: 8. What is graph BFS vs DFS?

A: BFS: level-by-level, uses queue, shortest path in unweighted graph, O(V+E). DFS: depth-first, uses stack/recursion, topological sort, cycle detection, connected components, O(V+E).

Q: 9. What is Dijkstra's algorithm?

A: Shortest path in weighted graph (non-negative weights). Priority queue (min-heap). Greedy: always process closest unvisited node. O((V+E) log V). Use when: positive weights. Bellman-Ford for negative weights.

Q: 10. What is the two-pointer technique?

A: Two pointers moving toward/away from each other to reduce O(n^2) to O(n). Examples: pair sum in sorted array, remove duplicates, palindrome check, container with most water. One of the most useful coding interview patterns.

Q: 11. What is the sliding window technique?

A: Maintain a window [l, r] over array/string. Expand right, shrink left when condition violated. O(n). Examples: max sum subarray of size k, longest substring without repeating chars, minimum window substring.

Q: 12. What is the binary search pattern?

A: O(log n) search on sorted array. Template: find leftmost/rightmost position, search on answer (minimum feasible value). Not just for sorted arrays — also for: rotated array, matrix, answer space problems.

Q: 13. What is Kadane's algorithm?

A: Maximum subarray sum. dp[i] = max(arr[i], dp[i-1] + arr[i]). Track global max. O(n), O(1) space. Also find start/end indices of max subarray.

Q: 14. What is fast and slow pointer (Floyd's cycle detection)?

A: Two pointers: fast moves 2 steps, slow moves 1. They meet if cycle exists. Used for: linked list cycle detection, finding middle of list, detecting cycle start.

Q: 15. What is a monotonic stack?

A: Stack maintaining monotonically increasing or decreasing values. O(n). Used for: next greater element, largest rectangle in histogram, daily temperatures, trapping rain water.

Q: 16. What is topological sort?

A: Linear ordering of DAG vertices where for each edge u->v, u comes before v. Kahn's algorithm: BFS with in-degree. DFS: add to result after finishing DFS from node. Used for: build systems, course scheduling.

Q: 17. What is union-find (disjoint set)?

A: Data structure tracking which elements are in the same set. find(x): returns set representative. union(x, y): merges sets. With path compression + rank: near O(1) amortized. Used for: connected components, MST (Kruskal's).

Q: 18. What is the difference between BFS and Dijkstra?

A: BFS finds shortest path by hops (edges) in unweighted graph. Dijkstra finds shortest path by weight in weighted graph. BFS = Dijkstra with all weights = 1.

Q: 19. What is a segment tree?

A: Tree for range queries and point updates. Build: O(n). Query/update: O(log n). Used for: range sum, range min/max, range GCD. More complex but much faster than prefix sums for updates.

Q: 20. What is a Fenwick tree (Binary Indexed Tree)?

A: Compact tree for prefix sum queries and point updates. O(log n) update and query. More cache-friendly and simpler to implement than segment tree. Limitation: additive operations only (not min/max without modification).

Q: 21. What is the backtracking pattern?

A: Explore all possibilities recursively. Make choice, recurse, undo choice. Used for: permutations, combinations, N-queens, Sudoku, word search. Prune early (constraint checking) to avoid exploring dead branches.

Q: 22. What is prefix sum?

A: preSum[i] = sum(arr[0..i-1]). Range sum [l, r] = preSum[r+1] - preSum[l]. O(n) build, O(1) query. Extension: 2D prefix sum for matrix. Used for: subarray sum equals k, equilibrium index.

Q: 23. What is merge sort?

A: Divide and conquer. Split array in half recursively until size 1. Merge sorted halves. O(n log n) time, O(n) space. Stable sort. Best for linked lists. Used internally by Java's Arrays.sort for objects (TimSort).

Q: 24. What is quicksort?

A: Pick pivot, partition array (elements < pivot left, > pivot right). Recurse on both sides. O(n log n) average, O(n^2) worst (poor pivot). In-place, not stable. Java uses dual-pivot quicksort for primitives.

Q: 25. What is a hash table collision resolution?

A: Chaining: bucket holds linked list of colliding elements (HashMap). Open addressing: probe other slots (linear probing, quadratic, double hashing). Robin Hood hashing: steal from richer buckets.

Q: 26. What is the LRU cache implementation?

A: Combine HashMap + doubly linked list. HashMap: O(1) lookup. LinkedList: O(1) insertion/deletion at any position. get(): update access order (move to tail). put(): add to tail, evict head if capacity exceeded.

Q: 27. What is a balanced BST operation?

A: AVL tree: maintains |height(left) - height(right)| <= 1 via rotations after insert/delete. Red-Black tree: 5 properties, 2 rotation types, used by TreeMap/HashMap tree buckets. Both guarantee O(log n).

Q: 28. What is the difference between DFS preorder, inorder, postorder?

A: Preorder: node, left, right. Inorder: left, node, right (gives sorted order for BST). Postorder: left, right, node (delete tree, evaluate expression tree). Level-order = BFS.

Q: 29. What is graph coloring?

A: Assign colors to vertices such that no adjacent vertices share color. Minimum colors = chromatic number. NP-hard in general. Bipartite check: can graph be 2-colored? BFS with alternating colors.

Q: 30. What is the knapsack problem?

A: 0/1 Knapsack: choose items with weights/values to maximize value within weight capacity. O(n*W) DP. Each item: include (value[i] + dp[j-weight[i]]) or exclude (dp[j]). Classic DP interview problem.

Q: 31. What is string matching algorithms?

A: Naive: O(nm). KMP (Knuth-Morris-Pratt): O(n+m) using failure function. Rabin-Karp: rolling hash O(n+m) average. Z-algorithm: O(n+m). Boyer-Moore: sub-linear average.

Q: 32. What is a heap sort?

A: Build max-heap O(n). Repeatedly extract max, place at end, heapify O(log n). Total: O(n log n). In-place (O(1) extra space). Not stable. Rarely used in practice (worse cache performance than quicksort).

Q: 33. What is the difference between greedy and dynamic programming?

A: Greedy: locally optimal choice at each step, hopes for global optimal. Fast but not always correct. DP: considers all subproblems, guaranteed optimal. Greedy works for: interval scheduling, Huffman coding, Dijkstra.

Q: 34. What is Bellman-Ford algorithm?

A: Shortest path with negative weights. Relax all edges V-1 times. O(VE). Detects negative cycles (extra relaxation round). Slower than Dijkstra but handles negative weights.

Q: 35. What is Floyd-Warshall algorithm?

A: All-pairs shortest paths. dp[i][j][k] = shortest path from i to j using nodes 0..k. O(V^3). Simple code but cubic time. Use for small graphs or when all-pairs distances needed.

19. Relational Databases & SQL [35 Questions]

Q: 1. What are ACID properties?

A: Atomicity: all-or-nothing. Consistency: transaction brings DB from valid to valid state. Isolation: concurrent transactions don't interfere. Durability: committed data survives failure.

Q: 2. What is normalization?

A: Organizing tables to reduce redundancy. 1NF: atomic values, unique rows. 2NF: no partial dependencies (all non-key cols depend on full PK). 3NF: no transitive dependencies. BCNF: every determinant is a candidate key.

Q: 3. What is denormalization?

A: Intentionally introducing redundancy for query performance. Stores duplicated data to avoid joins. Trade-off: faster reads, slower writes, data inconsistency risk. Common in OLAP/data warehouses.

Q: 4. What are SQL indexes?

A: B-Tree index: default, good for range queries and equality. Hash index: equality only, O(1). Composite index: multiple columns — column order matters (leftmost prefix rule). Covering index: includes all columns query needs.

Q: 5. What is a covering index?

A: Index contains all columns the query needs. No table lookup needed (index-only scan). Example: SELECT name, email FROM users WHERE email = ? — index on (email, name) covers this query.

Q: 6. What is the EXPLAIN statement?

A: Shows query execution plan: table access method (Seq Scan, Index Scan, Index Only Scan, Bitmap), join strategy (Nested Loop, Hash Join, Merge Join), estimated cost/rows. Use EXPLAIN ANALYZE for actual runtime stats.

Q: 7. What are SQL isolation levels?

A: READ UNCOMMITTED: dirty read possible. READ COMMITTED: no dirty read, non-repeatable read possible. REPEATABLE READ: no dirty/non-repeatable read, phantom read possible (MySQL default). SERIALIZABLE: all anomalies prevented.

Q: 8. What is a dirty read?

A: Reading uncommitted data from another transaction. Only possible at READ UNCOMMITTED. If that transaction rolls back: you read data that never officially existed.

Q: 9. What is a non-repeatable read?

A: Read same row twice in same transaction, get different values (another transaction updated and committed between reads). Prevented at REPEATABLE READ and above.

Q: 10. What is a phantom read?

A: Query returning different set of rows on second execution (another transaction inserted/deleted rows matching WHERE). Prevented at SERIALIZABLE isolation.

Q: 11. What is a SELECT FOR UPDATE?

A: Pessimistic row-level lock. Prevents other transactions from updating or locking selected rows until current transaction commits/rolls back. Used for: read-modify-write patterns (inventory, balances). SELECT ... FOR SHARE for read locks.

Q: 12. What is a deadlock in SQL?

A: Two transactions each hold a lock the other needs. DB detects and kills one (victim). Fix: always acquire locks in consistent order, use shorter transactions, use SELECT FOR UPDATE only when necessary.

Q: 13. What is a gap lock?

A: Locks the gap between index records. Prevents phantom reads in REPEATABLE READ (MySQL InnoDB). Can cause unexpected blocking. Disable with READ COMMITTED isolation if phantoms acceptable.

Q: 14. What is a CTE (Common Table Expression)?

A: Named temporary result set defined with WITH clause. Readable alternative to subqueries. Recursive CTE: WITH RECURSIVE — used for hierarchical data (org charts, tree traversal). Multiple CTEs composable.

Q: 15. What is a window function?

A: Performs calculation across rows related to current row without collapsing to group. ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER(PARTITION BY ... ORDER BY ...). Runs after WHERE, GROUP BY, HAVING.

Q: 16. What is the difference between WHERE and HAVING?

A: WHERE: filters rows before GROUP BY. HAVING: filters groups after GROUP BY. WHERE is faster (eliminates rows before aggregation). Cannot use aggregate functions in WHERE.

Q: 17. What is an index on a JOIN column?

A: Both joined columns should be indexed. DB uses nested loop or hash join. Without index on join column: full table scan per outer row = O(n*m). With index: O(n log m).

Q: 18. What is the N+1 problem in SQL?

A: For each of N parent records, execute a separate query for related data = N+1 queries total. Fix: JOIN or subquery to fetch all related data in one query. ORM fix: JOIN FETCH, @EntityGraph, @BatchSize.

Q: 19. What is connection pooling?

A: Reuse DB connections instead of creating new ones per request. Creating connection is expensive (~100ms). Pool holds N open connections; requests borrow and return them. HikariCP, c3p0, DBCP are popular Java pools.

Q: 20. What is a stored procedure?

A: Named precompiled SQL block stored in DB. Called by name. Advantages: precompiled, network traffic reduction, centralized logic. Disadvantages: hard to test, version control, language-specific, tight coupling to DB.

Q: 21. What is an index cardinality?

A: Number of unique values in indexed column. High cardinality (userId): index very selective, efficient. Low cardinality (boolean active): index rarely beneficial — full scan often faster. Rule of thumb: index columns with > 20% unique values.

Q: 22. What is a composite index?

A: Index on multiple columns: (status, created_at). Most selective column first. Leftmost prefix rule: index used for queries on (status) or (status, created_at) but not (created_at) alone.

Q: 23. What is a partial index?

A: Index on subset of rows meeting a condition: CREATE INDEX ON orders(status) WHERE status = 'PENDING'. Smaller index, faster for that specific filter. Not all DBs support it (PostgreSQL does, MySQL doesn't natively).

Q: 24. What is a foreign key?

A: Column referencing primary key of another table. Enforces referential integrity: cannot insert FK value that doesn't exist in parent, or delete parent row with children. Index FK columns for JOIN performance.

Q: 25. What is a UNIQUE constraint vs UNIQUE index?

A: Functionally equivalent — both enforce uniqueness and create an index. UNIQUE constraint is semantically clearer (DDL intent). Composite unique: UNIQUE(user_id, product_id) for no duplicate entries.

Q: 26. What is database sharding?

A: Horizontally partition data across multiple DB instances by shard key. Each shard has subset of data. Routes queries to correct shard. Challenges: cross-shard joins, transactions, schema changes at scale.

Q: 27. What is a materialized view?

A: Precomputed, stored query result. Unlike views, data is physically stored. Must be refreshed (manually or on schedule). Faster reads for complex aggregate queries. Cost: storage + refresh complexity.

Q: 28. What is the difference between UNION and UNION ALL?

A: UNION: removes duplicates (sorts/hashes to deduplicate). UNION ALL: keeps all rows, no deduplication. UNION ALL is faster when duplicates are acceptable or impossible.

Q: 29. What is a lateral join?

A: JOIN LATERAL: subquery in join references columns from preceding table (like a correlated subquery but for each row). PostgreSQL, MySQL 8+. Example: for each user, get their last 3 orders via LATERAL.

Q: 30. What is MVCC (Multi-Version Concurrency Control)?

A: DB maintains multiple versions of rows. Readers see consistent snapshot without blocking writers. Writers don't block readers. PostgreSQL, Oracle, MySQL InnoDB use MVCC. Eliminates read-write contention.

Q: 31. What is a B-Tree index structure?

A: Balanced tree where leaf nodes contain data pointers and values. Internal nodes contain separator keys. O(log n) lookup, range scans efficient (leaf nodes linked). Stays balanced on insert/delete via splits/merges.

Q: 32. What is tablespace?

A: Logical storage grouping in database. Assign tables/indexes to specific tablespaces (different disks). PostgreSQL: CREATE TABLESPACE fast LOCATION '/ssd'. Useful for separating hot and cold data by storage type.

Q: 33. What is VACUUM in PostgreSQL?

A: Reclaims space from dead tuples created by MVCC (old versions not needed). VACUUM ANALYZE: also updates table statistics. AUTOVACUUM: runs automatically. Without vacuum: table bloat, performance degradation.

Q: 34. What is a row estimate and why does it matter for query plans?

A: DB optimizer uses table statistics (from ANALYZE) to estimate rows per operation. Wrong estimates lead to bad plans (wrong join order, hash vs nested loop). Stale stats = bad plans. Run ANALYZE after large data changes.

Q: 35. What is the difference between OLTP and OLAP databases?

A: OLTP (PostgreSQL, MySQL): normalized, many small transactions, index-heavy, row-oriented storage. OLAP (Redshift, BigQuery, ClickHouse): denormalized, few large analytical queries, columnar storage, parallel scans. Columnar greatly speeds aggregation.

20. NoSQL Databases [35 Questions]

Q: 1. What is NoSQL?

A: Non-relational databases: document, key-value, wide-column, graph. Flexible schema, horizontal scalability, eventual consistency. Choose when: flexible schema needed, massive scale, specific access patterns.

Q: 2. What are NoSQL types?

A: Document (MongoDB): JSON documents. Key-Value (Redis, DynamoDB): hash map. Wide-Column (Cassandra, HBase): rows with dynamic columns. Graph (Neo4j): nodes and edges. Each optimized for specific access patterns.

Q: 3. What is MongoDB?

A: Document database. BSON (binary JSON) documents. Dynamic schema per document. Ad-hoc queries, secondary indexes, aggregation pipeline. ACID transactions (multi-document since 4.0). Replica sets for HA.

Q: 4. What is the MongoDB aggregation pipeline?

A: Processing pipeline: $match (filter), $group (aggregate), $project (reshape), $sort, $limit, $lookup (join), $unwind (array expansion), $addFields. Chainable stages. Runs on server.

Q: 5. What is embedding vs referencing in MongoDB?

A: Embedding: nest related data in same document. Fast reads, no join. But: document size limit (16MB), update complexity. Referencing: store ID, use $lookup for join. Normalizing for frequently updated data.

Q: 6. What is Cassandra?

A: Wide-column, distributed, masterless. Designed for write-heavy, high-availability workloads. Data model: keyspace -> table -> partition key + clustering columns. Tunable consistency. No joins or transactions (traditionally).

Q: 7. What is a Cassandra partition key?

A: Determines which node stores the row (hash of partition key -> token range -> node). All rows with same partition key stored together (efficient range scans within partition). Choose key to avoid hot partitions.

Q: 8. What is a clustering column in Cassandra?

A: Columns that determine sort order within a partition. Range queries within partition use clustering columns. Example: (userId, timestamp) — userId is partition key, timestamp is clustering column — efficient time-range queries per user.

Q: 9. What is Cassandra consistency level?

A: Per-query: ONE (any one replica), QUORUM (majority), ALL (all replicas), LOCAL_QUORUM (majority in local DC). Higher consistency: slower, less available. QUORUM writes + QUORUM reads ensures latest data.

Q: 10. What is Redis?

A: In-memory key-value store with rich data structures: String, List, Hash, Set, Sorted Set, Stream, HyperLogLog, Bitfield, JSON. Sub-ms latency. Persistence: RDB snapshots, AOF log. Pub/Sub. Distributed with Cluster.

Q: 11. What is Redis Sorted Set?

A: Set where each member has a score (float). Members sorted by score. ZADD, ZRANK, ZRANGE, ZRANGEBYSCORE, ZREVRANK. O(log n) add/remove. Used for: leaderboards, rate limiting (sliding window), time-based queues.

Q: 12. What is Redis persistence?

A: RDB: periodic snapshots (fork + dump). Fast restart from compact file. Risk: data since last snapshot lost on crash. AOF: append-only log of write commands. Safer, slower, larger file. Hybrid: RDB + AOF.

Q: 13. What is Redis Sentinel?

A: Monitors Redis primary/replica, auto-promotes replica on primary failure. Requires 3+ Sentinels for quorum. Clients connect via Sentinel for automatic failover handling. Single-shard HA solution.

Q: 14. What is Redis Cluster?

A: Horizontal sharding across 16,384 hash slots. Each master owns a slot range; replicates to slaves. Gossip protocol for cluster state. Cross-slot operations limited. Hash tags: {key}:field forces same slot.

Q: 15. What is DynamoDB?

A: AWS-managed NoSQL. Table with composite key: partition key (mandatory) + sort key (optional). GSI/LSI for alternate access patterns. DynamoDB Streams for change events. Point-in-time recovery. Auto-scaling.

Q: 16. What is a GSI in DynamoDB?

A: Global Secondary Index: index with different partition/sort key than table. Eventually consistent reads. Can be added after table creation. Enables querying by non-primary-key attributes.

Q: 17. What is a DynamoDB projection?

A: Attributes projected into an index: ALL, KEYS_ONLY, or specific attributes. Fewer projected attributes = smaller index = lower cost. Query returns only projected attributes; additional attributes require table fetch (expensive).

Q: 18. What is Elasticsearch?

A: Distributed search/analytics engine based on Lucene. Documents indexed in shards. Full-text search (inverted index), aggregations, geo-search. RESTful API. Used for: search, logging (ELK stack), analytics.

Q: 19. What is an inverted index?

A: Maps terms to documents containing them. Core of full-text search. term -> [doc1, doc2, doc3]. Query: find intersection of term lists. Scoring: TF-IDF or BM25 ranks by relevance.

Q: 20. What is Elasticsearch mapping?

A: Schema definition for index: field types (keyword, text, integer, date, geo_point). Text: analyzed (tokenized, lowercased). Keyword: exact match, aggregations. Once created, cannot change field type (must reindex).

Q: 21. What is the difference between NoSQL BASE vs ACID?

A: ACID: strong consistency, transactions. BASE: Basically Available, Soft state, Eventually consistent. Most NoSQL favors BASE for scalability. Modern NoSQL (MongoDB 4.0+, DynamoDB with transactions) adds ACID transactions.

Q: 22. What is CAP theorem applied to NoSQL?

A: MongoDB: CP (consistent, partition-tolerant, may reject writes during partition). Cassandra: AP (available, partition-tolerant, eventually consistent). Redis: CA without partition tolerance (single node); CP in Cluster mode.

Q: 23. What is horizontal scaling of NoSQL?

A: Add more nodes to distribute load. Each node handles a subset of data (sharding). No single point of bottleneck. Consistent hashing used for key distribution. Scales to petabytes (Cassandra, DynamoDB, MongoDB Atlas).

Q: 24. What is document TTL in MongoDB?

A: TTL index on date field: db.sessions.createIndex({createdAt:1},{expireAfterSeconds:3600}). MongoDB automatically deletes documents after TTL. Used for: session expiry, temporary data, cache.

Q: 25. What is a capped collection in MongoDB?

A: Fixed-size circular buffer. Overwrites oldest documents when full. Natural insertion order maintained. Fast writes. Used for: activity logs, message buffers, audit trails where old data can be lost.

Q: 26. What is Cassandra's LWT (Lightweight Transaction)?

A: Compare-and-swap using Paxos consensus. IF NOT EXISTS or IF condition. Serializable per partition. Much slower than normal writes (roundtrip to all Paxos participants). Use sparingly: user registration uniqueness, seat reservation.

Q: 27. What is Redis HyperLogLog?

A: Probabilistic data structure for counting unique elements. Uses ~12KB regardless of set size. PFADD, PFCOUNT. ~0.81% error rate. Use for: approximate unique visitors, unique search terms, without storing all values.

Q: 28. What is the MongoDB replica set?

A: Group of MongoDB nodes: one primary (writes), multiple secondaries (reads, HA). Automatic failover via election if primary fails. writeConcern: w:majority ensures durability. readPreference: secondaryPreferred for read scaling.

Q: 29. What is graph database?

A: Optimized for relationship-heavy data. Nodes (entities) + Edges (relationships) + Properties. Neo4j uses Cypher query language. Traversals are fast regardless of dataset size. Use for: social networks, fraud detection, recommendations, knowledge graphs.

Q: 30. What is the difference between MongoDB $push and $addToSet?

A: $push: appends element to array (allows duplicates). $addToSet: appends only if not already present (set semantics). Both are atomic update operators for array fields.

Q: 31. What is read/write concern in MongoDB?

A: writeConcern: how many nodes must acknowledge write before success (w:1 primary only, w:majority quorum). readConcern: consistency level for reads (local, available, majority, linearizable). Higher concern = more consistency, more latency.

Q: 32. What is Elasticsearch shard and replica?

A: Index divided into primary shards (parallelism). Each primary has 0+ replicas (HA + read scalability). Default: 1 primary, 1 replica. Primary shard count fixed at index creation. Replica count changeable later.

Q: 33. What is a time-series collection in MongoDB?

A: MongoDB 5.0+: optimized collection type for time-series data. Automatically bucketed and compressed. Efficient storage and queries for metrics, IoT, financial data. Define timeField, metaField, granularity.

Q: 34. What is Bigtable?

A: Google's wide-column distributed storage. Basis for HBase and Cassandra design. Cells: (rowkey, column family, column qualifier, timestamp) -> value. Sorted by rowkey. Designed for petabyte scale. Used for: Google Search, Gmail, YouTube.

Q: 35. What is the Redis SETNX command?

A: SET if Not Exists: SETNX key value. Returns 1 if set (key didn't exist), 0 if key already exists (not set). Atomic. Used for distributed locking (with PX for expiry). Modern: SET key value NX PX milliseconds (atomic with TTL).

21. Unit Testing & Test Automation [31 Questions]

Q: 1. What is the testing pyramid?

A: Unit tests (many, fast, isolated), Integration tests (moderate, test interactions), E2E tests (few, slow, full system). More units: cheap feedback. Fewer E2E: expensive, brittle. Prioritize units.

Q: 2. What is unit vs integration vs E2E test?

A: Unit: tests single class/method in isolation (mocks dependencies). Integration: tests interaction between components (real DB, real service). E2E: tests full user journey (browser, UI, full stack).

Q: 3. What is JUnit 5?

A: Jupiter testing framework. @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll, @Disabled, @DisplayName, @Nested, @ParameterizedTest, @Tag. Assertions: assertEquals, assertThrows, assertAll.

Q: 4. What is @ParameterizedTest?

A: Run same test with multiple inputs. Sources: @ValueSource, @CsvSource, @MethodSource, @EnumSource, @ArgumentsSource. Reduces test duplication for equivalence partitions.

Q: 5. What is Mockito?

A: Mocking framework. @Mock: create mock. @InjectMocks: inject mocks into SUT. @Spy: partial mock (real object with some methods mocked). @Captor: capture method arguments. when().thenReturn() for stubbing.

Q: 6. What is the difference between @Mock and @Spy?

A: @Mock: all methods return defaults (null, 0, false) unless stubbed. @Spy: wraps real object — real methods called unless stubbed. Use Spy for partial mocking of real implementation.

Q: 7. What is verify() in Mockito?

A: verify(mock).method(args) asserts method was called. verify(mock, times(2)), never(), atLeastOnce(), atMost(n). Captures interaction rather than state. Use ArgumentCaptor for verifying complex arguments.

Q: 8. What is ArgumentCaptor?

A: Captures arguments passed to mocked methods: captor.capture() in verify, captor.getValue() to inspect. Example: verify(emailService).send(captor.capture()); assertEquals('alice@co', captor.getValue().getTo()).

Q: 9. What is @ExtendWith(MockitoExtension.class)?

A: JUnit 5 extension that activates Mockito annotations (@Mock, @InjectMocks, @Spy, @Captor). Replaces MockitoAnnotations.openMocks(this) in @BeforeEach.

Q: 10. What is assertThrows in JUnit 5?

A: Asserts exception thrown: Exception ex = assertThrows(IllegalArgumentException.class, () -> service.method(-1)); assertEquals('must be positive', ex.getMessage());

Q: 11. What is assertAll in JUnit 5?

A: Groups multiple assertions — all are evaluated even if some fail. Reports all failures at once: assertAll(() -> assertEquals(a, b), () -> assertNotNull(c), () -> assertTrue(d)).

Q: 12. What is TestNG?

A: Testing framework (alternative to JUnit). Groups, dependencies (@Test(dependsOnMethods)), data providers, parallel execution, more configuration. Still used in some enterprise projects.

Q: 13. What is TestNG DataProvider?

A: @DataProvider(name='data') returns Object[][] with test data. @Test(dataProvider='data') uses it. Similar to JUnit @ParameterizedTest but more flexible for complex objects.

Q: 14. What is Spring @SpringBootTest?

A: Full application context integration test. Loads all beans. @Autowired works normally. Slow but comprehensive. Specify webEnvironment for web layer: NONE, MOCK, RANDOM_PORT, DEFINED_PORT.

Q: 15. What is MockMvc?

A: Test MVC layer without starting server. perform(post('/api').content(json).contentType(APPLICATION_JSON)).andExpect(status().isCreated()).andExpect(jsonPath('$.id').exists()). Fast integration test.

Q: 16. What is Testcontainers?

A: Java library that starts Docker containers for tests. Use real PostgreSQL, Redis, Kafka in tests. @Testcontainers + @Container + GenericContainer or specific containers (PostgreSQLContainer). Slower but realistic.

Q: 17. What is test isolation?

A: Each test is independent — doesn't depend on order, doesn't share state. @BeforeEach resets state. @Transactional + rollback for DB tests. Separate test data per test. Prevents flaky tests.

Q: 18. What is a flaky test?

A: Test that passes sometimes and fails other times without code changes. Causes: timing issues, shared state, random data, network calls. Fix: mock external dependencies, use awaitility for async, fix concurrency issues.

Q: 19. What is the AAA pattern?

A: Arrange (set up test data and mocks), Act (call the method under test), Assert (verify result). Clearly separates test phases. Makes tests readable and maintainable.

Q: 20. What is WireMock?

A: Stub HTTP server for testing. Record/replay real service interactions or define custom stubs. WireMock.stubFor(get('/api').willReturn(aResponse().withBody(json))). Test without real external service.

Q: 21. What is the difference between stub and mock?

A: Stub: returns predefined data (when().thenReturn()). Mock: verifies interactions (verify()). Mockito objects are both — but semantics differ. Use stub for state-based testing, mock for interaction-based.

Q: 22. What is code coverage?

A: Percentage of code executed by tests. Line coverage, branch coverage, method coverage. JaCoCo: most popular Java coverage tool. Target: >80% meaningful coverage. Don't test private methods directly; don't chase 100%.

Q: 23. What is mutation testing?

A: Automatically introduces bugs (mutations) into code and checks if tests catch them (kill mutations). PiTest for Java. High mutation score = tests actually verify behavior. Tests that don't kill mutations are weak.

Q: 24. What is Awaitility?

A: Library for testing asynchronous code: await().atMost(5, SECONDS).until(() -> result.isDone()). Polls condition until true or timeout. Cleaner than Thread.sleep() in async tests.

Q: 25. What is @MockBean in Spring Boot tests?

A: Replaces bean in Spring context with Mockito mock. Used in slice tests (@WebMvcTest, @SpringBootTest). Different from @Mock (which doesn't interact with Spring context).

Q: 26. What is a test double?

A: Generic term for all test replacements: dummy (placeholder), stub (returns canned data), spy (records calls), mock (verifies interactions), fake (working but simplified implementation, e.g., in-memory DB).

Q: 27. What is BDD (Behavior Driven Development)?

A: Tests written as: Given (context), When (action), Then (outcome). Readable by non-developers. Cucumber: Gherkin syntax feature files executed by Java step definitions. JUnit 5 @DisplayName for BDD-style names.

Q: 28. What is contract testing (Pact)?

A: Consumer defines expected API contract. Provider verifies it fulfills the contract. Detects breaking API changes before deployment. Runs as part of CI. Consumer publishes pacts; provider verifies.

Q: 29. What is a TestConfiguration?

A: @TestConfiguration: test-only Spring configuration. Beans defined here are added to context only in tests. Use to override production beans or define test-specific beans without polluting main configuration.

Q: 30. What is @DirtiesContext?

A: Marks that test has modified Spring application context — forces rebuild for subsequent tests. Very slow — avoid if possible. Fix root cause (don't modify context state) instead of using @DirtiesContext.

Q: 31. What is property-based testing?

A: Generate random inputs to find edge cases. jqwik (JUnit 5 integration) or QuickCheck-style. @Property with @ForAll parameters. Framework generates many values, shrinks failing case to minimal reproducer.

22. Design Patterns [35 Questions]

Q: 1. What is the Singleton pattern?

A: Ensures one instance per JVM. Thread-safe lazy init: private static volatile instance; double-checked locking with synchronized. Best: enum singleton (thread-safe, serialization-safe). Use for: connection pools, config, loggers.

Q: 2. What is the Factory Method pattern?

A: Defines interface for creating objects; subclasses decide which class to instantiate. Decouples client from concrete classes. Example: DocumentFactory.createDocument() returns PDF or Word based on type param.

Q: 3. What is the Abstract Factory pattern?

A: Creates families of related objects. GUIFactory creates Button + Checkbox. WindowsFactory creates WindowsButton + WindowsCheckbox. Client uses factory interface, not concrete classes. More complex than Factory Method.

Q: 4. What is the Builder pattern?

A: Constructs complex objects step by step. Fluent API: Person.builder().name('Alice').age(30).build(). Immutable result. Avoids telescoping constructors. Lombok @Builder auto-generates. Used in: StringBuilder, Stream.builder().

Q: 5. What is the Prototype pattern?

A: Create new objects by copying existing ones. Implements Cloneable.clone() (shallow) or deep copy constructor. Useful when object creation is expensive. Java: Object.clone() is problematic — prefer copy constructors.

Q: 6. What is the Adapter pattern?

A: Converts interface of class to another interface clients expect. Class adapter (extends adaptee) or object adapter (wraps adaptee). Example: InputStream to Reader adapter. Used when integrating third-party code.

Q: 7. What is the Decorator pattern?

A: Adds behavior dynamically without modifying object. Wraps object and delegates to it, adding behavior before/after. Java I/O: BufferedReader(new FileReader()). Spring: AOP proxies are decorators. Flexible alternative to subclassing.

Q: 8. What is the Proxy pattern?

A: Surrogate that controls access to real object. Types: Virtual proxy (lazy load), Protection proxy (access control), Remote proxy (network call), Caching proxy. Spring AOP creates proxies for @Transactional, @Cacheable.

Q: 9. What is the Facade pattern?

A: Simplified interface to complex subsystem. Hides subsystem complexity. Service classes that coordinate repositories and other services are facades. Reduces dependencies on internals. Example: OrderService hiding inventory + payment + notification.

Q: 10. What is the Composite pattern?

A: Treats individual objects and groups uniformly. Tree structure: Component interface implemented by Leaf and Composite (contains Components). Example: filesystem (File and Directory both implement FileSystemNode).

Q: 11. What is the Strategy pattern?

A: Define family of algorithms, encapsulate each, make interchangeable. Example: SortStrategy with BubbleSort, QuickSort, MergeSort. Pass strategy to context. Replaces if-else/switch. Aligns with Open-Closed Principle.

Q: 12. What is the Observer pattern?

A: Subject notifies registered observers on state change. Subject.addObserver(), removeObserver(), notifyObservers(). Java: java.util.Observable (deprecated), PropertyChangeListener, Spring ApplicationEventPublisher, ReactiveX.

Q: 13. What is the Command pattern?

A: Encapsulates request as an object. Fields: receiver + action params. execute() and undo(). Queue commands, log commands, support undo. Example: GUI action buttons, macro recording, job queue.

Q: 14. What is the Template Method pattern?

A: Defines algorithm skeleton in base class; subclasses fill in steps. Final method calls abstract/overridable hook methods. Example: JdbcTemplate, AbstractBeanFactory.getBean(). Hollywood Principle: don't call us, we'll call you.

Q: 15. What is the Chain of Responsibility pattern?

A: Pass request along chain of handlers until one handles it. Each handler decides to process or pass to next. Example: Servlet filters, Spring Security filter chain, logging handlers. Decouples sender from receivers.

Q: 16. What is the State pattern?

A: Object changes behavior based on internal state. State interface with methods. Context delegates to current State object. Example: Order (PENDING -> CONFIRMED -> SHIPPED -> DELIVERED). Replaces state-based if-else.

Q: 17. What is the Iterator pattern?

A: Sequential access to elements without exposing internal structure. Java: Iterable + Iterator. for-each uses Iterator internally. Custom collections implement Iterable. Supports multiple simultaneous traversals.

Q: 18. What is the Mediator pattern?

A: Encapsulates interactions between objects. Objects communicate via mediator, not directly. Reduces coupling. Example: ChatRoom mediating messages between users. Similar to event bus or message broker.

Q: 19. What is the Flyweight pattern?

A: Share objects to minimize memory when many similar objects exist. Intrinsic state (shared) vs extrinsic state (per instance). Example: Java String pool. Integer cache (-128 to 127). Character cache in JVM.

Q: 20. What is the Bridge pattern?

A: Separates abstraction from implementation so both vary independently. Abstraction contains reference to implementor. Example: Shape abstraction + DrawingAPI implementor. Avoids Cartesian product class explosion.

Q: 21. What is the Memento pattern?

A: Captures and restores object state without violating encapsulation. Originator creates Memento. Caretaker stores it. Used for: undo functionality, snapshots, state rollback. Game save state.

Q: 22. What is the Visitor pattern?

A: Add operations to object hierarchy without modifying classes. Visitor interface: visit(ConcreteA), visit(ConcreteB). Elements accept(Visitor). Double dispatch. Complex but powerful for adding operations to stable hierarchies.

Q: 23. What is the Repository pattern?

A: Mediates between domain and data mapping. Collection-like interface for accessing domain objects. Abstracts DB queries. Implemented by Spring Data JPA. Domain code doesn't know if data comes from DB, cache, or file.

Q: 24. What is the CQRS pattern?

A: Command Query Responsibility Segregation. Separate models for reads and writes. Write model: handles commands (changes state). Read model: handles queries (returns data, often denormalized). Enables independent scaling.

Q: 25. What is the Event Sourcing pattern?

A: Store state as sequence of events rather than current state. Append-only event log. Replay events to reconstruct state. Benefits: audit trail, temporal queries, replay. Complexity: eventual consistency, event versioning.

Q: 26. What is the Outbox pattern?

A: Solve dual-write problem (write to DB + publish event). Write event to OUTBOX table in same transaction as domain change. Separate poller publishes events from outbox to message broker. Guarantees at-least-once event delivery.

Q: 27. What is the Saga pattern?

A: Distributed transaction via sequence of local transactions. Each step publishes event triggering next. Compensating transactions for rollback. Choreography (event-driven) vs Orchestration (central coordinator). Replaces 2PC.

Q: 28. What is the Circuit Breaker pattern?

A: Prevent cascading failure. States: CLOSED (normal), OPEN (failing, reject calls), HALF-OPEN (test recovery). Open on failure threshold. Reset after timeout. Resilience4j, Hystrix. Fail fast with fallback response.

Q: 29. What is the Bulkhead pattern?

A: Isolate failures in one component from spreading. Thread pool bulkhead: separate pool per service (slow service uses own pool, doesn't exhaust shared pool). Semaphore bulkhead: limit concurrent calls. Named after ship compartments.

Q: 30. What is the Retry pattern?

A: Retry transient failures with exponential backoff and jitter. Max retry count. Retry only on transient errors (503, timeout), not client errors (400, 401). Use idempotency keys to make retries safe. Spring Retry: @Retryable.

Q: 31. What is the Strangler Fig pattern?

A: Gradually migrate legacy monolith to microservices. New functionality in new service. Gradually move features from monolith. Route traffic to new services via API Gateway/facade. Monolith shrinks until eventually replaced.

Q: 32. What is the API Gateway pattern?

A: Single entry point for all clients. Handles: routing, auth, rate limiting, SSL termination, request aggregation, protocol translation. Clients unaware of backend service topology. Examples: Kong, AWS API Gateway, Spring Cloud Gateway.

Q: 33. What is the Service Mesh pattern?

A: Infrastructure layer for service-to-service communication. Sidecar proxy (Envoy) handles: mTLS, circuit breaking, retries, observability. Istio, Linkerd. Moves cross-cutting concerns out of application code.

Q: 34. What is the Anti-Corruption Layer (ACL) pattern?

A: Isolates domain from external systems with different models. Translates between your domain model and external model. Prevents external system's design from corrupting your domain. Used when integrating with legacy systems or third parties.

Q: 35. What is the Specification pattern?

A: Encapsulates business rules as composable predicates. canBePurchased = inStock.and(priceUnder(100)).and(categoryIs('Books')). Used in DDD for business rule validation and querying. Spring Data Specifications extend this.

23. Caching Strategies [30 Questions]

Q: 1. What are the levels of caching?

A: L1 CPU cache (ns, tiny), L2/L3 CPU cache, JVM heap cache (in-process), distributed cache (Redis, Memcached), CDN (edge cache), DB query cache, disk cache. Each level trades speed for capacity and cost.

Q: 2. What is cache-aside (lazy loading)?

A: App checks cache first; on miss loads from DB, stores in cache. App controls cache. Pro: only requested data cached, cache failure doesn't break app. Con: cache miss penalty (read from DB), potential stale data.

Q: 3. What is read-through cache?

A: Cache handles miss transparently — loads from DB, returns to app. App only talks to cache. Pro: simpler app code, consistent cache population. Con: cache is critical path, initial misses are slow.

Q: 4. What is write-through cache?

A: Write to cache AND DB synchronously on every write. Cache always up-to-date. Pro: no stale cache. Con: every write hits DB (no write offloading). Good for read-heavy, write-infrequent data.

Q: 5. What is write-behind (write-back) cache?

A: Write to cache only; flush to DB asynchronously. Pro: very fast writes, absorbs write spikes. Con: data loss if cache crashes before flush, complex consistency. Used for high-throughput writes (analytics, counters).

Q: 6. What is refresh-ahead cache?

A: Proactively refresh cache entries before they expire. Predict what will be needed. Pro: no read latency spikes at expiry. Con: may refresh unused entries. Used for: CDN edge cache prefetch, product catalog.

Q: 7. What is cache penetration?

A: Query for key that doesn't exist in cache OR DB. Repeated queries bypass cache, hit DB each time. Fix: cache null results with short TTL, bloom filter to reject definitely-missing keys before cache/DB check.

Q: 8. What is cache breakdown (stampede)?

A: Hot cache entry expires; many concurrent requests miss and all query DB simultaneously. Fix: mutex lock (one thread loads, others wait), background refresh, jitter in TTL, never-expire + async refresh.

Q: 9. What is cache avalanche?

A: Many cache entries expire simultaneously causing mass DB load. Fix: randomized TTL (base + jitter), circuit breaker on DB, layered caching (L1 in-process + L2 Redis), graceful degradation.

Q: 10. What is cache pollution?

A: Cache filled with rarely-used data, evicting frequently-used data. Caused by: large scans populating cache with cold data. Fix: bypass cache for bulk reads, use different cache for different workloads.

Q: 11. What is eviction policy?

A: Redis: noeviction (reject writes when full), allkeys-lru, allkeys-lfu, allkeys-random, volatile-lru (only keys with TTL), volatile-lfu, volatile-random, volatile-ttl. Choose by access pattern: LRU for recency, LFU for frequency.

Q: 12. What is LRU vs LFU eviction?

A: LRU (Least Recently Used): evict item accessed least recently. Works for recency-based access. LFU (Least Frequently Used): evict item accessed least times. Works for frequency-based access. LFU handles long-tail better than LRU.

Q: 13. What is distributed cache consistency?

A: Cache-aside: can read stale data between DB write and cache update. Solutions: write-through, cache invalidation on write (delete cache entry), event-driven invalidation (Debezium CDC -> delete cache entry). No perfect solution.

Q: 14. What is cache invalidation?

A: 'There are only two hard things in CS: cache invalidation and naming things.' Event-based: when data changes, invalidate/update cache. TTL-based: let entries expire. Version-based: include version in cache key. Each has trade-offs.

Q: 15. What is Spring @Cacheable?

A: @Cacheable(value='products', key='#id'): caches return value. Cache key derived from method params. condition/unless attributes for conditional caching. @CacheEvict: removes entries. @CachePut: always updates. @Caching: combines multiple.

Q: 16. What is a near cache?

A: Two-tier: local in-process cache (L1) + remote distributed cache (L2). Check L1 first (sub-ms). On L1 miss: check L2 Redis (1-5ms). On L2 miss: query DB. L1 invalidation challenging in distributed systems.

Q: 17. What is a distributed lock with Redis?

A: SET key value NX PX 30000 (atomic). NX = only set if not exists. PX = expire in ms. Value = unique token (client ID + timestamp). Release: compare token then delete (Lua script for atomicity). Prevents double processing.

Q: 18. What is a Redis pipeline?

A: Send multiple commands to Redis in one round-trip. Client buffers commands, sends batch, receives batch response. Reduces network latency for bulk operations. Different from transactions (not atomic). Throughput improvement: 10x+.

Q: 19. What is a Redis transaction (MULTI/EXEC)?

A: MULTI: begin transaction. Queue commands. EXEC: execute atomically. WATCH: optimistic locking — abort if watched key changed. Not true ACID (no rollback on command failure). Atomic execution, not atomic consistency.

Q: 20. What is Redis Pub/Sub?

A: Publisher sends messages to channels. Subscribers receive all messages on subscribed channels. Fire-and-forget (no persistence, no delivery guarantee). Use for: real-time notifications, live updates, chat. For persistence: use Redis Streams.

Q: 21. What is Redis Streams?

A: Append-only log (like Kafka). XADD adds entries. XREAD/XREADGROUP for consumer groups. Consumer groups: multiple consumers, each reads different entries, acknowledgment required. Persistent, replayable. Better than Pub/Sub for reliable messaging.

Q: 22. What is a cache key design?

A: Key should identify exactly what data it represents. Include entity type, ID, version: user:123:profile, product:456:v2. Namespacing prevents collisions. Avoid user-controlled key parts (injection). Max key length: Redis 512MB but keep short.

Q: 23. What is Memcached vs Redis?

A: Memcached: simple key-value, multi-threaded (better CPU utilization), no persistence, no replication built-in. Redis: rich data structures, persistence, replication, Pub/Sub, scripting, atomic operations. Redis is almost always preferred now.

Q: 24. What is local (in-process) caching with Caffeine?

A: Java caching library. CaffeineCache.newBuilder().maximumSize(1000).expireAfterWrite(10, MINUTES).build(). High performance, W-TinyLFU eviction. Spring Boot: spring.cache.type=caffeine. Better than Guava Cache.

Q: 25. What is cache warming?

A: Proactively populate cache before load hits. On startup: load common data (product catalog, config). Schedule warming jobs. Prevents cold start problems. Trade-off: startup time vs first-request latency.

Q: 26. What is a distributed cache hotspot?

A: One cache key receives disproportionate traffic (celebrity user, flash sale product). Saturates single cache node. Fix: local in-process caching for hotspot keys, key replication (product:123:0 through product:123:9), CDN for read-heavy data.

Q: 27. What is cache serialization?

A: Objects stored as bytes in cache. Serializers: Java serialization (slow, fragile), JSON (readable, flexible), Protobuf/Avro (compact, fast). Spring Cache uses Java serialization by default — configure Jackson for JSON or Kryo for performance.

Q: 28. What is a write-through vs write-around cache?

A: Write-through: writes go to cache + DB. Cache has all written data. Write-around: writes go directly to DB, bypass cache. Cache filled only on reads. Use write-around when written data is unlikely to be read soon (log files, bulk imports).

Q: 29. What is a TTL strategy for caching?

A: Static reference data: hours/days. User sessions: 30 min. Product catalog: 5-15 min. Stock levels: seconds or no cache. Auth tokens: match token expiry. Short TTL = more DB load but fresher data. TTL + invalidation = hybrid approach.

Q: 30. What is cache observability?

A: Metrics: hit rate, miss rate, eviction rate, memory usage, key count. Prometheus: redis_keyspace_hits_total / (hits + misses) = hit rate. Alert on low hit rate, high eviction. Cache hit rate should be >90% for effective caching.

24. Git Version Control [15 Questions]

Q: 1. What are the three Git areas?

A: Working directory (local file changes), Staging area/index (git add stages changes for commit), Repository (committed history). git status shows state. git diff: working vs staging. git diff --cached: staging vs HEAD.

Q: 2. What is the difference between merge and rebase?

A: Merge: creates merge commit, preserves history, non-destructive. Rebase: replays commits on top of another branch, linear history, rewrites commit SHAs. Golden rule: never rebase shared/public branches.

Q: 3. What is git stash?

A: Temporarily saves uncommitted changes. git stash push, stash pop (apply + drop), stash apply (apply, keep stash), stash list. Useful for: switching branches mid-task, pulling changes without committing WIP.

Q: 4. What is git cherry-pick?

A: Apply specific commit from one branch to another by commit SHA. git cherry-pick abc123. Creates new commit with same changes but different SHA. Use for: hotfixes applied to multiple branches, selective feature port.

Q: 5. What is git bisect?

A: Binary search to find commit that introduced a bug. git bisect start, bisect bad (current is broken), bisect good sha (known good commit), mark each suggested commit good/bad. Finds culprit in O(log n) steps.

Q: 6. What is a fast-forward merge?

A: When target branch has no new commits since feature branch diverged, Git moves branch pointer forward without creating merge commit. --no-ff forces merge commit (preserves branch history). --ff-only fails if not fast-forward.

Q: 7. What is the difference between git reset and git revert?

A: reset --soft/mixed/hard: moves HEAD (rewrites history — dangerous on shared branches). revert: creates new commit undoing changes (safe for shared branches). Prefer revert for public branches, reset for local work.

Q: 8. What is git reflog?

A: Log of all HEAD movements including resets, rebase, amend. Recover 'lost' commits. git reflog to see history. git reset --hard SHA to restore. Entries kept for 30 days by default. Lifesaver for recovery.

Q: 9. What is a detached HEAD state?

A: HEAD points to specific commit rather than branch. Changes made here are 'floating' — no branch points to them. git checkout branch to re-attach. Create branch: git checkout -b new-branch to capture commits.

Q: 10. What is Trunk Based Development?

A: All developers commit to single trunk (main) frequently. Short-lived feature branches (1-2 days max). Feature flags for incomplete features. Reduces merge conflicts and integration delays. Enables true CI/CD.

Q: 11. What is a conventional commit?

A: Structured commit message format: type(scope): description. Types: feat (new feature), fix, docs, style, refactor, test, chore, perf. Breaking changes: BREAKING CHANGE footer. Enables automated changelog generation, semantic versioning.

Q: 12. What is git tag?

A: Marks specific commit (release point). Lightweight: just a pointer. Annotated: stores tagger, date, message (preferred for releases). git tag v1.0.0 -m 'Release 1.0'. Push tags: git push origin --tags.

Q: 13. What is git blame?

A: Shows who last modified each line of a file: git blame filename. Find when/who introduced a line. -L 10,20 for line range. Useful for understanding code history. Available in IDEs (Annotate).

Q: 14. What is git submodule?

A: Reference to another Git repository at specific commit. git submodule add url path. Useful for shared libraries across repos. Complexity: updating submodules requires explicit steps. Many teams prefer monorepo or package manager instead.

Q: 15. What is a Git hook?

A: Scripts triggered on Git events. Client-side: pre-commit (lint, format), commit-msg (validate message), pre-push. Server-side: pre-receive (enforce policies), post-receive (trigger CI). Tool: Husky for JS projects; pre-commit framework.

25. Maven & Gradle Build Tools [15 Questions]

Q: 1. What are Maven lifecycle phases?

A: validate -> compile -> test -> package -> verify -> install -> deploy. Each phase executes all previous phases. Default lifecycle plus clean (pre-clean, clean, post-clean) and site lifecycles. Goals bound to phases by plugins.

Q: 2. What is Maven dependency scope?

A: compile (default): available everywhere. test: only tests. runtime: not needed for compilation, needed at runtime (JDBC driver). provided: compile-time only, provided by container (servlet-api). import: BOM import. system: explicit path (avoid).

Q: 3. What is a Maven BOM (Bill of Materials)?

A: POM file defining dependency versions for a set of related artifacts. Import with scope=import in dependencyManagement. Spring Boot parent POM is a BOM. Ensures compatible versions across all included artifacts.

Q: 4. What is the difference between pom.xml dependencies and dependencyManagement?

A: dependencies: actually include the dependency in classpath. dependencyManagement: declare version/scope without including — child modules inherit version without re-specifying. BOM pattern uses dependencyManagement.

Q: 5. What is Maven multi-module project?

A: Parent POM with modules list. Each module has its own pom.xml with parent reference. mvn install on parent builds all modules in order. Inter-module dependencies work with SNAPSHOT versions. Useful for: shared libraries, monorepo.

Q: 6. What is Gradle?

A: Build automation tool. Groovy/Kotlin DSL (build.gradle / build.gradle.kts). Task graph (not lifecycle). Incremental builds: only rebuild changed. Build cache: reuse task outputs across builds. 10-100x faster than Maven for large projects.

Q: 7. What is Gradle implementation vs api?

A: implementation: dependency not exposed in compile classpath to consumers (preferred — better encapsulation, faster compilation). api: exposed to consumers (for library modules that expose types). compileOnly: compile not runtime. runtimeOnly: runtime not compile.

Q: 8. What is Gradle incremental build?

A: Gradle tracks task inputs and outputs. If unchanged: task is UP-TO-DATE, skipped. If inputs changed: re-execute. Declared with @InputFiles, @OutputDirectory annotations. Key to fast incremental builds.

Q: 9. What is Gradle build cache?

A: Cache task outputs by hash of inputs. Local cache: ~/.gradle/caches. Remote cache: shared across team (HTTP). If another developer already ran the task with same inputs: YOUR build uses cached output. Huge time savings in CI.

Q: 10. What is Spring Boot Maven plugin?

A: spring-boot-maven-plugin: creates executable fat JAR (java -jar). repackage goal adds BOOT-INF/lib (dependencies) and BOOT-INF/classes (your code). run goal: run app from Maven. build-image: build OCI image with Buildpacks.

Q: 11. What is Maven Wrapper (mvnw)?

A: Scripts (mvnw, mvnw.cmd) that download and use specific Maven version defined in .mvn/wrapper/maven-wrapper.properties. Team uses consistent Maven version without system installation. Gradle has gradlew equivalent.

Q: 12. What is a Maven snapshot vs release?

A: SNAPSHOT: development version (1.0-SNAPSHOT). Maven always checks for newer snapshot in remote repo. RELEASE (1.0): immutable — once deployed, never changes. CI builds should never depend on SNAPSHOTs for releases.

Q: 13. What is Gradle Convention Plugin?

A: Share build logic across modules. Written as a precompiled script plugin in buildSrc/ or gradle/plugins. Apply common config (Java version, checkstyle, publishing) once, reuse in all modules. Alternative to parent POM in Gradle.

Q: 14. What is Maven Enforcer plugin?

A: Enforce project constraints: required Java version, Maven version, no duplicate dependencies, banned dependencies. Fails build if rules violated. Useful for: ensuring team uses correct Java version, preventing dependency conflicts.

Q: 15. What is dependency conflict resolution?

A: Maven: nearest definition wins (shorter path in tree), then first declaration. Gradle: uses highest version (default). Inspect: mvn dependency:tree, gradle dependencies. Exclude transitive deps: in Maven, exclude() in Gradle.

26. DevOps & CI/CD [20 Questions]

Q: 1. What is DevOps?

A: Cultural and technical movement: Dev and Ops teams collaborate. Practices: CI/CD, IaC, automated testing, monitoring, short feedback loops. Goal: faster, more reliable software delivery.

Q: 2. What is CI vs CD vs Continuous Deployment?

A: CI: merge frequently, automated build + test on every commit. CD (Continuous Delivery): CI + deployable artifact always ready, manual deploy gate. Continuous Deployment: every passing build auto-deployed to production. Each requires the previous.

Q: 3. What is a CI pipeline?

A: Automated steps on each commit: checkout -> build -> unit test -> static analysis -> integration test -> build image -> security scan -> push artifact -> deploy to staging. Fails fast, short feedback loop.

Q: 4. What is GitHub Actions?

A: CI/CD built into GitHub. Workflow: YAML in .github/workflows/. Trigger: push, PR, schedule, workflow_dispatch. Jobs run on runners (GitHub-hosted or self-hosted). Actions: reusable steps from Marketplace.

Q: 5. What is a deployment strategy — blue-green?

A: Two identical environments: blue (current), green (new version). Route traffic to green after testing. If issues: instantly switch back to blue. Zero downtime. Double infrastructure cost.

Q: 6. What is a canary deployment?

A: Route small % traffic to new version (5-10%). Monitor errors/latency. If stable: gradually increase %. Automatic rollback on alerts. More complex than blue-green but reduces blast radius. Istio/Argo Rollouts manage traffic splitting.

Q: 7. What is a rolling deployment?

A: Replace instances one by one. Never all down simultaneously. Fewer resources than blue-green. Harder to rollback (must roll forward or reverse rolling). K8s Deployment default strategy. maxUnavailable and maxSurge control.

Q: 8. What is Infrastructure as Code (IaC)?

A: Define infrastructure in version-controlled code. Tools: Terraform (multi-cloud, declarative), CloudFormation (AWS), Pulumi (general-purpose languages), Ansible (config management). Reproducible, reviewable, auditable infrastructure.

Q: 9. What is Terraform?

A: IaC tool. HCL (HashiCorp Configuration Language). Plan: show what will change. Apply: make changes. State file: tracks deployed resources. Remote state: S3 + DynamoDB lock. Modules for reuse. Provider: AWS, GCP, Azure.

Q: 10. What is Docker in CI/CD?

A: Build Docker image in CI: docker build -t app:git-sha . Push to registry. Deploy image to staging/prod. Benefits: consistent environment, reproducible builds, easy rollback (previous image tag).

Q: 11. What is a feature flag?

A: Toggle features on/off without deployment. Separate feature rollout from deployment. Types: release flags (gradual rollout), experiment flags (A/B test), ops flags (kill switch), permission flags (beta users). Tools: LaunchDarkly, Unleash, Flagsmith.

Q: 12. What is GitOps?

A: Use Git as single source of truth for infrastructure AND application state. Argo CD watches Git; automatically reconciles cluster state to match. Pull-based model (Argo CD pulls from Git) vs push-based (kubectl apply in pipeline). Better audit trail, declarative.

Q: 13. What is Argo CD?

A: Declarative GitOps CD tool for Kubernetes. Syncs K8s cluster with Git repo. Detects drift and alerts or auto-heals. ApplicationSet for multi-cluster. Integrates with Helm, Kustomize. Web UI for visibility.

Q: 14. What is a DORA metric?

A: Four key DevOps metrics: Deployment Frequency (how often deploy), Lead Time for Changes (commit to production), Change Failure Rate (% deployments causing failures), Time to Restore Service (MTTR). Elite: weekly+ deploys, <1h lead time, <5% failure rate, <1h MTTR.

Q: 15. What is a Jenkins pipeline?

A: Jenkinsfile: Groovy DSL defining CI pipeline stages. Declarative: pipeline { agent { docker 'maven' } stages { stage('Build') { steps { sh ' mvn package' } } } }. Scripted: more flexible but verbose.

Q: 16. What is SonarQube?

A: Code quality and security scanner. Checks: bugs, code smells, security vulnerabilities, technical debt, coverage. Quality Gate: fails build if metrics below threshold. Integrates with Maven, Gradle, CI pipelines.

Q: 17. What is Docker layer caching in CI?

A: Each Dockerfile instruction cached. In CI: pull previous image to seed cache (--cache-from). BuildKit inline cache. GitHub Actions: actions/cache for Docker layers. Without caching: every CI build downloads all dependencies.

Q: 18. What is a container registry?

A: Stores and distributes Docker images. Push: docker push registry/image:tag. Pull: docker pull. Private: AWS ECR, GCP GCR, Azure ACR, Nexus, Harbor. Public: Docker Hub. Scanning: ECR and Harbor scan images for vulnerabilities.

Q: 19. What is rollback in CI/CD?

A: Revert to previous version on failure. Docker: deploy previous image tag. K8s: kubectl rollout undo deployment/app. Feature flags: toggle off feature without redeployment. Database rollbacks: harder — use forward-compatible schema changes (expand-contract pattern).

Q: 20. What is the 12-factor app?

A: Methodology for cloud-native apps: codebase in version control, dependencies declared, config in environment, backing services as attached resources, build/release/run separation, stateless processes, port binding, concurrency, disposability, dev/prod parity, logs as streams, admin processes.

27. Observability & Grafana [15 Questions]

Q: 1. What are the three pillars of observability?

A: Metrics (aggregated numeric data over time — CPU, RPS, latency), Logs (timestamped text events from app), Traces (request flow across services). All three needed for complete visibility. OpenTelemetry provides unified collection.

Q: 2. What is Prometheus?

A: Time-series metrics system. Pull model: scrapes /actuator/prometheus endpoint. Metric types: Counter (monotonically increasing), Gauge (up/down), Histogram (bucket distribution), Summary (quantiles). PromQL for querying.

Q: 3. What is PromQL?

A: Prometheus Query Language. rate(http_requests_total[5m]): per-second rate over 5 min. histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m])): p99 latency. sum(rate(...)) by (service): group by label.

Q: 4. What is Micrometer?

A: Java metrics facade. Abstracts over Prometheus, DataDog, CloudWatch, etc. @Timed, Counter.builder, Gauge.builder, Timer.builder, DistributionSummary. Spring Boot Actuator auto-configures Micrometer.

Q: 5. What is Grafana?

A: Visualization platform for metrics, logs, traces. Connects to data sources (Prometheus, Loki, Tempo, CloudWatch, Elasticsearch). Dashboards with panels (time series, stat, table, heatmap). Alerting rules. Variables for dynamic dashboards.

Q: 6. What is Loki?

A: Grafana Labs' log aggregation system. Like Prometheus but for logs. LogQL: label-based filtering: {app='myapp'} |= 'ERROR'. Cheaper than Elasticsearch (index only metadata, not full text). Integrates with Grafana.

Q: 7. What is Tempo?

A: Grafana Labs' distributed tracing backend. Stores trace data cheaply (object storage). Integrates with Grafana for trace visualization. Trace to logs, trace to metrics correlation. OpenTelemetry and Jaeger/Zipkin compatible.

Q: 8. What is OpenTelemetry?

A: Vendor-neutral observability framework. Unified API for metrics, logs, traces. Java agent: -javaagent:opentelemetry-javaagent.jar. Auto-instruments Spring Boot, JDBC, Kafka, HTTP clients. Exports to any backend (Jaeger, Prometheus, Datadog).

Q: 9. What is Alertmanager?

A: Handles Prometheus alerts: routing (send to PagerDuty, Slack, email), grouping (deduplicate related alerts), silencing, inhibition (suppress downstream alerts when upstream down). Alert rules defined in Prometheus YAML.

Q: 10. What is a Grafana dashboard variable?

A: Dynamic parameter in dashboard. Dropdown lets user select: environment (prod/staging), service name, time window. Variables interpolated in queries: {app='$service'}. Enables single dashboard serving multiple services/environments.

Q: 11. What is distributed tracing?

A: Track request flow across multiple services. Each request gets TraceID; each operation gets SpanID. Spans linked by parent-child. Visualize in Jaeger/Zipkin/Tempo. Find which service is the latency bottleneck.

Q: 12. What is an SLI/SLO/SLA?

A: SLI (Service Level Indicator): metric measuring reliability (error rate, p99 latency). SLO (Service Level Objective): target for SLI (error rate < 0.1%, p99 < 200ms). SLA (Service Level Agreement): contractual SLO with penalties. Error budget = 1 - SLO.

Q: 13. What is log aggregation?

A: Collect logs from all services/pods to central system. Fluentd/Fluent Bit: collect from pods, ship to Loki/Elasticsearch. Structured logging (JSON) enables better filtering and querying. Correlation IDs link logs across services.

Q: 14. What is a RED method?

A: Rate (requests per second), Errors (error rate), Duration (latency). For each service: track these three. Simple, effective. Complements USE method (Utilization, Saturation, Errors) for infrastructure.

Q: 15. What is structured logging?

A: Log as JSON: {"timestamp":"...","level":"ERROR","service":"order-api","traceId":"...","message":"..."}. Enables log querying and filtering. Logback: JsonEncoder. Include traceId (from MDC) for correlation.

28. Pagination Strategies [15 Questions]

Q: 1. What is offset pagination?

A: SELECT * FROM items ORDER BY id LIMIT 20 OFFSET 100. Page N: OFFSET (N-1)*pageSize. Simple, random page access. Performance degrades: DB scans all OFFSET rows. Not suitable for large offsets.

Q: 2. What is keyset (seek) pagination?

A: WHERE id > lastSeenId ORDER BY id LIMIT 20. O(log n) via index. Consistent performance at any depth. No random page access (forward-only or known cursor). Best practice for production APIs.

Q: 3. What is cursor-based pagination?

A: Encode last seen position as opaque cursor (base64 of lastId+timestamp). Client passes cursor as nextPageToken. Server decodes and applies keyset query. Cursor hides implementation details from client.

Q: 4. What is the problem with OFFSET at large page numbers?

A: OFFSET 1000000 LIMIT 20: DB must scan and discard 1M rows, then return 20. O(n) performance. On sorted index: still must traverse index entries. Degrades user experience. Keyset pagination: O(log n) always.

Q: 5. What are cursor pagination API design best practices?

A: Response: items[], nextCursor (null if no more pages). Client: GET /api/items?cursor=xxx&limit;=20. Cursor opaque (don't expose internals). Use stable ordering: ORDER BY created_at, id (tiebreaker for created_at duplicates).

Q: 6. What is time-based pagination?

A: Paginate by timestamp: WHERE created_at > lastTimestamp ORDER BY created_at LIMIT 20. Natural for time-ordered data (feeds, events). Problem: duplicate timestamps — add tiebreaker ID. Partial page on exact timestamp boundary.

Q: 7. What is stable sort for pagination?

A: Sort order must be deterministic to ensure consistent pages. Add unique ID as secondary sort: ORDER BY created_at DESC, id DESC. Without stable sort: items may appear on multiple pages or be skipped during pagination.

Q: 8. What is Elasticsearch pagination?

A: from/size: same as SQL OFFSET, limited to 10,000 results. search_after: keyset-like, efficient deep pagination with pit (point in time). Scroll API: for bulk export (not user pagination). Use search_after + PIT for production pagination.

Q: 9. What is bidirectional cursor pagination?

A: Support both next and previous page. Cursor encodes direction + position. Reverse: flip ORDER BY direction, apply cursor condition in reverse. Complex but needed for some UIs.

Q: 10. What is total count in paginated APIs?

A: Accurate count requires full scan (expensive). Options: exact count (slow), estimated count (PostgreSQL reltuples), disable count, count cap (show '1000+' when count > threshold). REST: include total in headers or body. GraphQL: pageInfo.totalCount.

Q: 11. What is deferred join for pagination?

A: For queries with expensive joins: paginate IDs first (keyset on primary table), then JOIN: SELECT * FROM orders JOIN users ON orders.user_id = users.id WHERE orders.id IN (SELECT id FROM orders WHERE ... LIMIT 20). Avoids full join on entire result set.

Q: 12. What is cursor encoding?

A: Encode cursor as base64(JSON(sortFields)) or HMAC for tamper-proofing. Client passes as opaque token. Server decodes, validates, applies to query. Version field in cursor enables format migration.

Q: 13. What is page size guidance?

A: Default: 20-50 items. Max: 100-200. Never unlimited (DoS risk). Enforce server-side limit regardless of client request. Communicate via API response: defaultPageSize, maxPageSize.

Q: 14. What is link header for pagination?

A: RFC 5988: Link header with rel=next, prev, first, last. Link: </items?cursor=abc>; rel='next'. Standard for REST APIs. Response body can also include pagination metadata (nextCursor, hasNextPage, totalCount).

Q: 15. What is pagination with total count performance optimization?

A: COUNT(*) hits are expensive. Options: approximate count (PostgreSQL pg_class.reltuples), run count asynchronously (return when available), cache count for N seconds, skip total (return hasNextPage only). Trade-off: accuracy vs performance.

29. Authentication & Authorization [25 Questions]

Q: 1. What is the difference between authentication and authorization?

A: Authentication (AuthN): who are you? (verify identity — login). Authorization (AuthZ): what can you do? (verify permissions). 401 Unauthorized = not authenticated. 403 Forbidden = authenticated but not authorized.

Q: 2. What is JWT structure?

A: Three base64url-encoded parts: Header (alg, typ), Payload (claims: sub, iss, exp, iat, custom), Signature. Signed with HS256 (HMAC shared secret) or RS256 (RSA private key). Verify signature to trust claims. Never store sensitive data in JWT (it's encoded, not encrypted).

Q: 3. What is OAuth2?

A: Authorization framework (not authentication). Resource owner (user) grants client access to resource server via authorization server. Flows: Auth Code (web apps), Client Credentials (M2M), PKCE (SPA/mobile), Device Flow (CLI/TV).

Q: 4. What is OAuth2 Authorization Code Flow with PKCE?

A: Client generates code_verifier (random), code_challenge (SHA256 of verifier). Redirects to auth server with code_challenge. User authenticates. Auth server returns code. Client exchanges code + code_verifier for tokens. Prevents interception attacks.

Q: 5. What is OIDC (OpenID Connect)?

A: Identity layer on top of OAuth2. ID Token (JWT with user claims: sub, name, email). UserInfo endpoint. Standard scopes: openid, profile, email. Provides authentication (who is this person?) not just authorization.

Q: 6. What is a refresh token?

A: Long-lived token used to get new access tokens without re-authentication. Short-lived access token (15 min) + long-lived refresh token (7 days). Refresh token rotates on use (prevents theft). Store refresh token securely (HttpOnly cookie).

Q: 7. What is mTLS?

A: Mutual TLS: both server AND client present certificates. Server verifies client cert against trusted CA. Strongest API authentication. Used for: service-to-service auth, banking APIs, IoT. Client cert management is complex.

Q: 8. What is RBAC?

A: Role-Based Access Control. Users assigned roles (admin, editor, viewer). Roles have permissions. Check: hasRole('ADMIN'). Spring Security: @PreAuthorize('hasRole(ADMIN)'). Simpler to manage than ABAC.

Q: 9. What is ABAC?

A: Attribute-Based Access Control. Access based on attributes of user, resource, action, environment. Policy: user.department == resource.department AND resource.classification <= user.clearanceLevel. More flexible, more complex than RBAC.

Q: 10. What is API Key authentication?

A: Long-lived secret token in header (X-API-Key) or query param. Simple, good for server-to-server. No expiry (until revoked). Hash before storage. Associate with rate limits. Rotate regularly. Avoid in URLs (logged).

Q: 11. What is session-based auth?

A: Server creates session on login, stores in memory/Redis, sends session ID as HttpOnly cookie. Client sends cookie automatically. Session ID validates each request. Stateful: server must store session state.

Q: 12. What is token blacklisting?

A: Revoke JWT before expiry. Store invalidated tokens in Redis (or bloom filter). Check on each request. Trades JWT statelessness for revocation capability. More scalable: store only revoked tokens, not all valid tokens.

Q: 13. What is Spring Security filter chain?

A: Ordered list of security filters. Each processes request or delegates to next. Key filters: SecurityContextPersistenceFilter, UsernamePasswordAuthenticationFilter, JwtAuthenticationFilter (custom), ExceptionTranslationFilter, FilterSecurityInterceptor. Configure in SecurityFilterChain bean.

Q: 14. What is SAML?

A: XML-based SSO protocol. Identity Provider (IdP) asserts user identity to Service Provider (SP). SP-initiated: SP sends AuthnRequest to IdP, IdP redirects back with signed SAMLResponse. Enterprise, complex, used in B2B.

Q: 15. What is password hashing?

A: One-way function. BCrypt: adaptive cost factor, built-in salt. Argon2: winner of Password Hashing Competition, memory-hard. Never use MD5/SHA1 for passwords. Spring: BCryptPasswordEncoder. Cost factor should take 100ms+ to compute.

Q: 16. What is OAuth2 Client Credentials flow?

A: Machine-to-machine auth. No user involved. Client (service) authenticates with client_id + client_secret to get access token. POST to token endpoint. Token used for service-to-service API calls.

Q: 17. What is a JWT claim vs scope?

A: Claim: any key-value in JWT payload (sub, email, roles). Scope: OAuth2 concept — granted permissions (read:orders, write:orders). Scopes in JWT as scope claim (space-separated string) or array. Spring Security: hasAuthority('SCOPE_read:orders').

Q: 18. What is Spring Boot OAuth2 Resource Server?

A: Validates JWT Bearer tokens from Authorization header. Configure: spring.security.oauth2.resourceserver.jwt.jwk-set-uri=https://auth-server/.well-known/jwks.json. Auto-validates signature, expiry, audience. Populates SecurityContext.

Q: 19. What is the difference between HS256 and RS256?

A: HS256: HMAC-SHA256, symmetric (same secret signs and verifies). All parties need secret — only for same organization. RS256: RSA asymmetric (private key signs, public key verifies). Share public key freely — use for distributed systems, OAuth2 JWT.

Q: 20. What is OAuth2 token introspection?

A: Resource server calls auth server to validate opaque token: POST /introspect. Returns token metadata (active, scope, user, expiry). More secure than JWT (server validates, no local verification). Slower (network call per request).

Q: 21. What is Keycloak?

A: Open-source IAM (Identity and Access Management). Provides: User federation (LDAP, AD), Social login, SSO, Admin console, OIDC/SAML. Spring Boot integration: Keycloak Spring Boot adapter or standard OAuth2 resource server.

Q: 22. What is account enumeration attack?

A: Attacker determines if account exists by observing different error messages. Fix: same error for 'user not found' and 'wrong password'. Same timing (constant-time comparison). Prevents username harvesting.

Q: 23. What is OAuth2 state parameter?

A: Random value sent with authorization request, returned unchanged by auth server. Client verifies it matches stored value. Prevents CSRF attacks on OAuth2 flow (forged authorization requests).

Q: 24. What is PKCE code_challenge?

A: code_verifier = random 43-128 char base64url string. code_challenge = BASE64URL(SHA256(code_verifier)). Sent in authorization request. Prevents authorization code interception: attacker can't exchange code without code_verifier.

Q: 25. What is the difference between access token and ID token?

A: Access token: opaque or JWT, proves authorization to access resources, used in API calls (Authorization: Bearer). ID token: JWT, proves authentication, contains user identity claims (sub, email, name), consumed by client app only.

30. Session Management [15 Questions]

Q: 1. What is a server-side session?

A: Server creates session object on login, stores in memory or Redis, sends session ID via HttpOnly cookie. Each request: client sends cookie, server validates session ID. Stateful: server must maintain session store.

Q: 2. What is HttpOnly cookie?

A: Cookie attribute preventing JavaScript access (document.cookie). Mitigates XSS session theft — attacker's script can't read session cookie. Combine with: Secure (HTTPS only), SameSite=Strict/Lax (CSRF protection).

Q: 3. What is the SameSite cookie attribute?

A: Strict: cookie not sent on cross-site requests. Lax: sent on top-level navigation (GET), not on AJAX/form POST cross-site. None: always sent (requires Secure). Modern browsers default to Lax. Prevents CSRF attacks.

Q: 4. What is session fixation?

A: Attacker sets known session ID before login. User logs in. Server uses same session ID. Attacker can impersonate user. Fix: always regenerate session ID after successful login (Spring Security does this by default).

Q: 5. What is session hijacking?

A: Theft of valid session ID. Methods: XSS (steal from cookie/storage), network sniffing (HTTP), predictable IDs. Prevention: HttpOnly+Secure cookies, HTTPS only, long random session IDs, IP binding (optional).

Q: 6. What is Spring Session?

A: Framework for distributed session management. Replaces default HTTP session with Redis-backed session. spring-session-data-redis: stores session in Redis. Enables: session replication, multiple app instances sharing sessions, session clustering.

Q: 7. What is the difference between JWT and session cookies for auth?

A: Session cookie: server-side state, instant revocation, scalability challenge (shared store). JWT: client-side state, stateless/scalable, revocation difficult (wait for expiry or blacklist). JWT: better for microservices, CDN APIs. Session: better for traditional web apps.

Q: 8. What is a sliding session?

A: Session TTL extends on each activity. Active users never timed out. Idle users timeout after inactivity period. Redis: EXPIRE key seconds on each request. Downside: active session never expires (security concern for sensitive apps).

Q: 9. What is session store sizing?

A: Redis memory per session: ~1-10KB. 100K concurrent users = 100MB-1GB. Redis handles millions of sessions easily. Monitor: SESSION_KEY_COUNT, memory usage, eviction rate. Set maxmemory-policy: volatile-lru (evict expired sessions first).

Q: 10. What is concurrent session control?

A: Limit user to N active sessions. Spring Security: maximumSessions(1). When limit exceeded: kick oldest session or reject new login. Redis SessionRegistry tracks active sessions per user.

Q: 11. What is CSRF?

A: Cross-Site Request Forgery: malicious site tricks user's browser into sending authenticated request to your site. Prevention: SameSite cookie, CSRF tokens (synchronized pattern: server sends token in form, validates on submit), Double Submit Cookie.

Q: 12. What is a secure session ID?

A: Cryptographically random, minimum 128 bits (22+ chars base64). Not predictable from previous IDs. Not derived from user data. Stored in HttpOnly Secure cookie. Change after login (prevents fixation). Invalidate on logout.

Q: 13. What is token refresh strategy?

A: Access token: short-lived (15 min). Refresh token: long-lived (7-30 days), HttpOnly cookie. When access token expires: silently exchange refresh token for new access token. If refresh token expired: re-authenticate.

Q: 14. What is idle timeout vs absolute timeout?

A: Idle timeout: session expires after N minutes of inactivity (sliding). Absolute timeout: session always expires N hours after creation regardless of activity. Use both: idle=30min, absolute=8h. Absolute timeout protects against stolen active sessions.

Q: 15. What is session logout best practice?

A: Invalidate server-side session (or blacklist JWT). Clear session cookie (Set-Cookie with Max-Age=0). For SSO: trigger Single Logout (SLO) at IdP. For OAuth2: revoke refresh token at auth server. Redirect to login page.

31. Single Sign-On (SSO) [15 Questions]

Q: 1. What is SSO?

A: Single Sign-On: authenticate once, access multiple applications. User logs in to Identity Provider (IdP). IdP issues token/assertion. Service Providers (SPs) trust IdP. Protocols: OIDC (modern), SAML 2.0 (enterprise), Kerberos (internal Windows).

Q: 2. What is OIDC-based SSO?

A: User authenticates with IdP (Keycloak, Okta, Auth0). IdP issues ID token (JWT) + access token. User presents access token to each service. Services verify against IdP's public key. SLO: revoke at IdP; services check token status.

Q: 3. What is SAML 2.0 SP-initiated SSO?

A: User accesses SP. SP redirects to IdP with SAMLRequest (AuthnRequest). User authenticates at IdP. IdP posts SAMLResponse (XML, signed) to SP ACS URL. SP validates signature, extracts attributes, creates session.

Q: 4. What is IdP-initiated SSO?

A: User starts at IdP portal. Clicks app. IdP posts SAMLResponse directly to SP. No AuthnRequest. Less secure (no request binding). Some SPs allow only SP-initiated. Useful for: enterprise portals, app launchers.

Q: 5. What is Single Logout (SLO)?

A: Logout from one app propagates to all apps in SSO session. Front-channel: browser redirects to each SP's logout URL. Back-channel: IdP calls each SP's logout endpoint directly (more reliable). Spring Security supports SLO.

Q: 6. What is a federation?

A: Trust relationship between IdPs. User in Organization A can access Organization B's services. Protocols: SAML federation, OIDC federation. Enterprise: Active Directory Federation Services (ADFS) federates with external IdPs.

Q: 7. What is Keycloak realm?

A: Isolated tenant in Keycloak. Each realm has own users, clients, roles, identity providers. Master realm: admin only. Create separate realms per: customer (multi-tenant), environment (dev/prod). Realm = namespace for all identity config.

Q: 8. What is Keycloak client?

A: Application registered in Keycloak. ClientID + client secret (confidential) or public client (SPA). Configure: redirect URIs, scopes, token settings, mappers (add claims to token). Service account: client credentials flow.

Q: 9. What is token exchange (RFC 8693)?

A: Exchange one token for another at auth server. Use for: cross-service impersonation, service mesh auth, scope narrowing. POST to token endpoint: grant_type=urn:ietf:params:oauth:grant-type:token-exchange. Not widely implemented.

Q: 10. What is the difference between OIDC and SAML?

A: OIDC: JSON/JWT, REST-based, modern (mobile+SPA+web), simpler to implement, good for APIs. SAML: XML, redirect-based, enterprise legacy, complex, great for enterprise IdPs (ADFS, Okta SAML). New projects: OIDC. Enterprise integration: often SAML.

Q: 11. What is Okta?

A: Commercial IAM/SSO platform. User directory, OIDC/SAML, MFA, social login, lifecycle management. SaaS, no infrastructure to manage. Spring Boot: Spring Security OAuth2 client with Okta issuer URL. Competitive with Auth0 (also Okta-owned).

Q: 12. What is a just-in-time (JIT) provisioning?

A: User account created automatically on first SSO login. IdP sends user attributes (name, email, roles) in assertion. SP creates user account from these attributes. Eliminates manual user creation. Common in SAML-based enterprise SSO.

Q: 13. What is MFA in SSO?

A: Multi-Factor Authentication: something you know (password) + something you have (TOTP app, SMS, hardware key) + something you are (biometric). Configured at IdP level — applies to all SSO-connected apps. FIDO2/WebAuthn: phishing-resistant.

Q: 14. What is OAuth2 implicit flow?

A: DEPRECATED. Token returned directly in redirect URI fragment. Used for SPAs before PKCE. Vulnerable to token interception. Replaced by Auth Code + PKCE for all browser clients. Never implement new integrations with implicit flow.

Q: 15. What is OAuth2 device authorization flow?

A: For devices without browsers (smart TVs, CLI tools). Device shows user code + verification URL. User authenticates on another device. Device polls token endpoint until user authorizes. Used by: GitHub CLI, YouTube TV, Netflix.

32. Multi-Tenancy Design [15 Questions]

Q: 1. What is multi-tenancy?

A: Single application instance serves multiple customers (tenants). Each tenant's data isolated from others. Three main models: separate database, separate schema, shared tables (row-level tenant column).

Q: 2. What is separate DB multi-tenancy?

A: Each tenant has own database. Maximum isolation. Separate backup, scaling, compliance. Complex: connection pool management, schema migrations at scale. Use for: high-security tenants, large enterprise clients, compliance requirements.

Q: 3. What is separate schema multi-tenancy?

A: One DB, separate schema per tenant. Moderate isolation. Migrations needed per schema. Hibernate: @MultiTenantConnectionProvider + @CurrentTenantIdentifierResolver. Use for: medium isolation needs, dozens-to-hundreds of tenants.

Q: 4. What is shared schema multi-tenancy?

A: All tenants in same tables. tenant_id column on every table. Most economical. Hardest to ensure data isolation. Must filter every query by tenant_id. Risk: missing WHERE tenant_id = ? leaks cross-tenant data.

Q: 5. What is Hibernate multi-tenancy?

A: Hibernate supports: DATABASE (separate DB), SCHEMA (separate schema), DISCRIMINATOR (shared tables). Implement CurrentTenantIdentifierResolver and MultiTenantConnectionProvider. Spring Boot + Hibernate multi-tenancy integration.

Q: 6. What is a TenantContext?

A: ThreadLocal storing current tenant identifier. Set at request entry point (filter/interceptor from JWT/subdomain/header). Used by: JPA filter, Redis key prefix, S3 path prefix. Must be cleared after request (ThreadLocal memory leak prevention).

Q: 7. What is row-level security (RLS) for multi-tenancy?

A: PostgreSQL feature: CREATE POLICY on table, ALTER TABLE ENABLE ROW LEVEL SECURITY. Queries automatically filtered by tenant_id. Even if app forgets WHERE tenant_id =?, DB enforces isolation. Strongest multi-tenant isolation for shared schema.

Q: 8. What is tenant routing?

A: How to identify tenant per request: subdomain (tenant1.app.com), path prefix (/tenant1/api), JWT claim (tenant_id), HTTP header (X-Tenant-ID). Each has trade-offs: subdomain = SSL complexity; header = must be set correctly.

Q: 9. What is tenant isolation in cache?

A: Cache keys must include tenant ID: user:{tenantId}:{userId}. Without: tenant A's data served to tenant B. Redis namespacing: {tenantId}:* keys. Or separate Redis databases (select N) per tenant.

Q: 10. What is tenant isolation in file storage?

A: S3: separate bucket per tenant OR same bucket with prefix: s3://files/{tenantId}/path. IAM: bucket policy scoped to tenant prefix. Separate buckets: simpler IAM, bucket limit (100 default). Prefix: one bucket, IAM prefix policy.

Q: 11. What is a tenant-aware Kafka consumer?

A: Message must carry tenant context (header X-Tenant-ID). Consumer reads header, sets TenantContext before processing. With Spring Kafka: RecordInterceptor sets tenant context. Or separate topics per tenant (simpler isolation, more management).

Q: 12. What is schema migration in multi-tenancy?

A: Separate schema: Flyway per-schema migration. Separate DB: Flyway per-DB. Shared schema: single migration. Tooling: custom Flyway migration runner iterating all tenants. Risk: migration failure leaves tenants on different schema versions.

Q: 13. What is tenant onboarding?

A: Process of setting up new tenant: create schema/DB, run migrations, seed reference data, create admin user, configure IdP realm. Should be automated. Provision time: seconds to minutes depending on model.

Q: 14. What is a tiered multi-tenant architecture?

A: Different isolation levels for different customer tiers. Free/SMB: shared schema. Business: shared DB separate schema. Enterprise: dedicated DB or dedicated cluster. Balance between cost efficiency and isolation.

Q: 15. What is tenant rate limiting?

A: Each tenant gets own rate limit quota. Redis counter per tenant: INCR tenant:{id}:rps. Prevents one noisy tenant from affecting others. API Gateway: rate limit by API key (one per tenant) or JWT tenant claim.

33. Java Performance & Analysis [20 Questions]

Q: 1. What is JVM memory structure?

A: Heap: Young Gen (Eden + S0/S1) + Old Gen (Tenured). Non-Heap: Metaspace (class metadata), Code Cache (JIT compiled code), Direct Memory (NIO). Stack: per-thread, method frames. Tune: -Xms (initial heap), -Xmx (max heap), -XX:MaxMetaspaceSize.

Q: 2. What is GC and how to tune it?

A: Garbage Collector reclaims unreachable heap memory. G1GC (default Java 9+): regional, predictable pause. ZGC/Shenandoah: ultra-low pause (<10ms). Tune: -XX:+UseG1GC, -XX:MaxGCPauseMillis=200, -XX:G1HeapRegionSize. Monitor: GC logs with -Xlog:gc*.

Q: 3. What causes OutOfMemoryError?

A: Java heap space: heap exhausted. GC overhead limit exceeded: spending >98% time in GC. Metaspace: class metadata exhausted (dynamic classloading, proxies). Direct buffer: NIO direct memory exhausted. Fix: increase heap, fix memory leak, reduce class generation.

Q: 4. What is a memory leak in Java?

A: Objects reachable but never used again. Common causes: static collections growing unboundedly, ThreadLocal not removed, event listeners not unregistered, cache without eviction, inner class holding outer reference. Tools: Eclipse MAT, JVisualVM heap analysis.

Q: 5. What is jstack?

A: Java thread dump tool. jstack PID or kill -3 PID. Shows all threads with stack traces. Diagnose: deadlocks (jstack reports them), stuck threads (BLOCKED/WAITING), CPU spikes (RUNNABLE threads). Include in heap analysis.

Q: 6. What is JFR (Java Flight Recorder)?

A: Low-overhead profiling built into JVM (<1% overhead). Records: CPU profiles, GC events, locks, IO, exceptions, thread activity. JDK 11+: always-on possible. JMC (Java Mission Control) for analysis. Continuous recording in production.

Q: 7. What is async profiler?

A: Linux perf-based CPU/memory/lock profiler. Accurate CPU profiling (no safepoint bias). Generates flame graphs. Usage: ./profiler.sh -d 30 -f profile.html PID. Shows hot code paths. Supports allocation, lock, wall-clock modes.

Q: 8. What is a thread dump analysis?

A: Look for: BLOCKED threads (lock contention), all threads WAITING on same lock (thundering herd), deadlock cycle (jstack flags it). Count thread states: many RUNNABLE = CPU-bound, many BLOCKED = contention. Run 3 dumps 10s apart to confirm trend.

Q: 9. What is JMeter?

A: Load testing tool. Test Plans with Thread Groups (concurrent users), HTTP samplers, assertions, listeners (aggregate report, response time graph). Ramp-up: gradually increase load. Generate load: distributed testing with multiple JMeter servers.

Q: 10. What is Gatling?

A: Scala-based load testing tool. DSL for test scenarios. Better reports than JMeter. Async, non-blocking (handles more users per machine). Simulation code in version control. CI integration: fails on percentile thresholds.

Q: 11. What is a flame graph?

A: Visualization of CPU profiling. X-axis: time (width = CPU time). Y-axis: call stack (bottom = CPU, top = leaf). Wide base = hot method. Identify bottlenecks instantly. Generate with async profiler, perf, JFR.

Q: 12. What is p99 latency?

A: 99th percentile latency: 99% of requests complete within this time. Better metric than average (hides outliers). p50, p75, p90, p99, p999. Set SLOs on p99 (e.g., p99 < 200ms). Prometheus: histogram_quantile(0.99, ...).

Q: 13. What is the connection pool configuration?

A: HikariCP: maximumPoolSize = (core_count * 2) + effective_spindle_count. Typical: 10-20 for read-heavy, fewer for write-heavy. Too many: DB connection limit exceeded, context switching. Too few: requests queue waiting for connections.

Q: 14. What is virtual threads (Java 21)?

A: Lightweight JVM threads (not OS threads). Millions possible. Blocking operations (JDBC, HTTP) unmount carrier thread (no OS thread blocked). Throughput for blocking IO: massive improvement. Enable: Executors.newVirtualThreadPerTaskExecutor().

Q: 15. What is the Reactive paradigm?

A: Non-blocking, event-driven. Project Reactor (Spring WebFlux): Mono (0..1 item), Flux (0..N items). Operators: map, flatMap, filter, zip. Backpressure: subscriber requests only what it can handle. High-throughput IO-bound services.

Q: 16. What is DB connection pool exhaustion?

A: All connections in pool are in use. New requests queue or timeout. Causes: slow queries holding connections, missing connection release (try-with-resources), too many threads, connection leak. Fix: slow query optimization, connection timeout, pool size tuning.

Q: 17. What is a heap dump analysis with MAT?

A: Eclipse Memory Analyzer Tool. Load .hprof file. Leak Suspects: finds large object clusters. Dominator tree: shows biggest retained heaps. OQL: SQL-like queries on heap objects. Find: all instances of a class, objects with large retained size.

Q: 18. What is VisualVM?

A: Free JVM profiling tool bundled with JDK. Connect to local or remote JVM. Monitor: heap, threads, CPU, classes. CPU profiling: sample-based call trees. Heap dump analysis. Less powerful than JFR/async profiler but zero setup.

Q: 19. What is connection timeout vs socket timeout vs read timeout?

A: Connection timeout: time to establish TCP connection. Read timeout: time waiting for data after connection established. Socket timeout: general I/O timeout. In RestTemplate/Feign: set all three. Timeout too long: cascading delays. Too short: false failures.

Q: 20. What is Netty and why is it fast?

A: Non-blocking NIO event-driven networking framework. Event loop threads (one per CPU core) handle many connections. No thread-per-connection (eliminates context switching). Zero-copy: direct ByteBuf, file transfer. Used by: Spring WebFlux, gRPC, Kafka client, Elasticsearch.

34. Internationalization (i18n) [15 Questions]

Q: 1. What is the difference between i18n, l10n, and g11n?

A: i18n (Internationalization): design app to support multiple locales without code changes. l10n (Localization): adapt for specific locale (translations, date formats). g11n (Globalization): entire process of taking product to global markets.

Q: 2. What is a Locale in Java?

A: java.util.Locale identifies language + region: new Locale('en', 'US'), Locale.forLanguageTag('en-US'). Used for: date/number/currency formatting, message lookup, collation. Pass Locale to formatters, not store globally.

Q: 3. What is a ResourceBundle?

A: Loads locale-specific properties. messages.properties (default), messages_en_US.properties, messages_fr.properties. ResourceBundle.getBundle('messages', locale). Fallback chain: specific -> language -> default.

Q: 4. What is MessageSource in Spring?

A: Interface for resolving messages. ReloadableResourceBundleMessageSource: reads .properties files, supports hot-reload. messageSource.getMessage('key', args, locale). @Autowired everywhere in app. Default: classpath:/messages.

Q: 5. What is LocaleResolver in Spring?

A: Determines locale per request. AcceptHeaderLocaleResolver: from Accept-Language header. CookieLocaleResolver: from cookie. SessionLocaleResolver: from session. FixedLocaleResolver: fixed locale. Configure as bean.

Q: 6. What is MessageFormat?

A: Java class for parameterized messages. Pattern: 'Hello {0}, you have {1} messages'. MessageFormat.format(pattern, 'Alice', 5). Spring: messageSource.getMessage('key', new Object[]{'Alice', 5}, locale). Supports plurals and date formatting.

Q: 7. What is Unicode and UTF-8?

A: Unicode: universal character set for all scripts. UTF-8: variable-width encoding for Unicode (ASCII chars = 1 byte, others 2-4 bytes). Always use UTF-8 in Java: charset everywhere, properties files with UTF-8, DB charset=utf8mb4.

Q: 8. What is RTL (Right-to-Left) support?

A: Arabic, Hebrew, Persian read right-to-left. HTML: dir='rtl' on <html> or element. CSS: logical properties (margin-inline-start instead of margin-left). JavaScript: Intl.Bidi. Test with RTL pseudo-locale.

Q: 9. What is date/time formatting for i18n?

A: Use Locale-aware formatters. Java: DateTimeFormatter.ofLocalizedDateTime(MEDIUM).withLocale(locale). Don't hardcode format strings. Time zones: store UTC, display in user's time zone. Avoid ambiguous formats (01/02/03 — is it Jan 2? Feb 1? Feb 3?).

Q: 10. What is number formatting for i18n?

A: Different locales use different number formats. en-US: 1,234.56. de-DE: 1.234,56. fr-FR: 1 234,56. Java: NumberFormat.getNumberInstance(locale).format(1234.56). Never concatenate number + string without locale-aware formatting.

Q: 11. What is currency formatting?

A: Locale determines currency symbol and format. Java: NumberFormat.getCurrencyInstance(locale).format(amount). Store amounts in smallest unit (cents) to avoid floating point. java.util.Currency: ISO 4217 codes.

Q: 12. What is pluralization in i18n?

A: Different plural forms per language (English: 1 item / 2 items; Russian: 1 ■■■■■■■ / 2 ■■■■■■■■ / 5 ■■■■■■■■■■). ICU MessageFormat supports plural: {count, plural, one {# item} other {# items}}. Simple {0} substitution insufficient for plurals.

Q: 13. What is pseudo-localization?

A: Testing technique: replace all strings with visually similar but distinct characters (e.g., 'Submit' -> '[Sübmít]'). Reveals: hardcoded strings, layout breaks (strings expand in translation), RTL issues. Run in CI before localization.

Q: 14. What is character encoding in database?

A: MySQL: charset=utf8mb4 (NOT utf8 which is 3-byte — can't store emoji). collation=utf8mb4_unicode_ci for case-insensitive comparisons. PostgreSQL: UTF8 by default. Always specify connection charset: spring.datasource.url=jdbc:mysql://...?characterEncoding=UTF-8.

Q: 15. What is ICU4J?

A: IBM's internationalization library for Java. More complete than Java standard library. Supports: full Unicode, CLDR data, plural forms, collation, date/number formatting for all locales, bidirectional text. Used when Java Locale/MessageFormat is insufficient.

35. HashMap Internal Working [15 Questions]

Q: 1. What is the internal structure of HashMap?

A: Array of Node buckets (default 16). Each Node: key, value, hash, next (linked list for collision). Java 8+: bucket converts to balanced tree (TreeNode) when size >= 8. hash(key) determines bucket: (n-1) & hash.

Q: 2. What is the HashMap put() algorithm?

A: 1. If null key: bucket 0. 2. Compute hash. 3. Find bucket: (n-1) & hash. 4. If empty: insert Node. 5. If key exists: update value. 6. If tree: use tree insertion. 7. If list: append (check for duplicate key). 8. If size exceeds threshold: resize.

Q: 3. What is HashMap hashing?

A: hash(key) = (h = key.hashCode()) ^ (h >>> 16). XOR with upper 16 bits reduces collisions. bucket = hash & (capacity - 1). Works only because capacity is always power of 2. System.identityHashCode() if hashCode() not overridden.

Q: 4. What is treeification in Java 8 HashMap?

A: When bucket list length >= TREEIFY_THRESHOLD (8) AND table size >= MIN_TREEIFY_CAPACITY (64): convert bucket list to Red-Black Tree. Tree: O(log n) vs list O(n). Untreeify when shrinks below UNTREEIFY_THRESHOLD (6).

Q: 5. What is HashMap resize?

A: When size > capacity * loadFactor (default 0.75): resize to 2x capacity. Rehash all entries: new bucket = old bucket OR old bucket + old capacity (due to power of 2). Java 8: no re-computation of hash, just check high bit. Expensive O(n) operation.

Q: 6. What is the load factor in HashMap?

A: Threshold for resize = capacity * loadFactor. Default 0.75. Low load factor: fewer collisions, more memory, more resizes. High load factor: more collisions, less memory. 0.75 is empirical sweet spot (time-space tradeoff).

Q: 7. What is HashMap initial capacity?

A: Default 16 (power of 2). If you know expected size: initialCapacity = expectedSize / loadFactor + 1. Avoid unnecessary resizes. new HashMap<>(64) for 40 entries. Guava: Maps.newHashMapWithExpectedSize(n) computes this automatically.

Q: 8. What is the difference between HashMap and Hashtable?

A: Hashtable: synchronized (thread-safe but slow), no null keys/values, legacy class. HashMap: not synchronized, allows null key (one) and null values, faster. For concurrency: use ConcurrentHashMap, not Hashtable.

Q: 9. What is ConcurrentHashMap?

A: Thread-safe HashMap without full synchronization. Java 8+: only locks individual bucket (synchronized on first node of bucket during write). Reads: no locks (volatile reads). putIfAbsent, computeIfAbsent are atomic. Better than synchronized HashMap.

Q: 10. What is HashMap vs LinkedHashMap vs TreeMap?

A: HashMap: O(1) get/put, no order. LinkedHashMap: maintains insertion order (doubly-linked list connecting entries). TreeMap: sorted by key (Red-Black Tree), O(log n). Choose by access pattern: HashMap (performance), LinkedHashMap (insertion order), TreeMap (sorted).

Q: 11. Why must you override hashCode when overriding equals?

A: Contract: equal objects must have equal hashCodes. If break this: equal objects may go to different buckets, get() returns null even when key is 'present'. Both must use same fields. Java: Objects.hash(field1, field2) for consistent implementation.

Q: 12. What is a poor hashCode causing performance issues?

A: If hashCode() returns constant: all keys in same bucket, HashMap degrades to O(n) linked list / O(log n) tree. Test: System.identityHashCode distribution. Monitor: HashMap.entrySet().stream().collect(groupingBy(e->e.hashCode()&15, counting())).

Q: 13. What is consistent hashing vs HashMap?

A: Java HashMap: single-machine, keys distributed across fixed-size array by hash. Consistent hashing: distributed system, keys distributed across ring of servers. When server added/removed: only K/N keys need to move (not all). Used in: distributed caches, load balancers.

Q: 14. What is WeakHashMap?

A: Keys held via weak references. When key has no strong references: GC can collect key, entry removed from map. Used for: caches where key lifecycle controls entry lifecycle. Warning: entries can disappear at any time.

Q: 15. What is the EnumMap?

A: HashMap implementation for enum keys. Internally array indexed by enum ordinal. Faster and less memory than HashMap. Use whenever keys are enum values: private final Map<Status, Handler> handlers = new EnumMap<>(Status.class).